



Big Data

dr inż. Paweł Drzymała
pawel.drzymala@p.lodz.pl

Institute of Mechatronics and Information Systems



Lodz University of Technology, ul. Stefanowskiego 18/22, 90-924 Łódź

Learning outcomes :

Student will:

https://programy.p.lodz.pl/



1. have knowledge of advanced techniques for processing large, variable and various data sets.
2. have knowledge of engines for large-scale data analysis such as Big Data,
3. be able to use languages to perform complex analysis of large data sets,
4. be able to use various techniques to access data, extract it, transform it and analyze it

Laboratory:

Subject content:

The laboratory consists of several projects illustrating the use of individual elements of Big Data, completed with data analysis and processing. The scope of material includes:

1. Introduction to Big Data (we will talk about cluster processing systems based on the Hadoop environment)
2. Introduction and work in the Hadoop and Hadoop Distributed File System environment,
3. Big data processing platforms: Apache Spark, Apache Pig, Apache Hive (the use of these platforms in data processing (ETL))
4. Working with the Apache Spark engine for large-scale data processing and its components:
- **Spark Core,**
- **SparkSQL,**
- **Spark Streaming,**
- **Mllib machine learning library,**

5. Data stream processing – exercises
6. Developing predictive models
7. Developing recommendation models

Assessment criteria:

The final grade will consist of two partial grades:
1. based on a practical colloquium (will take place during 7 or 8 laboratory) - 50%
2. based on a group project (in a group of 2-3 persons) - 50%

The final grade is the arithmetic average of the all grades



Passing the course:

- The first assessment will take place during the laboratory - in the 8th week of laboratory

1.

… on November

- Number of exercises to complete:
  - Number of exercises to complete - 6-10 exercises

+

2.

+ Project:

The project can be discussed → during the final two laboratory

The final grade is the arithmetic average of the this grades

Requirements for the project:



(Project description):

The project should involve the implementation of one of the following models using Apache Spark:

1. model for prediction tasks

or

2. model for classification tasks

or

3. ~~model for recommendation tasks~~

REMEMBER: - The project must be developed using the Apache Spark engine and platform for large-scale data processing (including the use of at least one of the libraries:

- Spark Streaming and/or

Spark Streaming and/or Mllib machine learning).

Project description:



Requirements:

1) Description of the "Project Goal" :

The project should:

a.) adopt and formulate theses for the data processing process in order to obtain information and conclusions based on them,

b.) depending on the problems posed, assume specific use of statistical and other methods and ways of using the given tools for this purpose.

2) Selection of data for the intended purpose.

a.) for the aims of the project, prepare a description of the selection of data for the adopted topic, their native structure and formats,

b.) describe:
- the quantitative nature of raw data,
- their method of creation (i.e. primary data, secondary data),
- the dynamics of data changes (i.e. - continuous data registration,
  - registration at time intervals),
- sources of origin.

c.) describe the process of obtaining data in the form of data streams (for real-time events) that are made available via Apache Kafka brokers

Project description:



Requirements:

3) Data Extraction and Analysis:

a.) discuss:
- processes of organizing, transforming and modeling data in order to obtain useful information and conclusions,
- techniques and tools used to find regularities in the data that are hidden to humans,

b.) develop the required scripts → necessary to perform the tasks of:
- extraction,
- transformation and
- loading

of data into formats with a defined structure using Apache Spark.

c. discuss the machine learning process using Apache Spark:
- the process of building a model for the prediction …

….or recommendation system

4) Provide comments and final conclusions following the analysis and how they can be useful in supporting the decision-making process?

ATTENTION: →We describe everything in the report

… and …

… add sources.



NOTE - for a very good grade, the project must be submitted before the session begins.

Project description:



Requirements:

Once again

ATTENTION: →We describe everything in the report

… and …

… add sources. ☺

NOTE - for a very good grade, the project must be submitted before the session begins..

Sources of information :

Electronic: free sources also published in PDF format

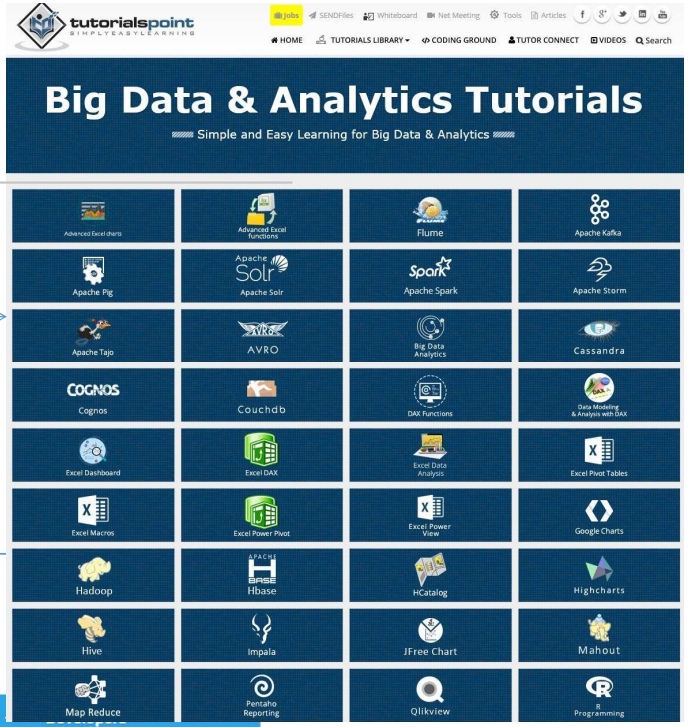
https://www.tutorialspoint.com/big_data_tutorials.htm

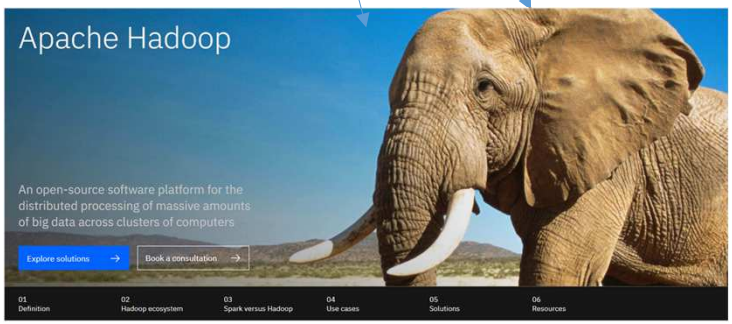
https://hadoop.apache.org/

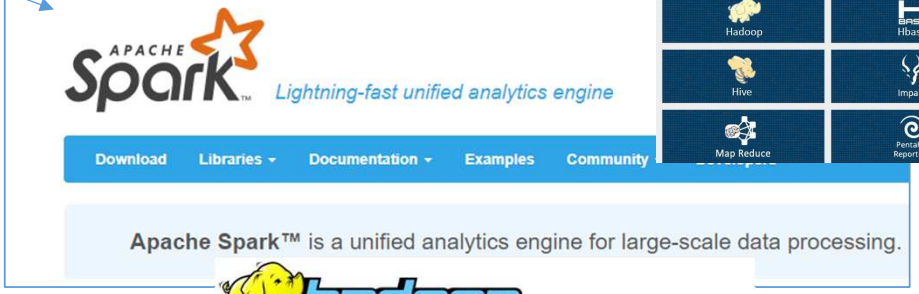
https://spark.apache.org/

https://developer.ibm.com/hadoop/

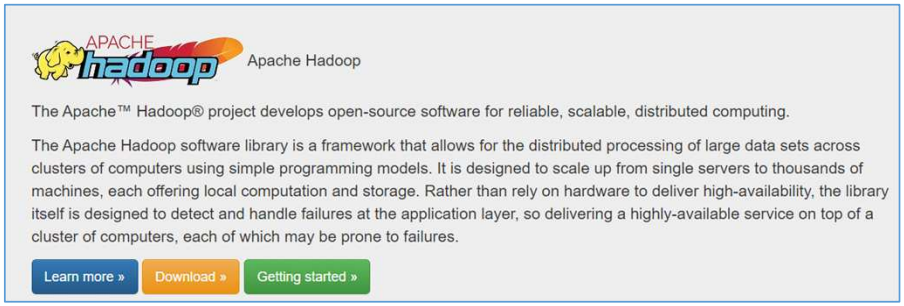
https://hadoop.apache.org/docs/r1.2.1/hdfs_design.html









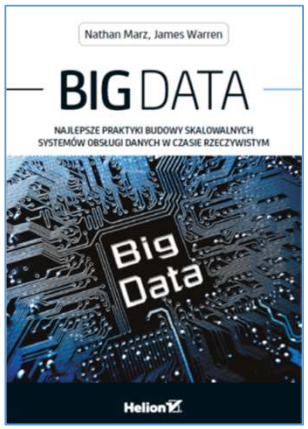


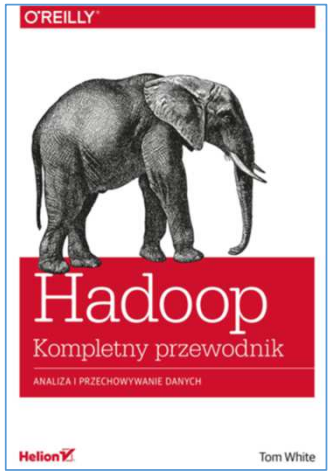


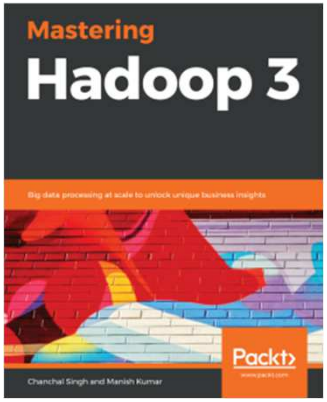
Inne...

Books in electronic and paper form:





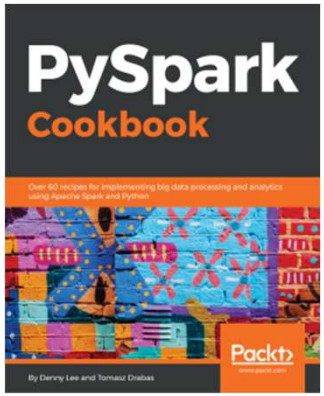




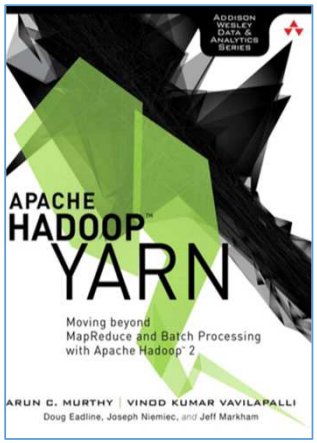
https://www.packtpub.com/product/mastering-hadoop-3/9781788620444

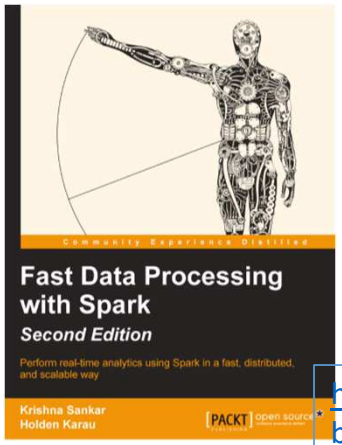


https://www.packtpub.com/product/apache-spark-3-for-data-engineering-and-analytics-with-python-video/9781803244303



https://www.packtpub.com/product/pyspark-cookbook/9781788835367

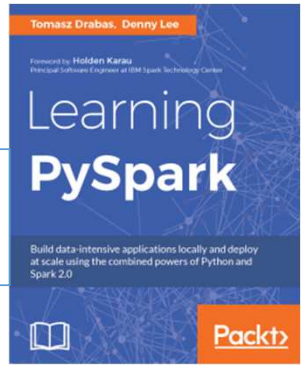






https://www.packtpub.com/product/frank-kane-s-taming-big-data-with-apache-spark-and-python/9781787287945

https://www.packtpub.com/product/learning-pyspark/9781786463708



Sample datasets can be found:



where are the interesting data sets?

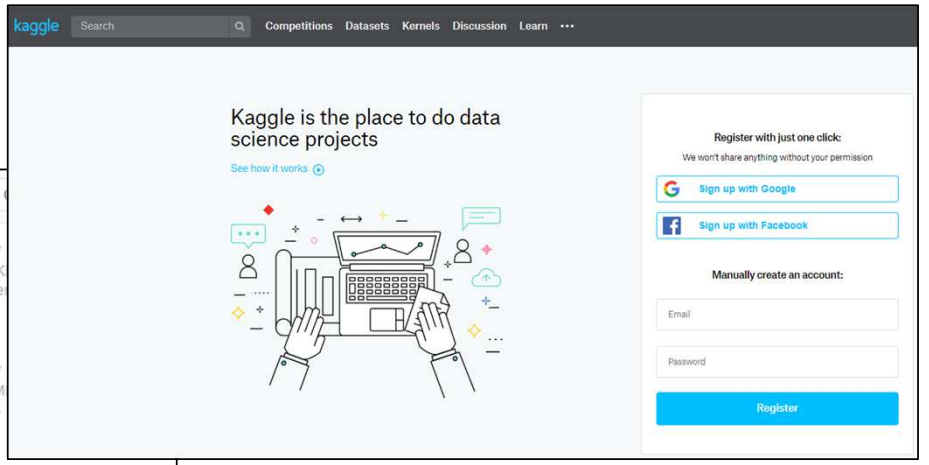
Stites with datasets:

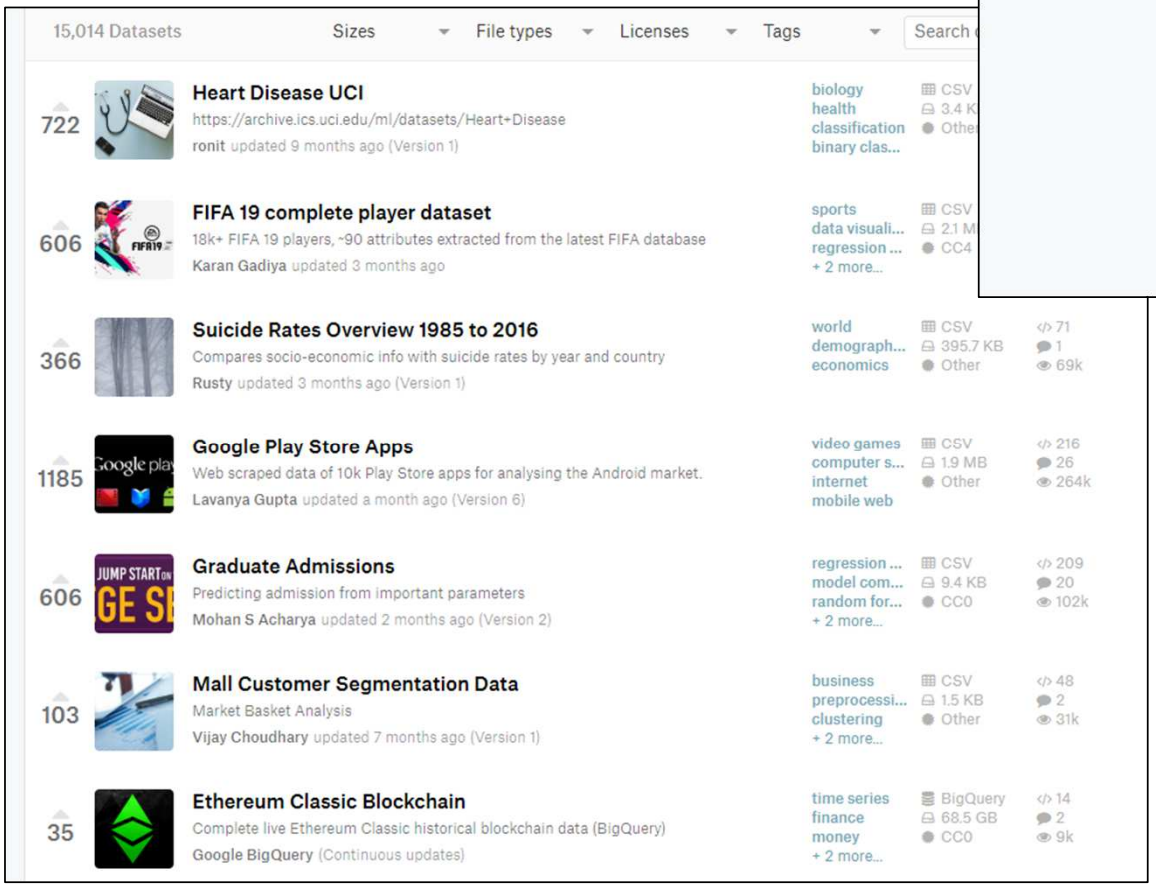
- https://archive.ics.uci.edu/ml/index.php
- https://www.kaggle.com
- http://stat-computing.org/

Sample datasets can be found:

Kaggle is the place to do data science projects:

https://www.kaggle.com

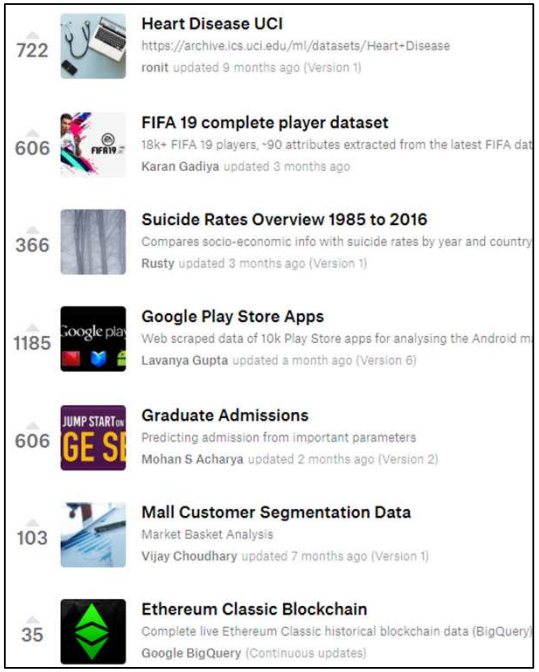


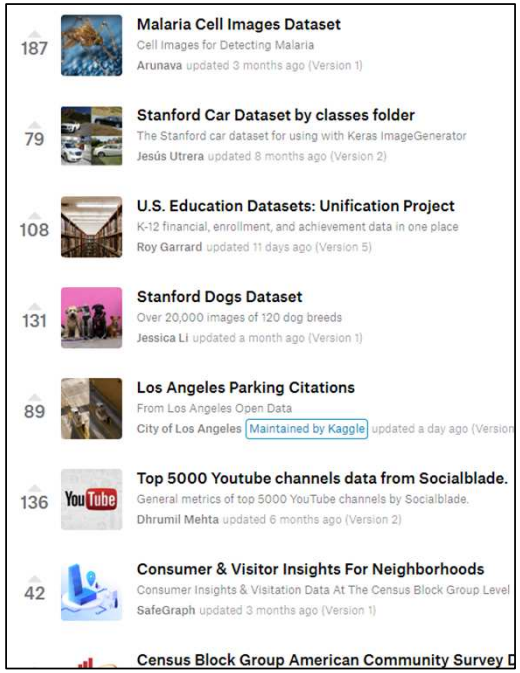


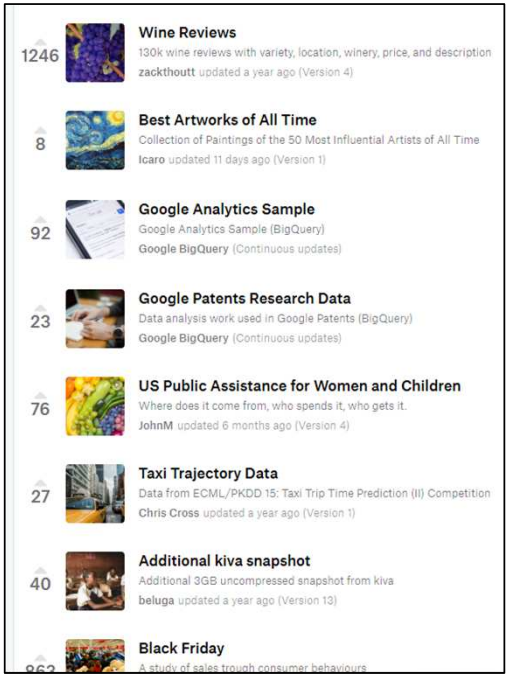
Sample datasets can be found:



https://www.kaggle.com → cd....







Sample datasets can be found:



Center for Machine Learning and Intelligent Systems:

https://archive.ics.uci.edu/ml/index.php

UCI

Machine Learning Repository

Center for Machine Learning and Intelligent Systems

About Citation Policy Donate a Data Set Contact

Search

Repository Web

View ALL Data Sets

Browse Through: **468** Data Sets

Table View List View

Default Task
Classification (349)
Regression (96)
Clustering (84)
Other (55)

Attribute Type
Categorical (38)
Numerical (306)
Mixed (55)

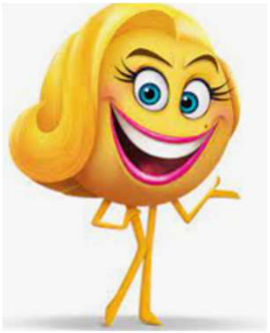
Data Type
Multivariate (356)
Univariate (23)
Sequential (47)
Time-Series (91)
Text (53)
Domain-Theory (23)
Other (21)

Area
Life Sciences (107)
Physical Sciences (49)
CS / Engineering (169)
Social Sciences (26)
Business (29)
Game (10)
Other (73)

Attributes
Less than 10 (113)
10 to 100 (210)
Greater than 100 (84)

Instances

Name	Data Types	Default Task	Attribute Types	# Instances	# Attributes	Year
Abalone	Multivariate	Classification	Categorical, Integer, Real	4177	8	1995
Adult	Multivariate	Classification	Categorical, Integer	48842	14	1996
Annealing	Multivariate	Classification	Categorical, Integer, Real	798	38	
Anonymous Microsoft Web Data		Recommender-Systems	Categorical	37711	294	1998
Arrhythmia	Multivariate	Classification	Categorical, Integer, Real	452	279	1998
Artificial Characters	Multivariate	Classification	Categorical, Integer, Real	6000	7	1992
Audiology (Original)	Multivariate	Classification	Categorical	226		1987
Audiology (Standardized)	Multivariate	Classification	Categorical	226	69	1992



… let's get started →



let's get started with an …

1. **… introduction to distributed computing and efficient data analysis**

big data

after this introduction …

… we should be able to:

- **… - be able to explain the need to use a Big Data solution**
- **… - describe what Big Data is.**
- **- list and use "open source" applications belonging to Big Data solutions and their purposes**

Processing Scalability:

Example:

Did you know that:

As much as 87 percent of all credit card transactions and payments worth almost $8 trillion annually are processed through these systems.

Annually, the platform manages 29 billion ATM (cash machine) transactions with an **average daily** value of $5 billion.

mainframe systems → handle flights of 4 billion passengers annually.



Enterprise Server Edition (Data Partitioning Feature)

mainframe systems

Small and medium companies



Enterprise Server Edition



Workgrup Server Edition



Db2 Community Edition



Personal Edition



Everyplace Edition

... however, in selected business processes - the use of this or similar platforms is not enough…

… and the cost of scaling is too high.

1. What is the concept of Big Data?

Terms such as:

- **kilobytes,**
- **megabytes**
- **gigabytes**

→ **… are widely known.**

When it comes to big data → **we are talking about quantities that are not part of our everyday considerations.**

The term terabyte has been added to our discussions over the last few years.

… but for most people the terms:

- **petabyte,**
- **exabyte**
- **zettabyte**
- **and yottabyte**

they sound like unknown words…

How many zeros does it have?

… however, these terms are necessary when working with large data sets.

International System of Units (SI)			Binary Usage (deprecated)
kilobyte	KB	10^3	2^{10}
megabyte	MB	10^6	2^{20}
gigabyte	GB	10^9	2^{30}
terabyte	TB	10^{12}	2^{40}
petabyte	PB	10^{15}	2^{50}
exabyte	EB	10^{18}	2^{60}
zettabyte	ZB	10^{21}	2^{70}
yottabyte	YB	10^{24}	2^{80}

… other main numbers (on the margin of considerations)

Cardinal numerals of powers of one thousand:

UWAGA !!

Such numerals are created in different languages of the world according to different rules.

For example:

Numeral	Value on a long scale (np. Polska)	Value on a short scale (np. USA)	Etymology
tysiąc	10^3	10^3	Słowo starosłowiańskie - prawdopodobnie od *tū-t-ḱm̥t-i- *potężna setka*[5]
milion	10^6	10^6	*Milioun* jest starofrancuskim słowem, pochodzącym od starowłoskiego *milione*, będącym wzmocnioną wersją słowa *mille*, tysiąc[6].
miliard	10^9	nie ma	Wyraz trafił do polszczyzny z średniowiecznego francuskiego, gdzie był zapisywany *miliart*. Słowo to jest przekształconą nazwą miliona ze zmienioną końcówką[7]. We francuskim pojawiło się w XVI wieku[8]
bilion	$10^{12}=10^{6\cdot\mathbf{2}}$	$10^9=10^{3\cdot\mathbf{2}+3}$	Łaciński przedrostek *bi-* oznacza *dwu-*. Dodana końcówka z *milion*
biliard	$10^{15}=10^{6\cdot\mathbf{2}+3}$	nie ma	Łaciński przedrostek *bi-* oznacza *dwu-*. Dodana końcówka z *miliard*
trylion	$10^{18}=10^{6\cdot\mathbf{3}}$	$10^{12}=10^{3\cdot\mathbf{3}+3}$	Łaciński przedrostek *tri-* oznacza *trój-*. Dodana końcówka z *milion*

… money and data have a lot in common (on the margin of considerations)

Question: **… is a billion exDeutschMarks or a billion dollars a lot?**

… money and data have a lot in common (on the margin of considerations)

Question: **… is a billion exDeutschMarks or a billion dollars a lot?**

Answer: → **…it depends…** → **…from what?**

from FAITH!!! (od WIARY (łac. *fides* – wiara)

VIDE next:

Fiat money (Fiduciary currency)

(łac. fides – wiara) – is a type of currency that is not backed by a commodity, such as gold or silver. It is typically designated by the issuing government to be legal tender.

nbportal.pl [access 2015-12-15].



100 bilionów dolarów Zimbabwe



Banknot 100 bilionów marek, (15 stycznia 1924)

Question :

What does this have to do …



… with the term Big Data?

?

… answer :

Question :

How many trillion dollars have been "released" since 2007 under the QE1, QE2 and QE3 programs to save the US economy?

Answer : → **3.5 trillion!!!** → **to illustrate:**

1 trillion "hundred dollar bills" printed and stacked on top of each other

would give a stack of paper **from Earth to the Moon**

Conclusion :

In the case of data, just like in the case of money, it is not only the quantity that counts, but also the value, place, access time, etc.!!!



Big data:

is a term referring to the strategy/technology of processing large data sets

… and what size of data sets…

… are large and what … are small?

???



There is no clear answer to this question

??? 

… because … it depends …

… what this data is and how we process it

???

???



therefore …

the term "Big Data" is a bit misleading.

many people are not a fan of this term…

… - because the term suggests that one piece of data is small (it may be) or that this particular type of data is large (it may or may not be the case).

Big data



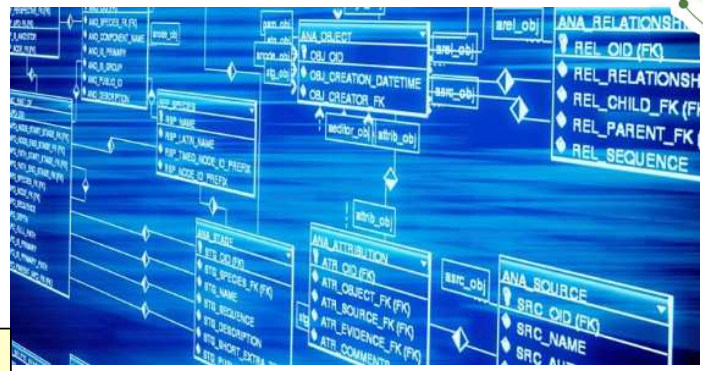
The term "Big Data„ ... is more ... than just a large database...

How big are the data sets?(Big data)?

- **over 90% of data → has increased over the last 2 years**

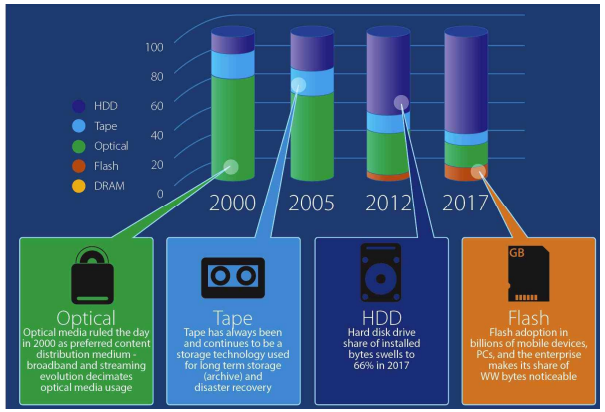
- **we currently produce as much data every 2 days as humanity produced from the beginning of civilization to the beginning of the 21st century**

- **by the end of 2023, the amount of stored digital data increased to 90.4 Zeta-bytes.**



-**52 PB (petabyte 10^{15}) - this is a data warehouse of 30% of ebay's customers or all collections of literature collected from around the world from the entire history of mankind or a movie in HD lasting 13 years**





- **in 2017, as much as 2/3 of the world's data is stored on rotating hard drives.**

https://www.backupacademy.pl/jak-bedzie-wygladal-swiat-storage-w-2017-roku/





what is the reason for collecting such large data sets?

data sets – in numbers:



what is the reason for collecting such large data sets?



The answer is: →



storage is getting cheaper

3 mln $

storage cost of 1GB in 1980

0,08 $

storage cost of 1GB in 2010

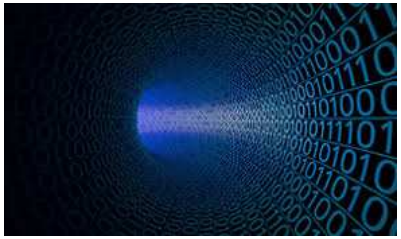


so we now store files on private network drives

data sets – in numbers:





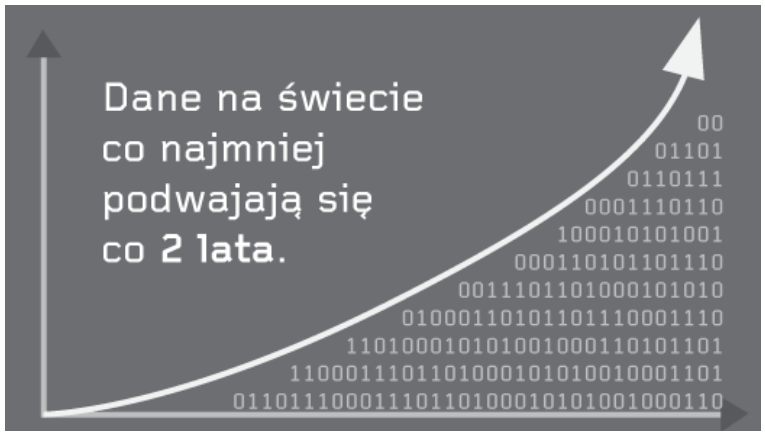


Did you know that:



2.5 peta bytes (according to IBM) → is the capacity of the human brain

13 peta bytes (according to IBM) → a value that can be downloaded via the Internet in two minutes if every American gets 300MB on their computer at the same time



https://www.backupacademy.pl/eksplozja-big-data-i-ewolucja-nosnikow-danych/



Where does the data …

… come from?



Where does the data come from?

in one minute:

- **277,000** tweets are created
- **2 million queries** on Google
- **72 hours** of new videos on YT
- Walmart processes almost **17 thousand** transaction **(2.5PB per day)**
- we write over **100 million** new e-mails
- we download **32 thousand** mobile applications

https://www.backupacademy.pl/eksplozja-big-data-i-ewolucja-nosnikow-danych/





In **2005, 1.3 billion RFID** readers were used, **currently 200 billion** are in use





The data varies in nature





that's not all - data are generated by industry, science, astronomy, etc..

Big data:

… is a term relating to:

- large,
- variable,
- various

data sets

… but for whom processing and analysis is difficult but at the same time valuable because it can lead to gaining new knowledge.

https://pl.wikipedia.org/wiki/Big_data

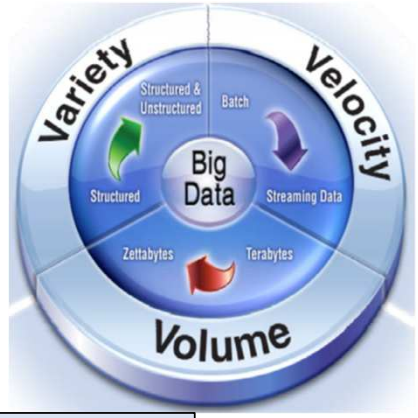


Big data is applicable wherever a large amount of digital data coexists → due to the need to acquire new information or knowledge.

In 2001, META Group published a report that included big data in the **3V** model:

- **Volume** → how much data is there?
- **Velocity** → how quickly is the data being created, moved, or accessed?
- **Variety** → how many different types of sources are there?

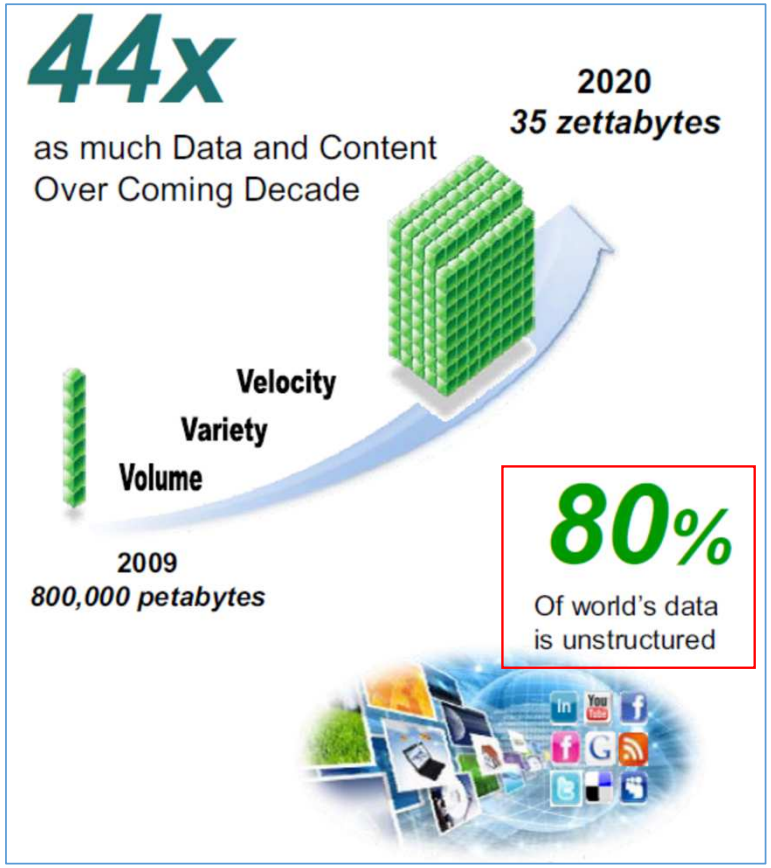
model **4V**



This model has been expanded with another component:

- **Value** → can we trust the data? … can this data produce a **meaningful** return on investment?
- **Validity** → is the data accurate and correct?
- **Visualization** → how quickly is the data being created, moved, or accessed?

Big data



!!!

1 in 3 — business leaders continue to make decisions based on information they often don't trust or have

1 in 2 — managers say they do not have access to information needed to perform their duties

83% — managers treat "Business Intelligence" as a plan to increase the competitiveness of their own organization

60% — CEOs say they need to understand information faster to make business decisions quickly

Data volumes can be divided into two types:





Data in motion → information streams

- Twitter / Facebook
- Stock data
- Sensors – e.g. for predicting the weather or measuring the vital signs of newborns

Data at rest → oceans of information

- Data that has already been submitted
  - i.e. → Web logs, emails, social media
- Unstructured documents:
  - i.e → forms
- Structured data from various systems

Types of data



We have three types of data:

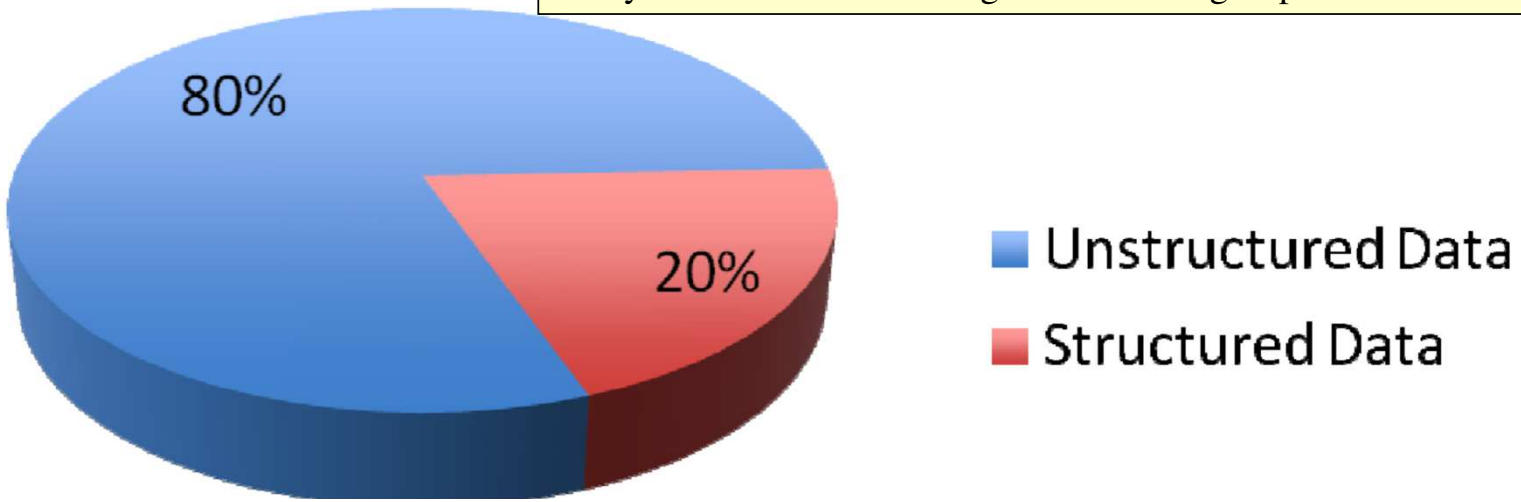
- Relational data (data with a defined structure)
- XML and JSON data (semi-structured data)
- Unstructured data (Word, PDF, Text, Media Logs)

About 80% of the world's data is without a clearly defined structure

This may be data collected previously, but cannot be processed in a standard way.

A new type of technology has been developed → to meet the challenges of Big Data.

Many of these new technologies have been grouped under the term NoSQL.



The Challenges of Big Data



The challenges of big data are as follows:

- Data capture
- Duration
- Searching through them
- Splitting them up
- Transfer
- Analysis
- Presentation

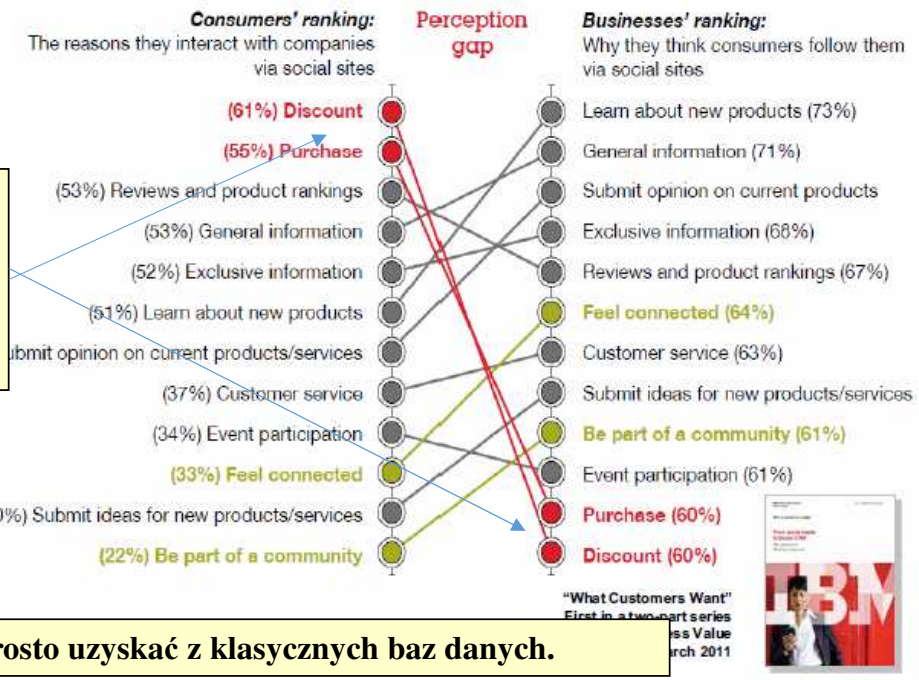
Przykład: Różnicy postrzegania mediów społecznościowych

Badanie przeprowadzone przez IBM CEO ujawniło, że dla wielu firm „media społecznościowe” stanowią podstawę strategii zbliżania się do oczekiwań klientów.

Jednak osobne badanie wykazało, że firmy często nie rozumieją rodzaju reklamy i jej celu, którego konsumenci naprawdę chcą od firm.

Przykładowo:

dwa najlepsze wybory dokonane przez konsumentów nie znalazły się na liście 10 najważniejszych celów prognoz firm dot. oczekiwań klientów.



Czy nie byłoby wspaniale, gdyby firmy nie musiały zgadywać?

Analiza „Big Data” pozwala odkrywać wiedzę, której nie da się prosto uzyskać z klasycznych baz danych.

Czy istnieje powiązanie pomiędzy rodzajem kupowanych przez internet produktów a porą roku oraz panująca pogodą?

Czy z analizy rachunków, można przewidzieć która z klientek banku spodziewa się dziecka?

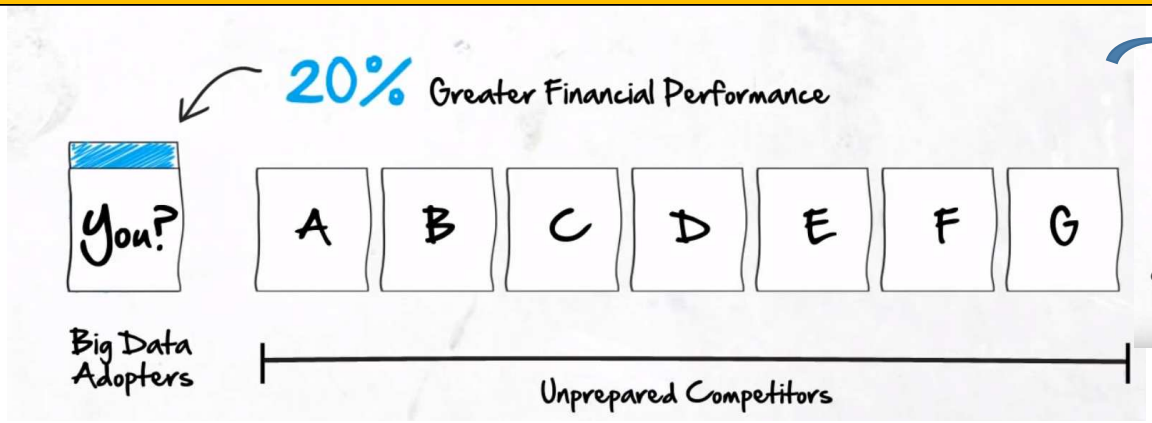
Czy smartwatch na podstawie analizy naszych zachowań zaobserwuje coś niepokojącego odnośnie stanu naszego zdrowia?

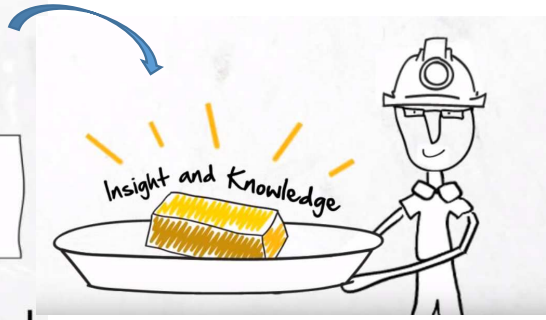
Big data - is it really worth?



Gartner research found → that only 10-15% of enterprises are fully exploit the benefits of big data.

… But those companies that succeed will gain a 20% advantage over unprepared competitors in key business metrics.





wg. Intel Corp.- https://www.youtube.com/watch?v=9XHAEZkOKY8

Big data is like a mountain from which you try to extract changing rules of the game that crush the competition

the gold is insight and knowledge that will change the effectiveness of the company's business



Today - DATA → is what oil was in the 19th century

– this statement was adopted by the World Economic Forum in 2011





What is Hadoop?

http://hadoop.apache.org/

https://github.com/big-data-europe/docker-hadoop

https://www.tutorialspoint.com/big_data_tutorials.htm

https://www.cloudera.com/tutorials.html

https://docs.microsoft.com/pl-pl/azure/hdinsight/hadoop/apache-hadoop-introduction (2022)

What is Hadoop?



platform for storage and processing

Structured data

Unstructured data

Semi-structured data

Hadoop - is a data processing platform:

hadoop

Analysis









Business decisions

Hadoop – it enables the storage and analysis of all types of data.

... in order to make the right decisions on their basis.

Structured data- Data that can be stored and processed in a fixed format, aka schema

Semi-structured - Data that does not have a formal structure of a data model (XML, JSON)

What is Hadoop?



- **is an "open source" platform**
- **Written mainly in Java**
- **Optimized to support:**

developed by: Doug Cutting

under the care of the Apache Foundation



The Apache Software Foundation



- huge amounts of data through parallel processing

- **Massive parallel processing is performed with great efficiency.**

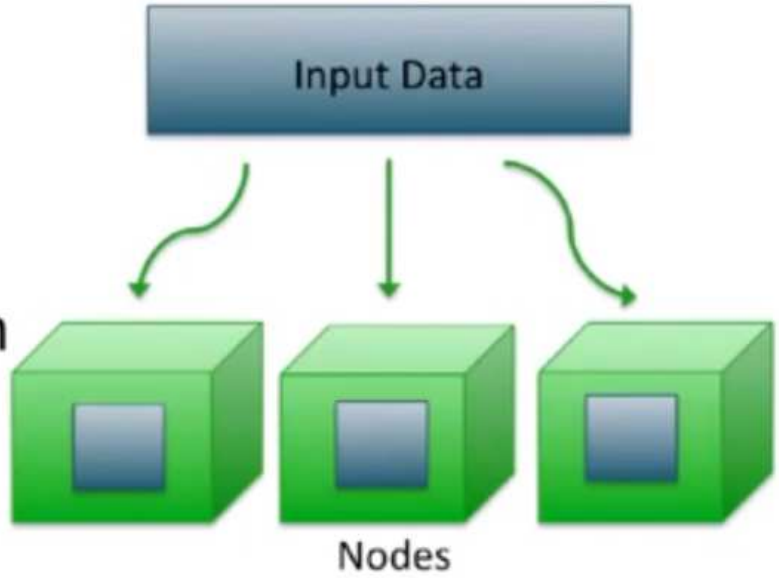
- **Enables data replication across the cluster.**

- **Hadoop → is not for transaction processing** **!!!**

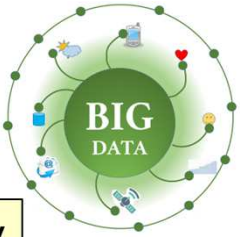
What is Hadoop?



- Open source
- Distributed data replication
- Commodity hardware
- Data and analysis co-location
- Scalability
- Reliable error handling



What is Hadoop?



Hadoop

… allows thousands of nodes to work together and process petabytes of data in a highly parallel and cost-effective way



- "Hadoop" node = CPU + HDDs
- computer nodes can be connected into clusters
- if necessary, new nodes can be added without changes:
  - data format
  - data loaded into the cluster
  - written tasks

Hadoop

is not intended for all types of work:

- not for processing based on transaction rules
- not good when the work cannot be parallelized
- not suitable for low latency data access
- not suitable for processing many small files
- not suitable for intensive processing with small amounts of data

Examples of Hadoop use:

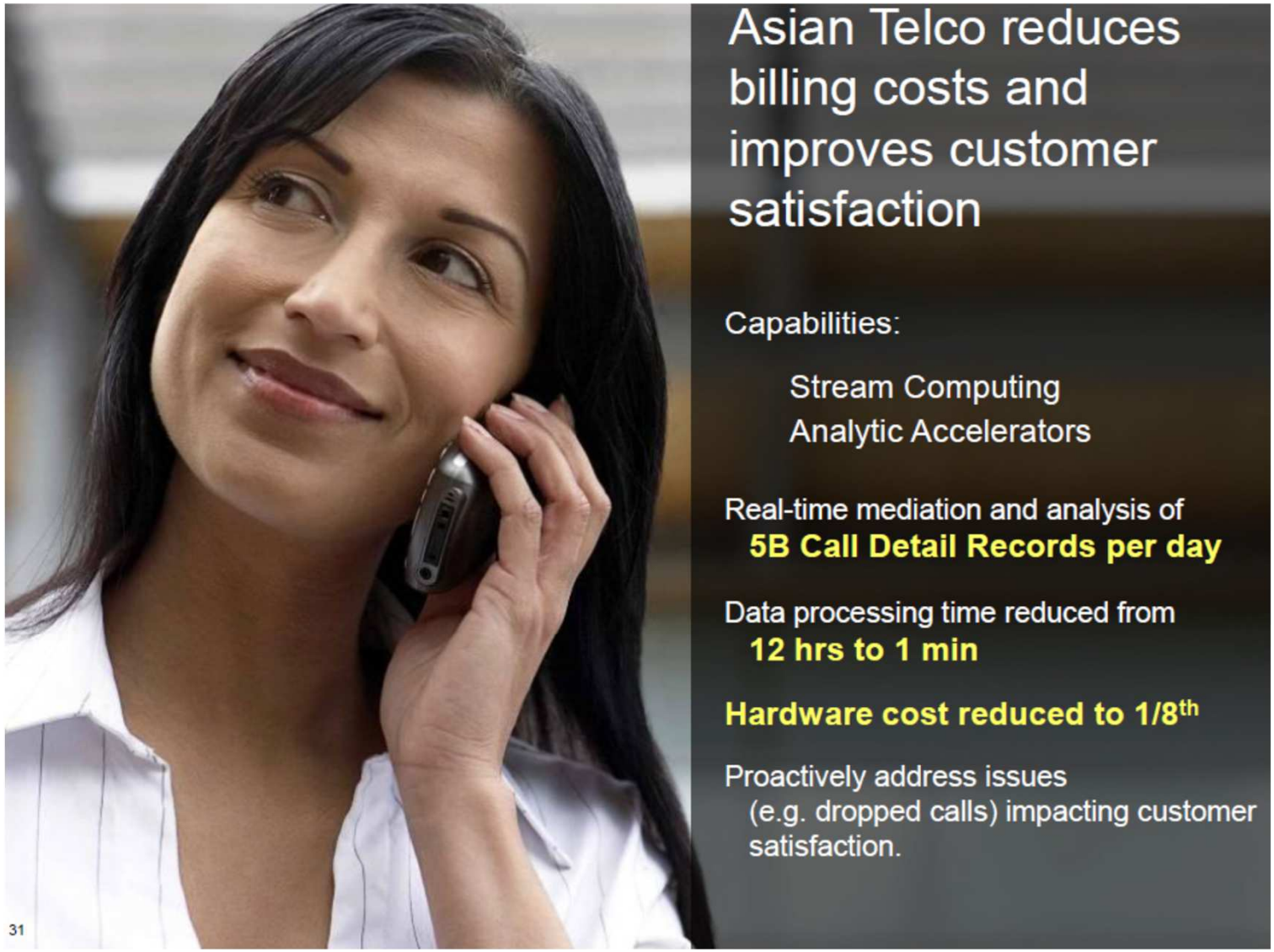




Examples of Hadoop use:

- Telecommunication
- Security and terrorist threats
- Health care – detection of disease outbreaks
  - Itd..

Przykłady zastosowania Hadoop-a







What is Hadoop Architecture?

- HDFS
- YARN → tools that cooperate with each other

Enterprise Hadoop Architecture (YARN)



The Ecosystem → within a cluster, various frameworks can cooperate via YARN

In-Memory	Scripts	SQL	operacje Batch	...	Stream	full-text search	NoSQL
Spark	Pig	Hive	Map Reduce		Storm	Solr	Hbase

Yarn

(Yet Another Resource Negotiator)

(system operowania na danych)

HDFS

(Hadoop Distributed File System)

(data storage management system)

The core of the Hadoop architecture



The core Hadoop architecture contains two main components:

A - Hadoop Distributed File System (HDFS)

~~B - MapReduce Engine~~

+ B - YARN (Hadoop v2)

Yet Another Resource Negotiator

YARN - allows you to use a set of tools that cooperate with each other.

Agenda

- Terminology review
- HDFS
- MapReduce
- Type of nodes
- Topology awareness

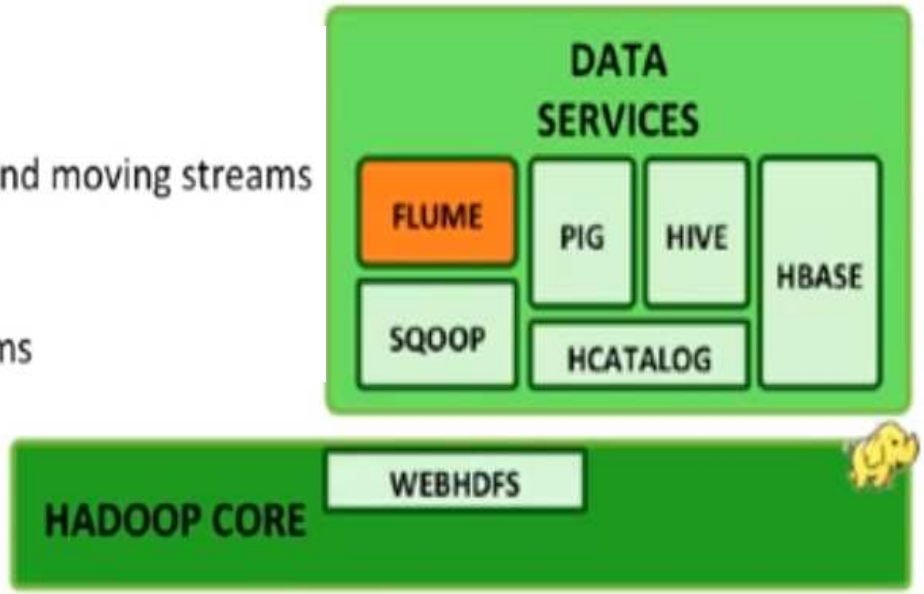
Enterprise Hadoop Architecture(YARN)



Apache Flume

Apache Flume: Store Log Files & Events

- Distributed service for efficiently collecting, aggregating, and moving streams of log data into HDFS
- Streaming capability with failover and recovery mechanisms



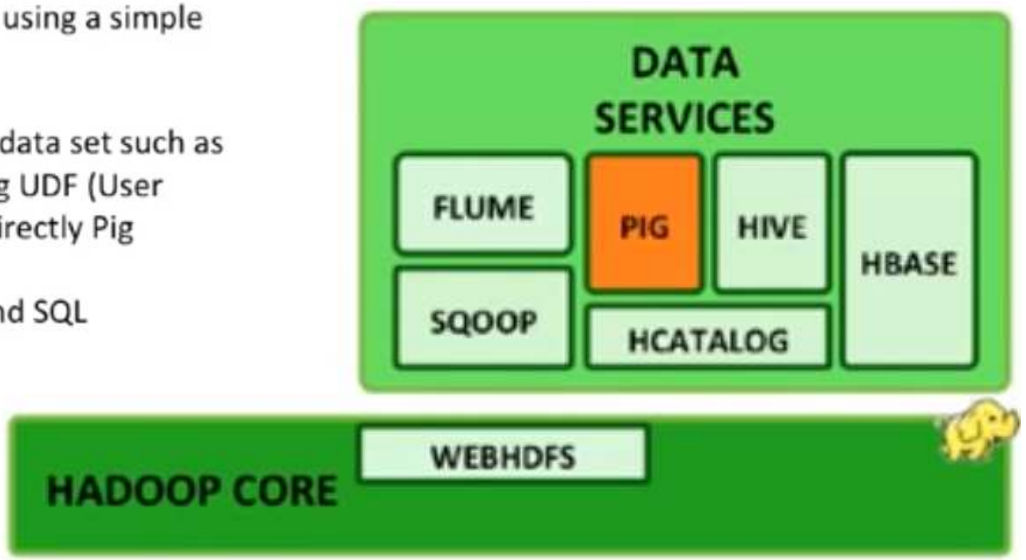
Enterprise Hadoop Architecture(YARN)



Apache PIG

Apache Pig: Scripting Language for Hadoop

- Enables data workers to write complex data transformations using a simple scripting language
- Pig latin (the language) defines a set of transformations on a data set such as aggregate, join and sort among others; can be extended using UDF (User Defined Functions), which can be written in Java and called directly Pig
- Pig appeals to developers familiar with scripting languages and SQL



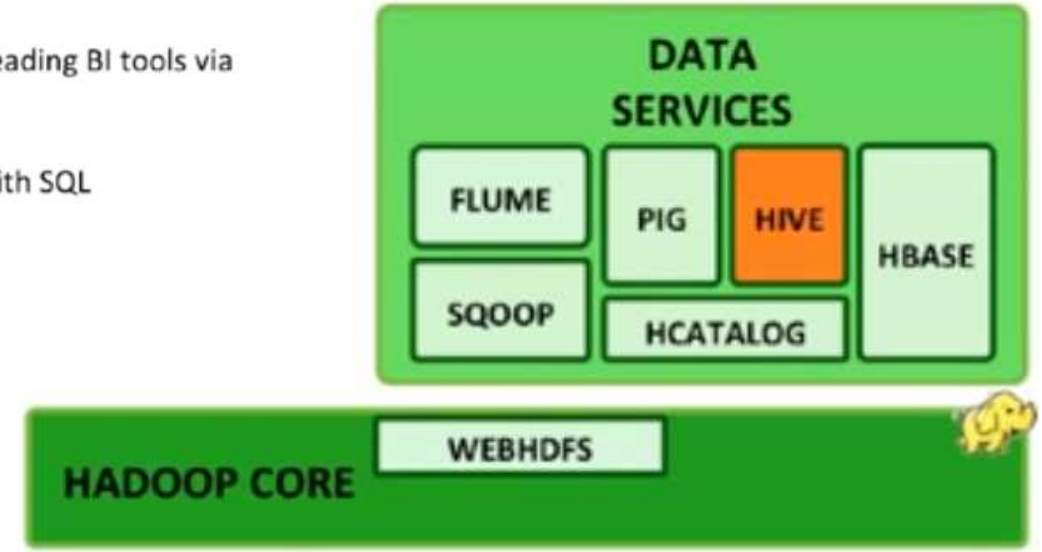
Enterprise Hadoop Architecture(YARN)



Apache Hive

Apache Hive: SQL Interface for Hadoop

- The de-facto SQL-like interface for Hadoop that enables data summarization, ad-hoc query, and analysis of large datasets
- Connects to Excel, Microstrategy, PowerPivot, Tableau and other leading BI tools via Hortonworks Hive ODBC Driver
- HiveQL (HQL) is a language that appeals to data analysts familiar with SQL



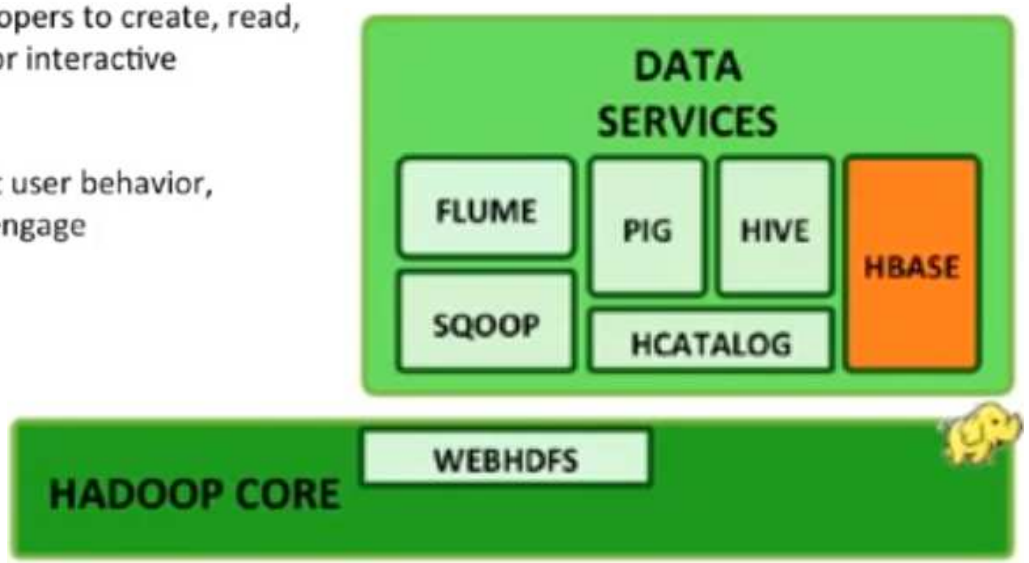
Enterprise Hadoop Architecture(YARN)



Apache HBASE:

Apache HBase: NoSQL DB for Interactive Apps

- Non-relational, columnar database that provides a way for developers to create, read, update, and delete data in Hadoop in a way that performs well for interactive applications
- Commonly used for serving "intelligent applications" that predict user behavior, detect shifting usage patterns, or recommend ways for users to engage



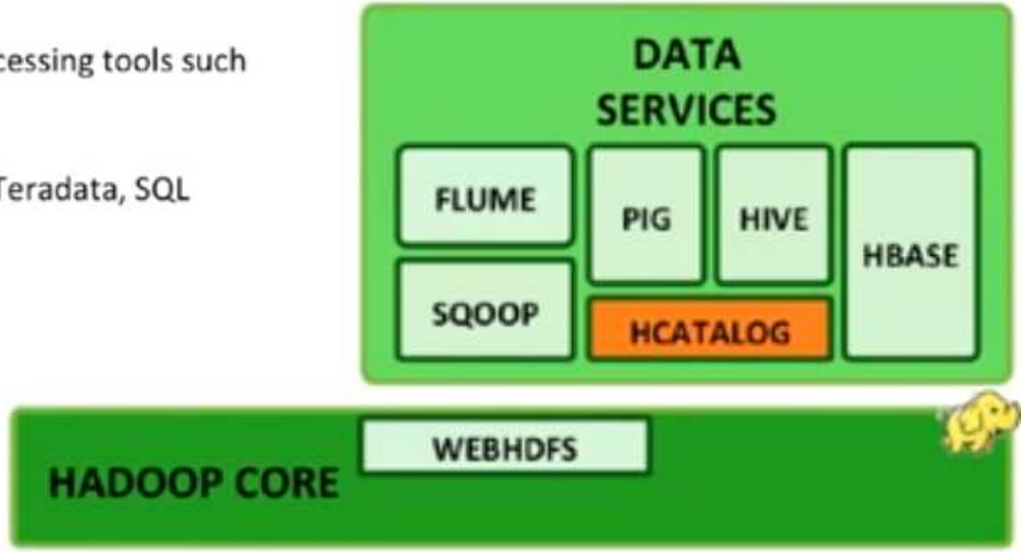
Enterprise Hadoop Architecture(YARN)



Apache HCATALOG:

Apache HCatalog: Metadata & Table Management

- Metadata service that enables users to access Hadoop data as a set of tables without needing to be concerned with where or how their data is stored
- Enables consistent data sharing and interoperability across data processing tools such as Pig, MapReduce and Hive
- Enables deep interoperability and data access with systems such as Teradata, SQL Server, etc.



Enterprise Hadoop Architecture(YARN)



Apache AMBARI:

Apache Ambari: Management & Monitoring

- Make Hadoop clusters easy to operate
- Simplified cluster provisioning with a step-by-step install wizard
- Pre-configured operational metrics for insight into health of Hadoop services
- Visualization of job and task execution for visibility into performance issues
- Complete RESTful API for integrating with existing operational tools
- Intuitive user interface that makes controlling a cluster easy and productive



Enterprise Hadoop Architecture(YARN)



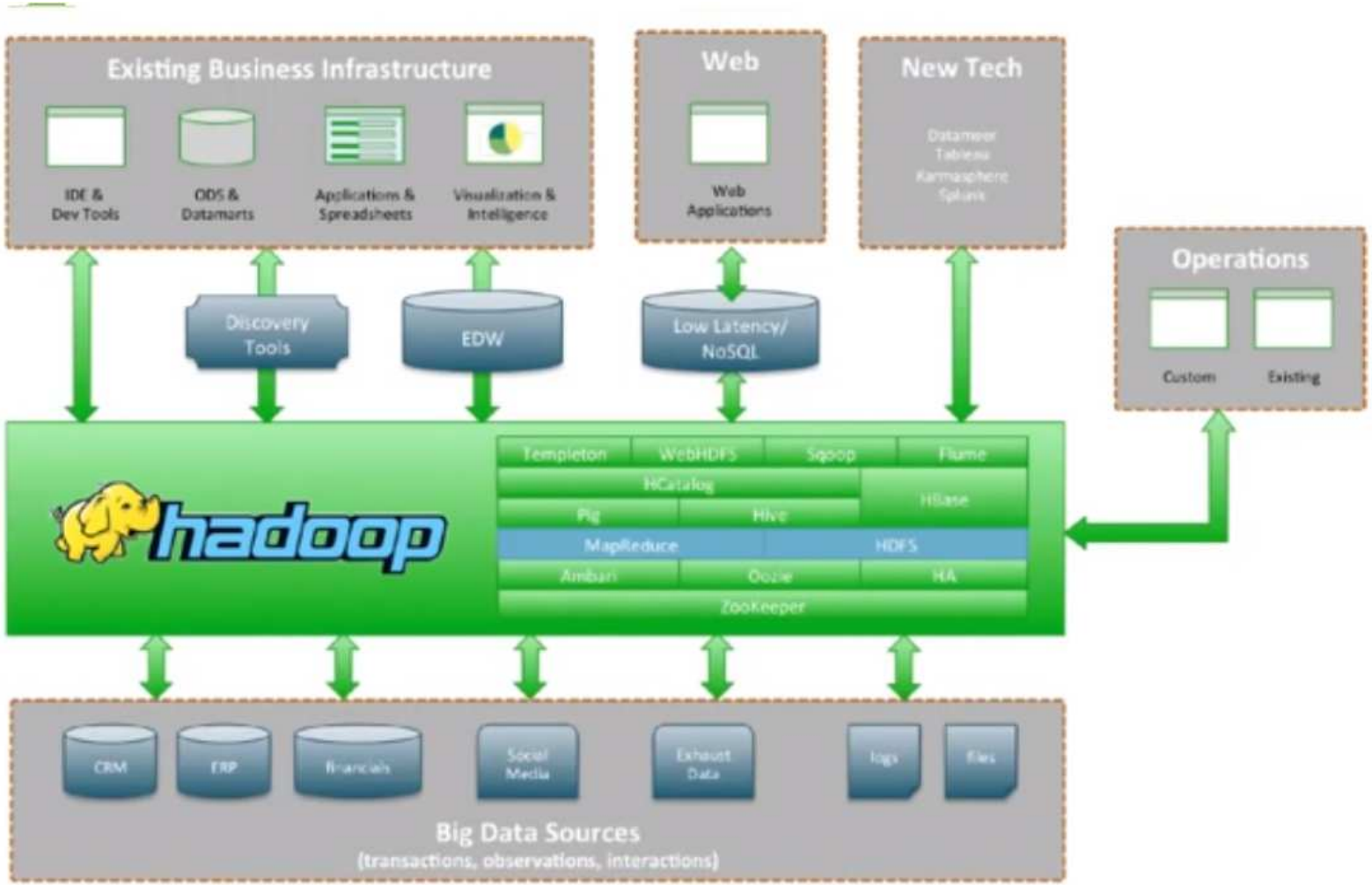
Apache OOZIE:

Apache Oozie: Workflow & Scheduling

- Workflow system that coordinates jobs written in multiple languages such as MapReduce, Pig and Hive
- Allows specification of order and dependencies between jobs so processing can happen in an orderly manner



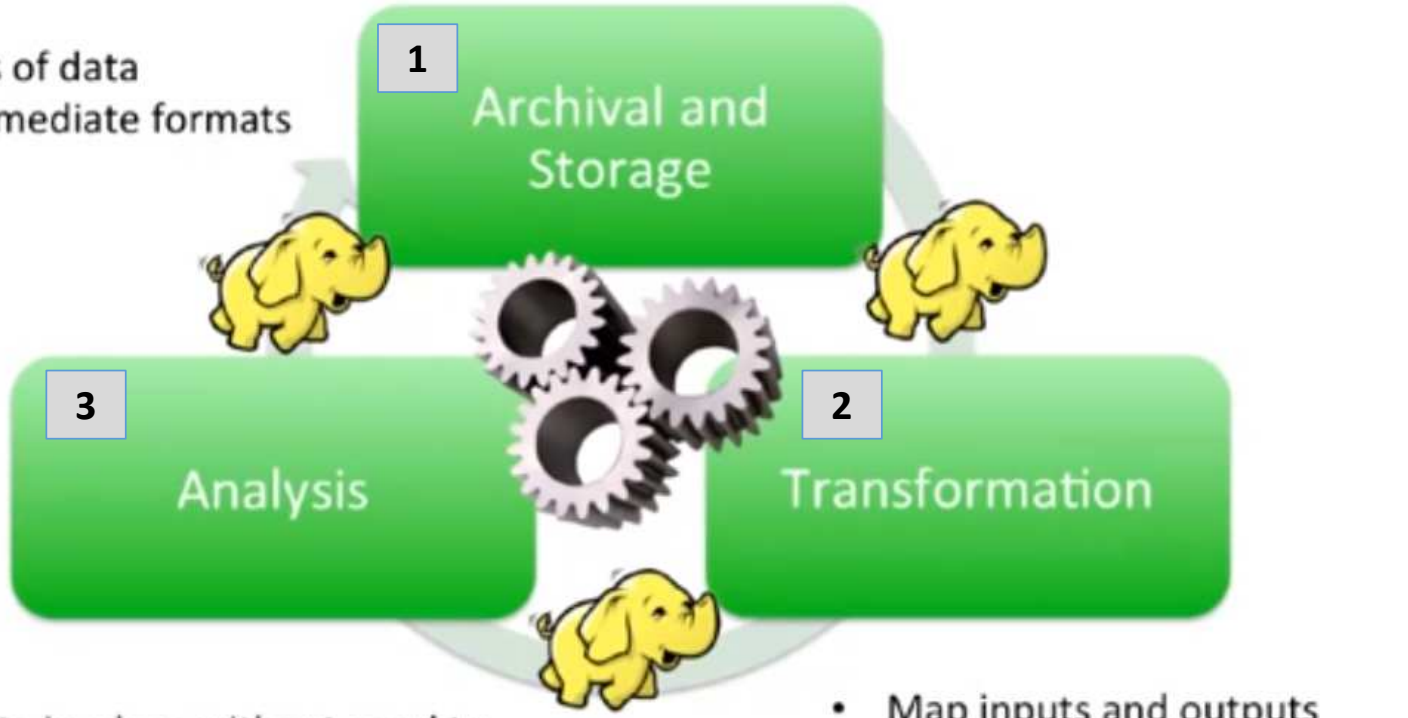
Enterprise Hadoop Architecture(YARN)



Hadoop – sequence of operations



- Retain years of data
- Retain intermediate formats



- Explore data in-place without need to transform first
- Execute arbitrary code

- Map inputs and outputs where needed
- Turn unstructured date into structured at run-time





Can a Hadoop system be compared to a data warehouse?





Can a Hadoop system be compared to a data warehouse?

Answer: – NO – it is a distributed (file) processing system,



RDBMS

differences

hadoop

RELATIONAL PROCESSING		HADOOP PROCESSING
Required before data can be saved	**scheme**	It is not valid until the data is used for reading
Data readings are fast	**speed**	Data records are fast
there are standards and structure	**management**	any standards and structure
processing only for stored data	**processing**	Processing coupled with data form
Well-defined structure	**data type**	Many forms and types of data
For Transaction tasks	**najlepsze dopasowanie**	For mass processing operations



What is the scaling of Hadoop architecture

up to 6K nodes per cluster

for processing approx. 500 PB of data per cluster

support for processing nearly a billion files and blocks

A brief introduction to Hadoop



Hadoop clusters → they can scale from single nodes (where all Hadoop units run on the same node) to thousands of nodes, where functionality is distributed among nodes to increase parallelization of processing processes.



Node 1

One (Node)

A brief introduction to Hadoop



Hadoop clusters → they can scale from single nodes (where all Hadoop units run on the same node) to thousands of nodes, where functionality is distributed among nodes to increase parallelization of processing processes.



Node 1



Node 2

...more nodes can be added if necessary

A brief introduction to Hadoop



Hadoop clusters → they can scale from single nodes (where all Hadoop units run on the same node) to thousands of nodes, where functionality is distributed among nodes to increase parallelization of processing processes.



Node 1



Node 2

…and more nodes

Up to 40 nodes can be placed in the Rack.

...

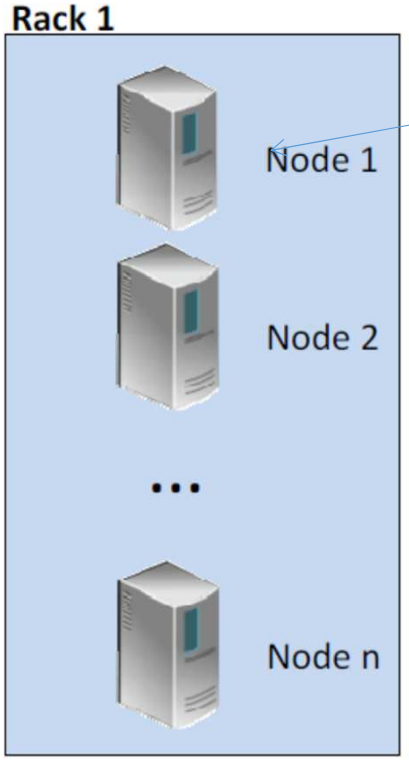


Node n

A brief introduction to Hadoop



Hadoop clusters → they can scale from single nodes (where all Hadoop units run on the same node) to thousands of nodes, where functionality is distributed among nodes to increase parallelization of processing processes.



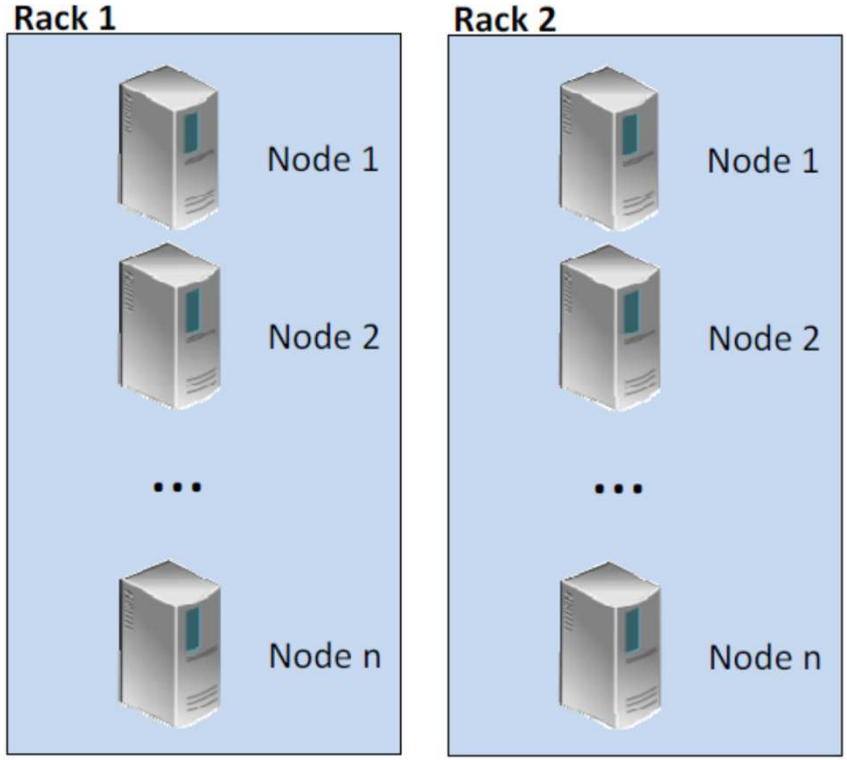
Rack
Up to 40 nodes can be placed in the Rack.



A brief introduction to Hadoop



Hadoop clusters → they can scale from single nodes (where all Hadoop units run on the same node) to thousands of nodes, where functionality is distributed among nodes to increase parallelization of processing processes.

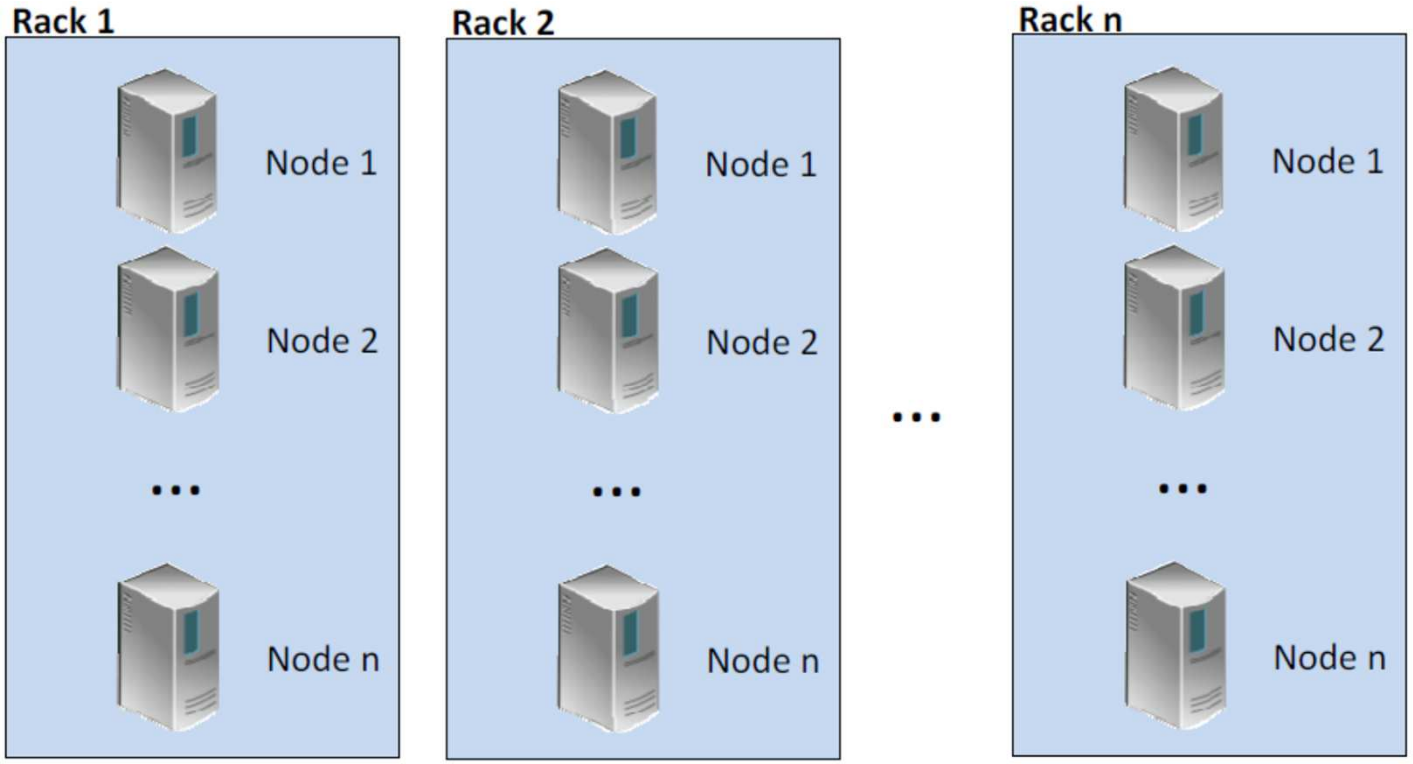


How does Hadoop use parallel processing?

A brief introduction to Hadoop



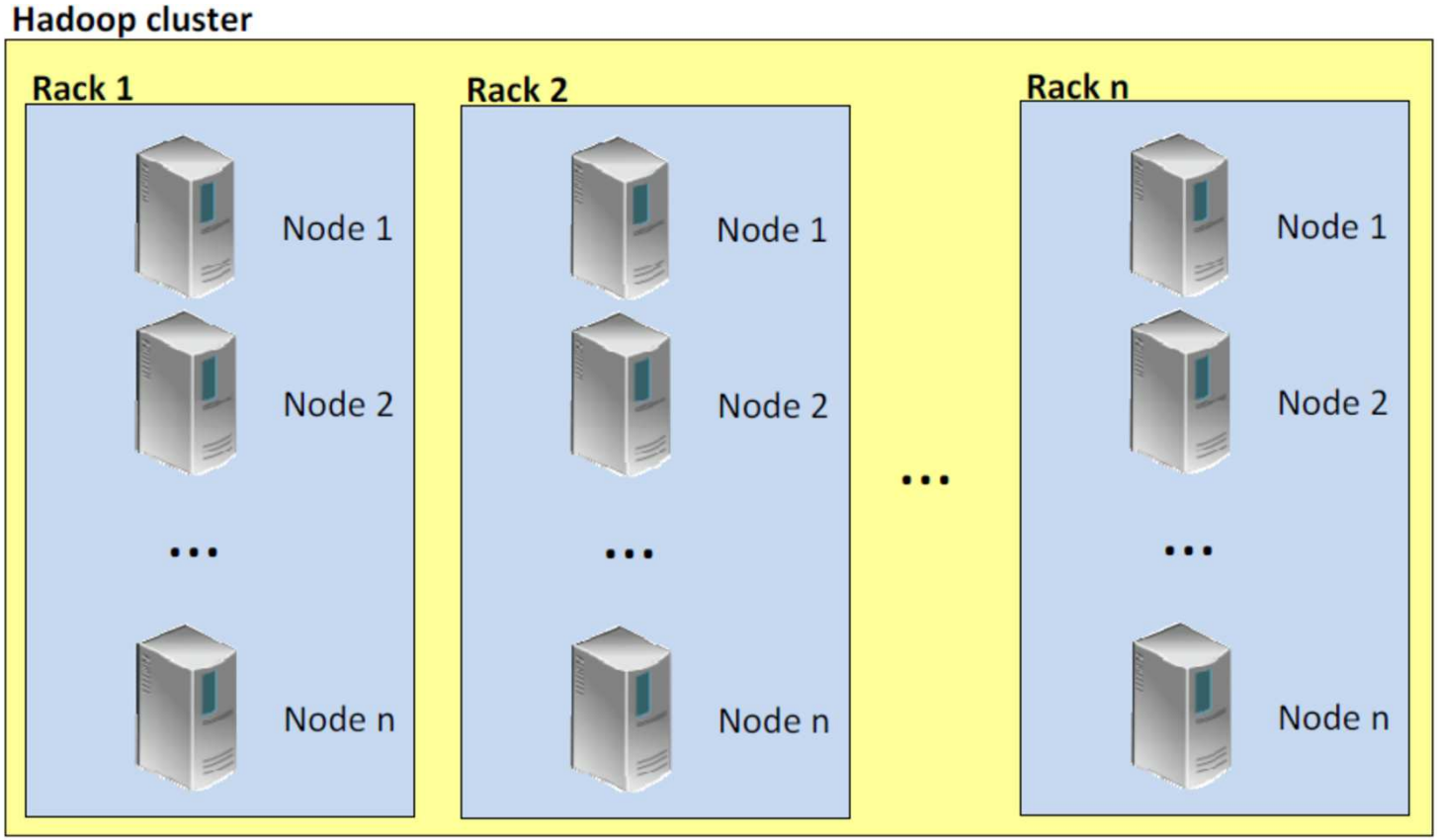
Hadoop clusters → they can scale from single nodes (where all Hadoop units run on the same node) to thousands of nodes, where functionality is distributed among nodes to increase parallelization of processing processes.



A brief introduction to Hadoop



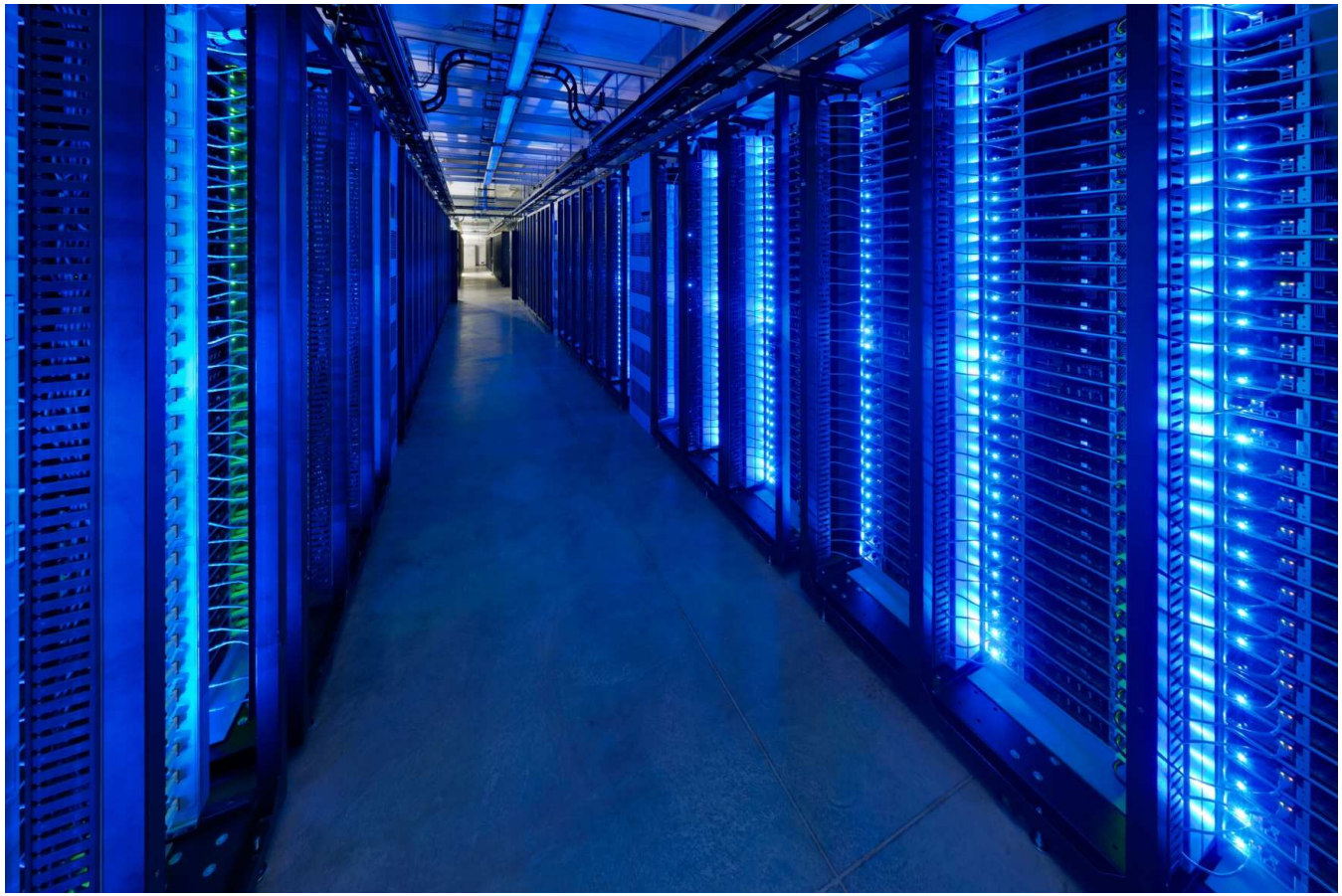
Hadoop clusters → Hadoop cluster ("cluster") is a collection of Racks



A brief introduction to Hadoop



Hadoop clusters → Inside a Data Center



A brief introduction to Hadoop





Pipes responsible for cooling the server room

A brief introduction to Hadoop



Inside a Google Data Center



A brief introduction to Hadoop



Inside a Google Data Center



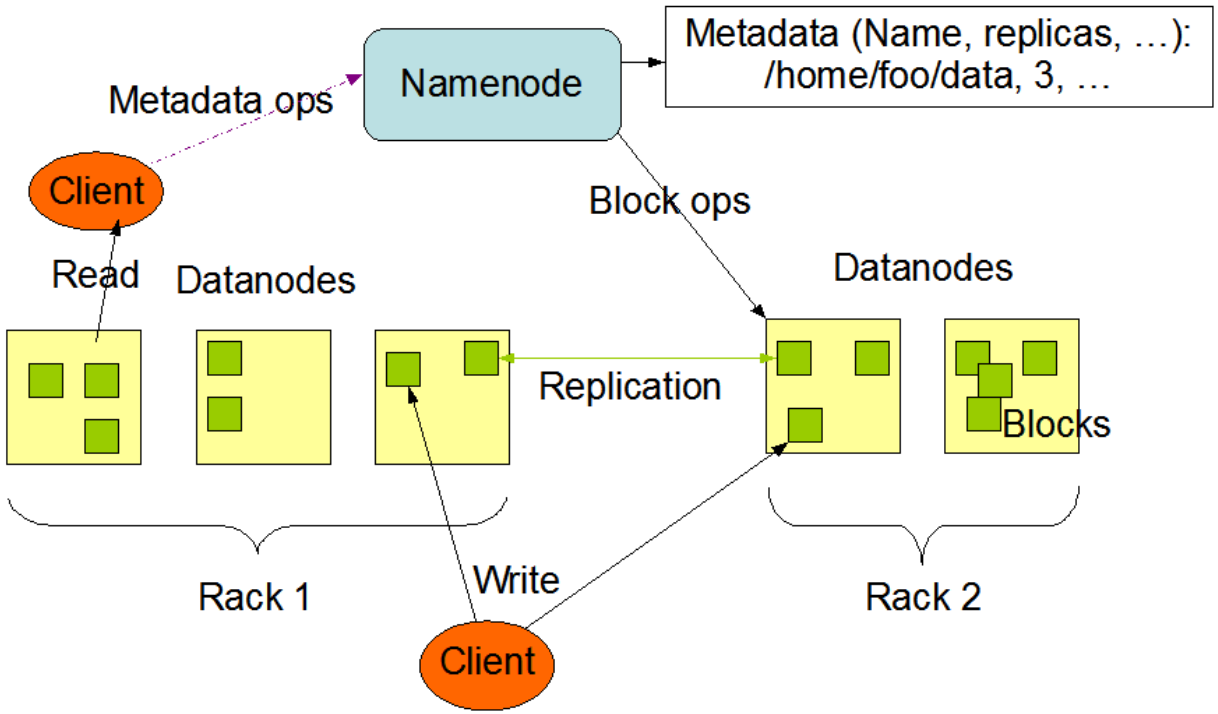




An HDFS cluster consists of:

NameNode(s) - manages cluster metadata, actively monitors the number of block replicas.
NameNode – knows HDFS block mappings to DataNodes, keeping the entire namespace image in RAM.

DataNodes - many per cluster
- they store and process data.
- priodically reports status to NameNode



Hadoop distributed file system (HDFS)



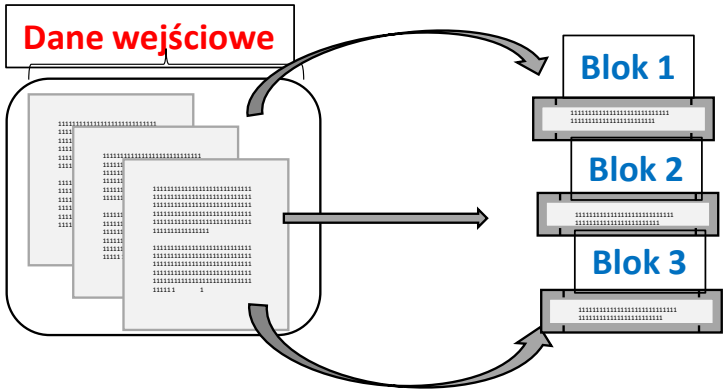
A

- Hadoop Distributed File System (HDFS)



- HDFS is designed to support very large files
- Each file is split into blocks: Hadoop default is 128 MB
- Blocks reside on different physical DataNodes

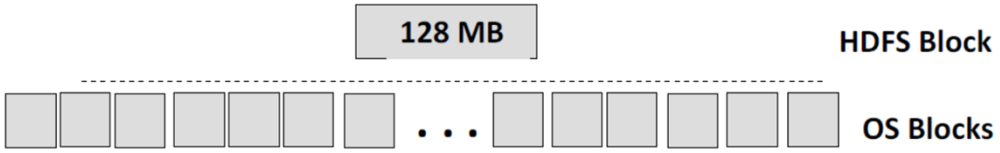
Hadoop works best with very large files. The larger the file, the less time Hadoop spends searching for the next data location on disk and the more time it spends operating at the limits of disk bandwidth.



HDFS - Blocks:

- Blok pliku:

- 64 MB, 128 MB (default), 256 (recommended) - compared to 4 KB on UNIX
- one HDFS block is supported by multiple operating system (OS) blocks



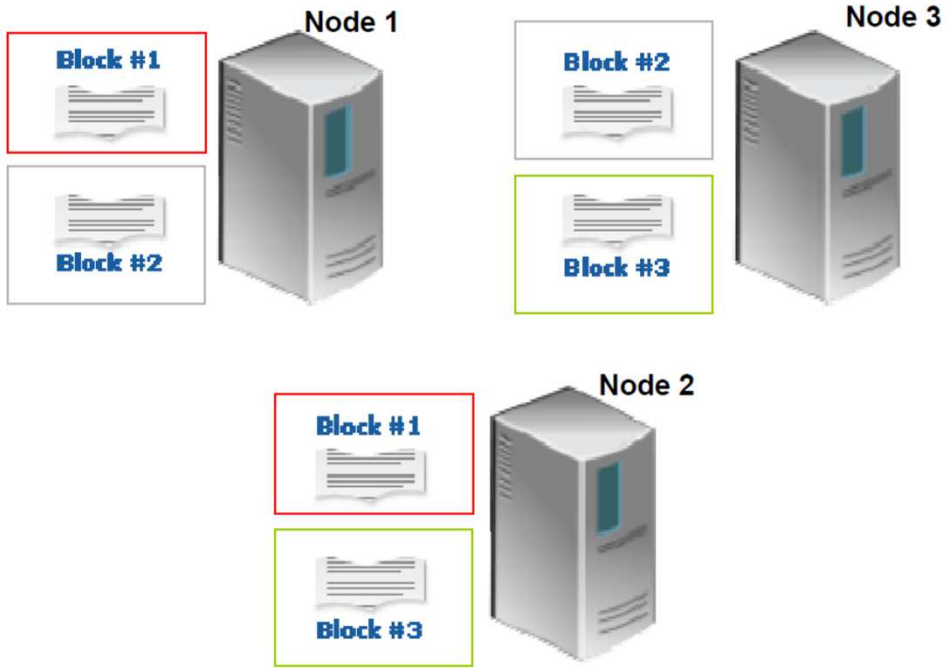
Hadoop distributed file system (HDFS)

A

- Hadoop Distributed File System (HDFS)

how are blocks replicated? (replication in HDFS):

- data blocks are replicated to many nodes
- allows node failure without data loss



Hadoop distributed file system (HDFS)



A

- Hadoop Distributed File System (HDFS)

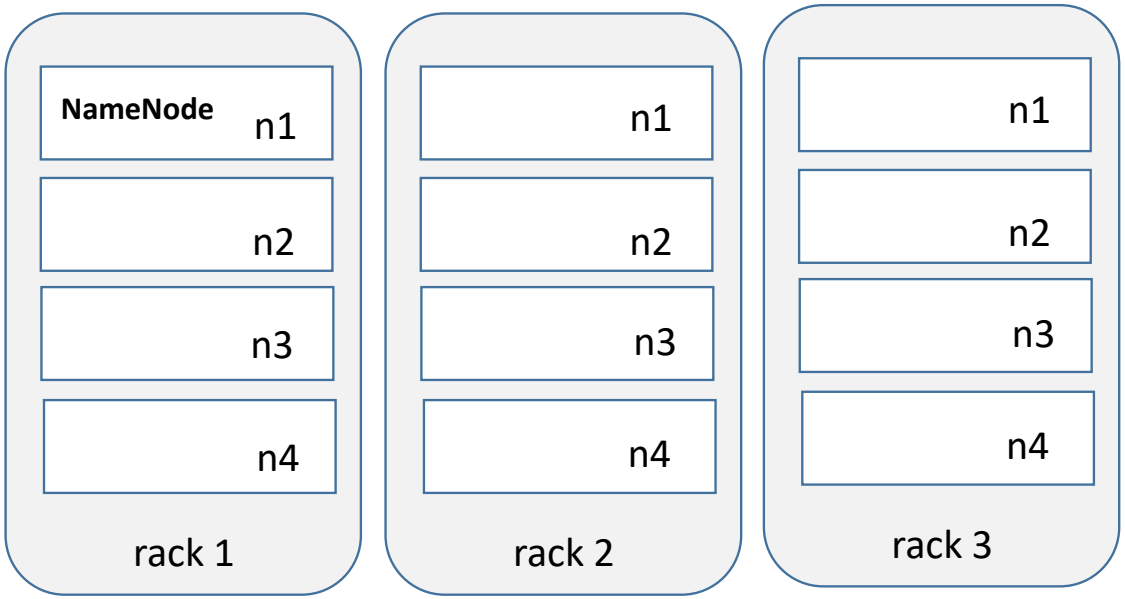
Saving a file to HDFS is done as follows:

HDFS client

File 1

B1 B2 B3

Let's assume we need to change the replication factor for this file to: 3.



Hadoop distributed file system (HDFS)

A

- Hadoop Distributed File System (HDFS)

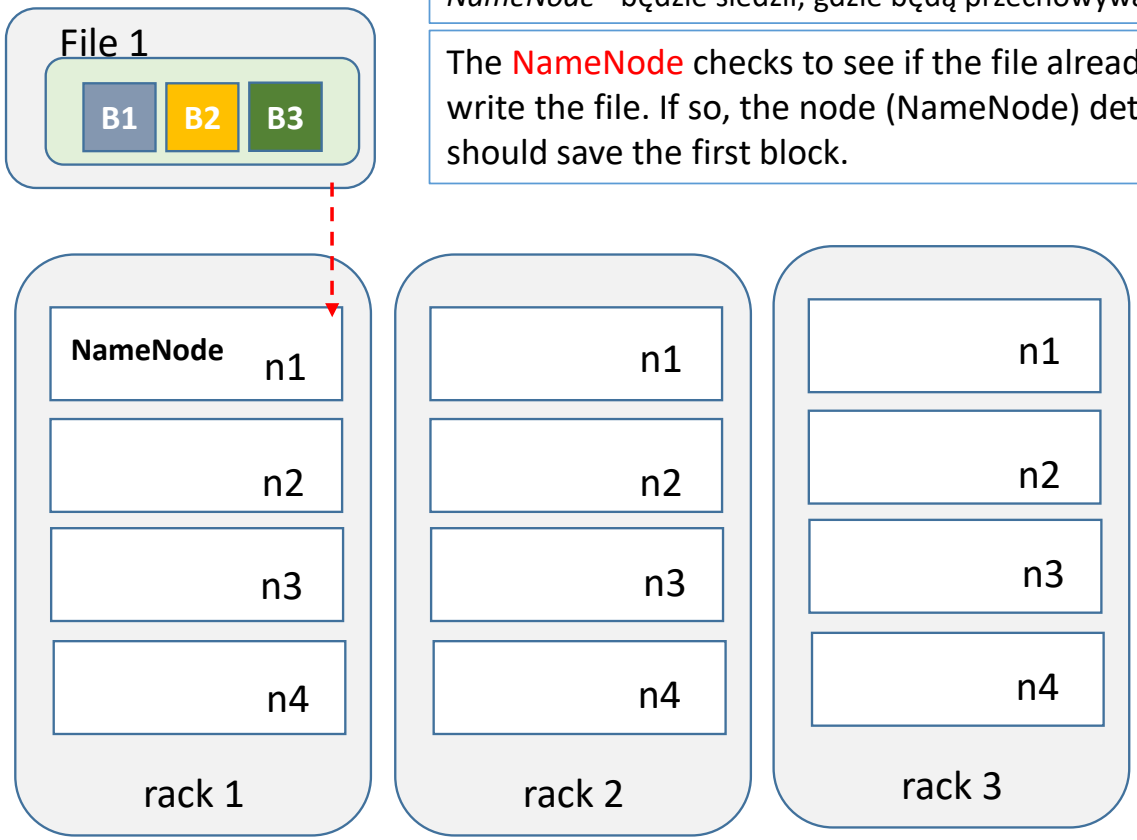


HDFS client

client approves "Create" request to NameNode

NameNode - będzie śledził, gdzie będą przechowywane wszystkie bloki pliku.

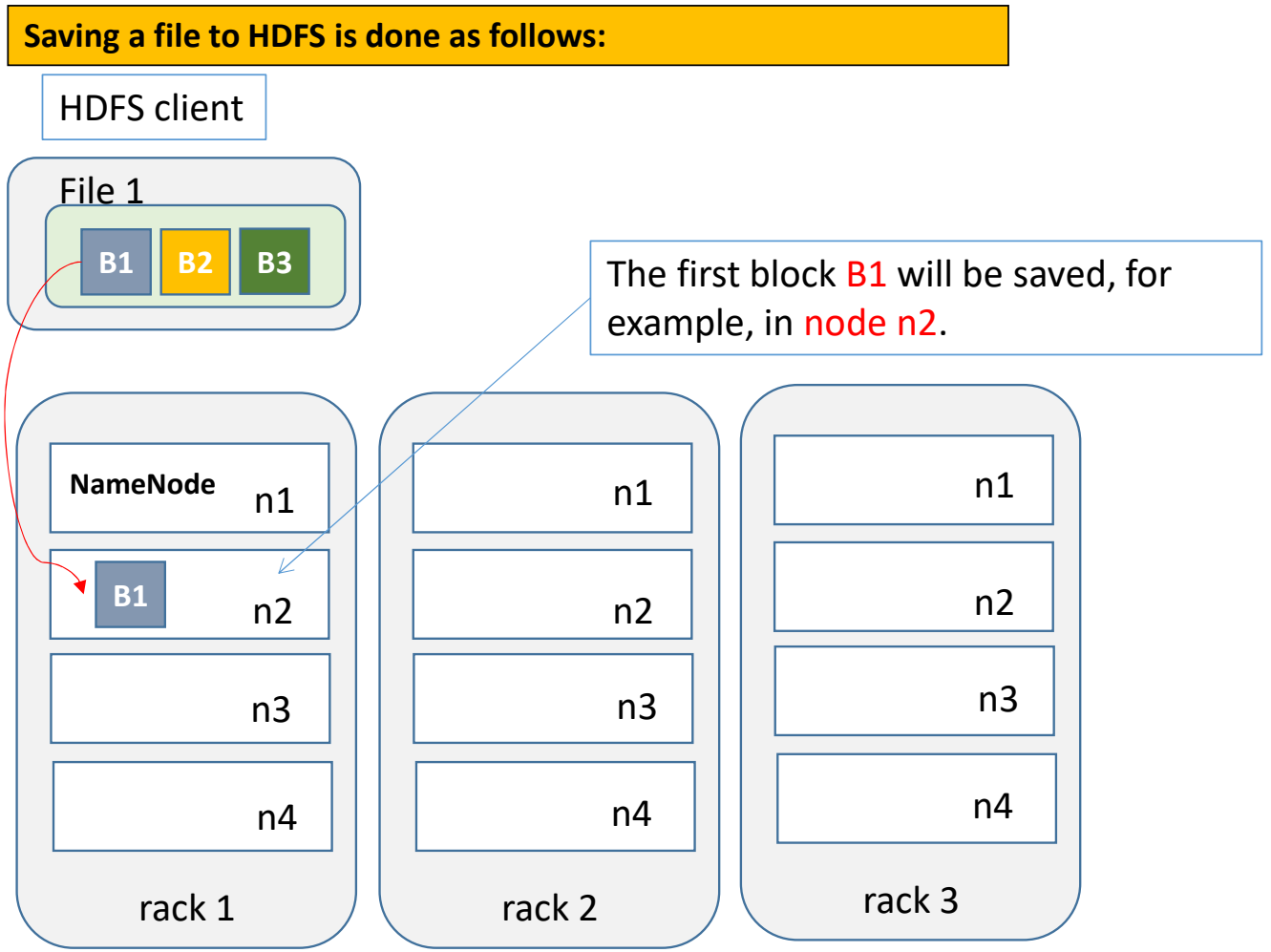
The NameNode checks to see if the file already exists in HDFS and if the client has permission to write the file. If so, the node (NameNode) determines which data storage node (DataNode) should save the first block.



Hadoop distributed file system (HDFS)

A

- Hadoop Distributed File System (HDFS)



Hadoop distributed file system (HDFS)

A

- Hadoop Distributed File System (HDFS)

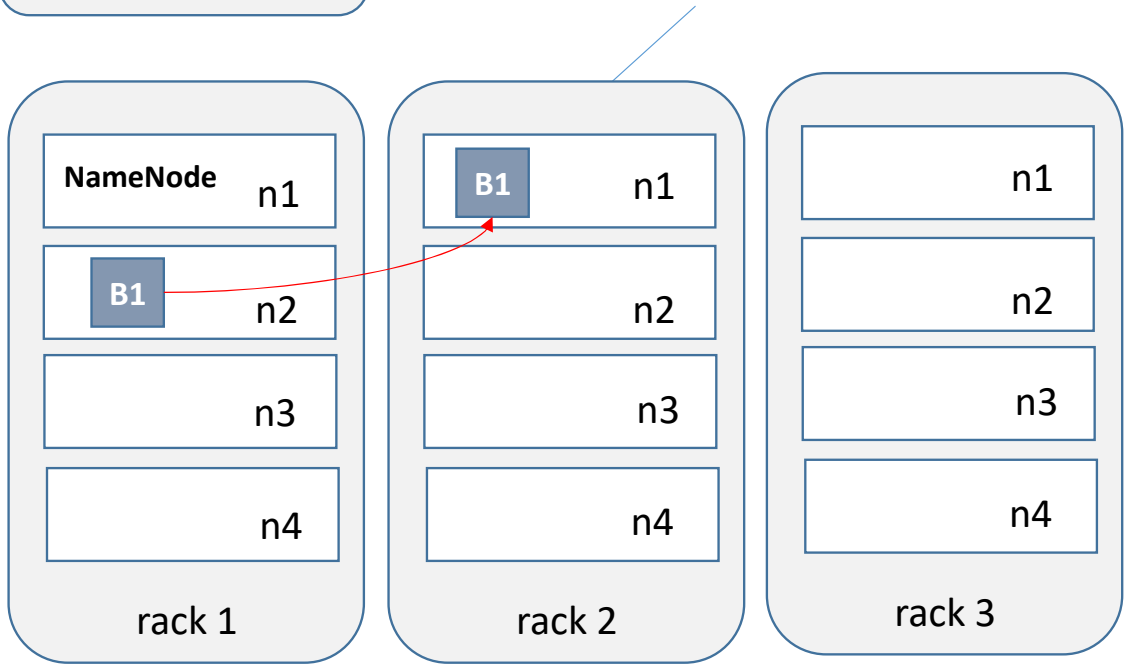
Saving a file to HDFS is done as follows:

HDFS client

File 1

B1 B2 B3

Then, a second DataNode is randomly selected on a Rack other than the one from the first block replication. This is to increase redundancy and fault tolerance.



Hadoop distributed file system (HDFS)

A

- Hadoop Distributed File System (HDFS)

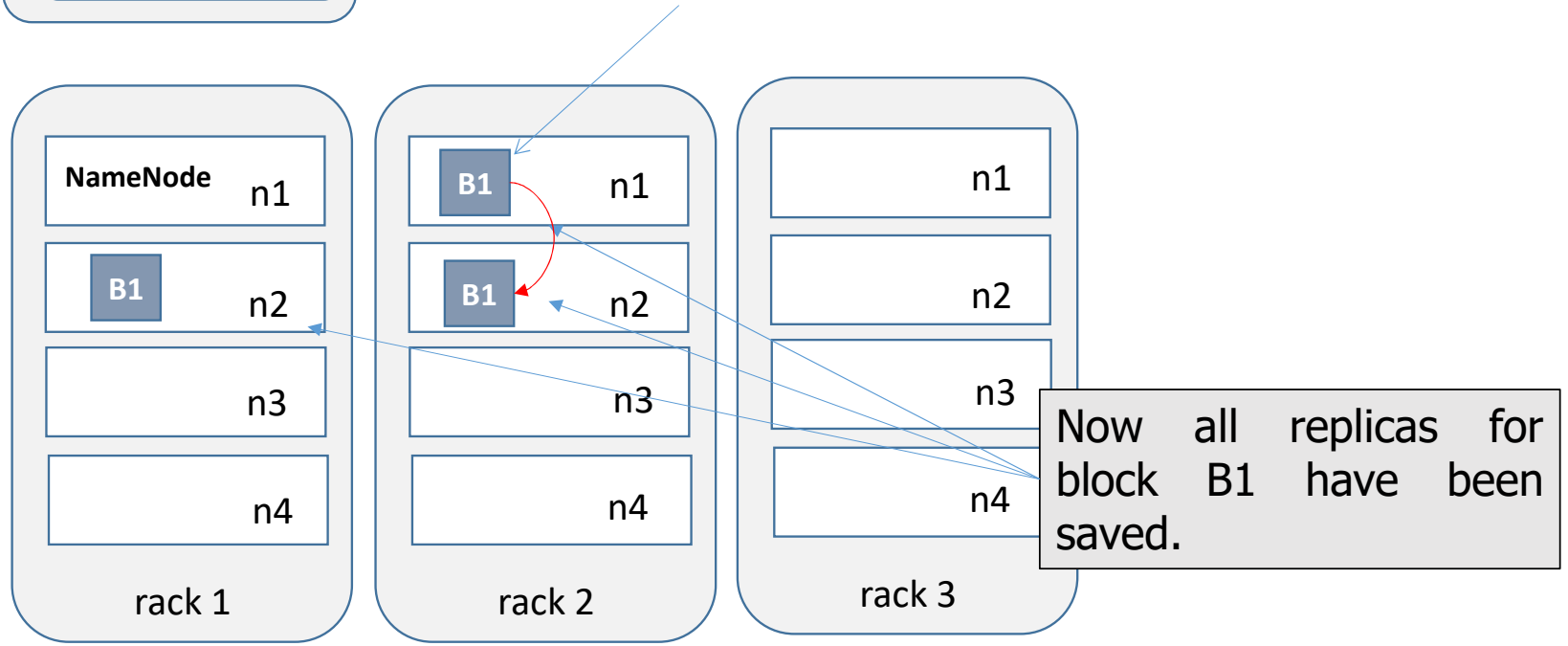
Saving a file to HDFS is done as follows:

HDFS client

File 1

B1 B2 B3

The last replica of block B1 is placed on a random node in the same Rack as the second replica of block B1.



Now all replicas for block B1 have been saved.

Hadoop distributed file system (HDFS)

A

- Hadoop Distributed File System (HDFS)

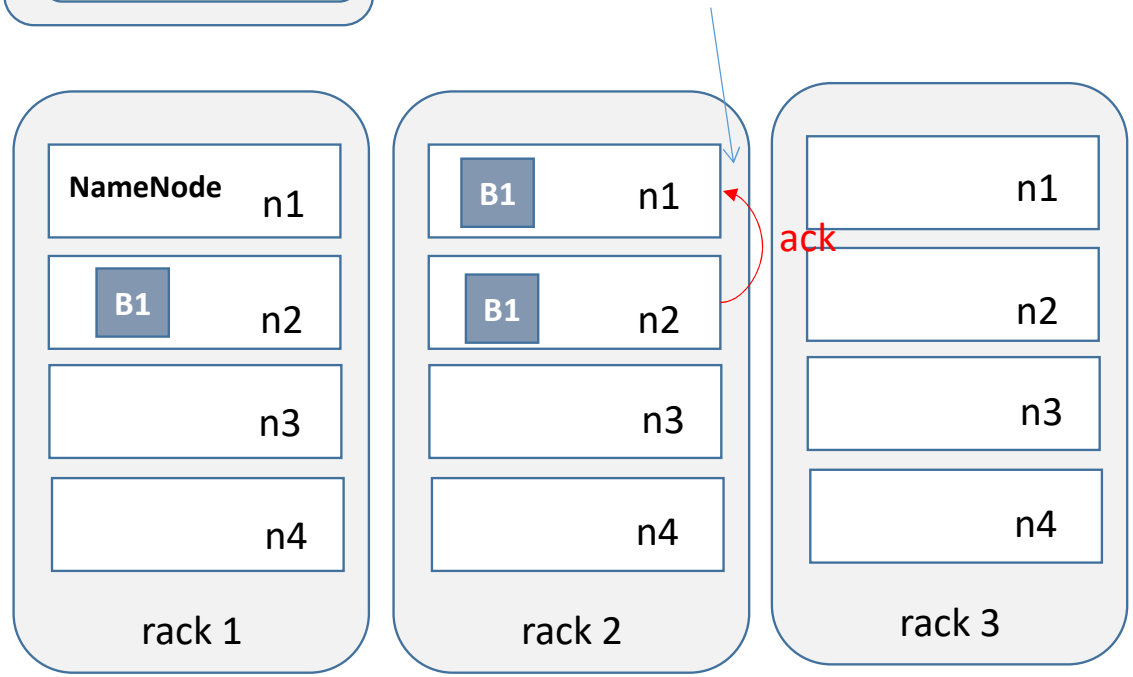
Saving a file to HDFS is done as follows:

HDFS client

File 1

B1 B2 B3

Then a confirmation of writing block B1 is sent from the third data node to the second one (in rack-u2, to node n1 in this example)



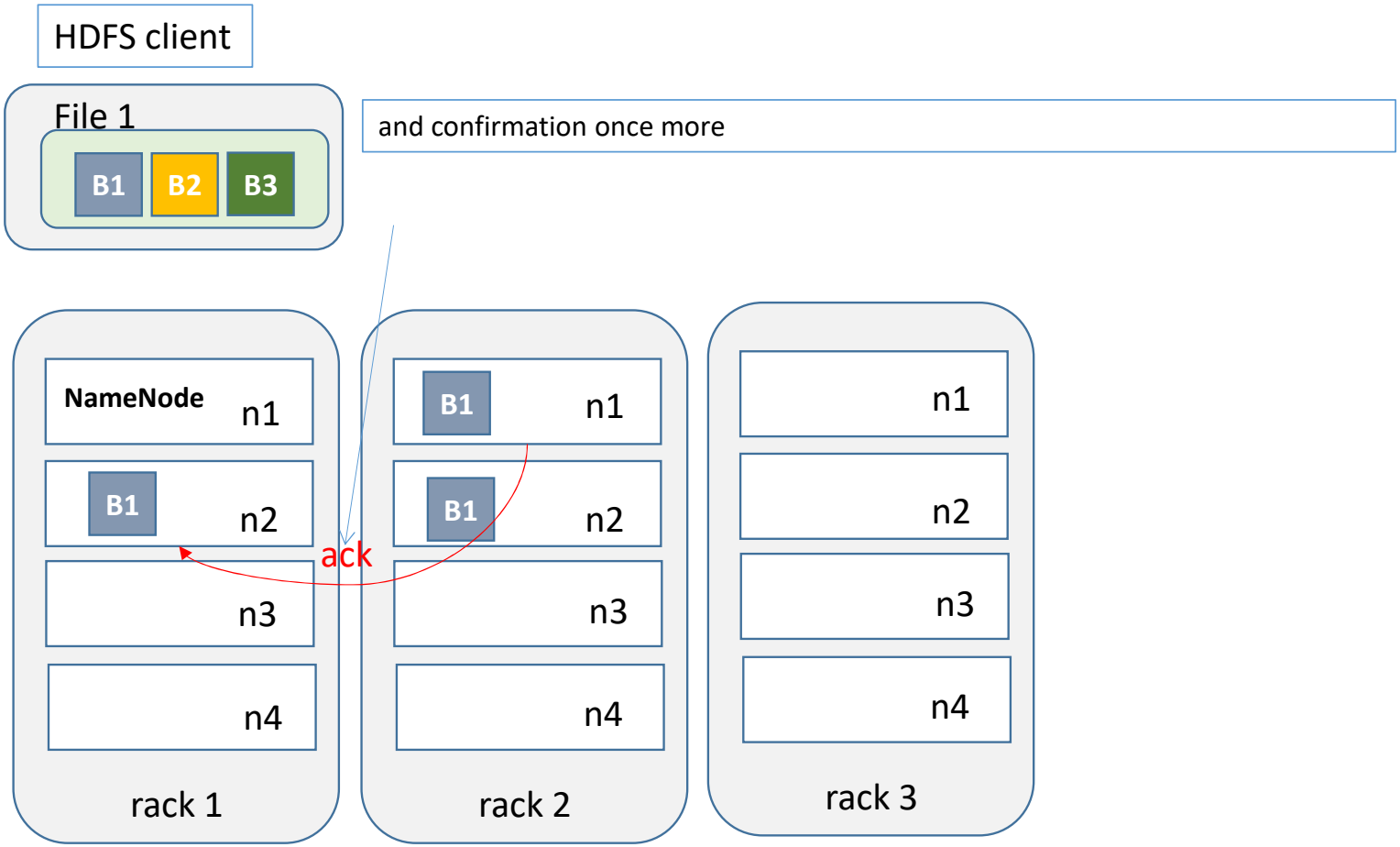
Hadoop distributed file system (HDFS)

A

- Hadoop Distributed File System (HDFS)



Saving a file to HDFS is done as follows:



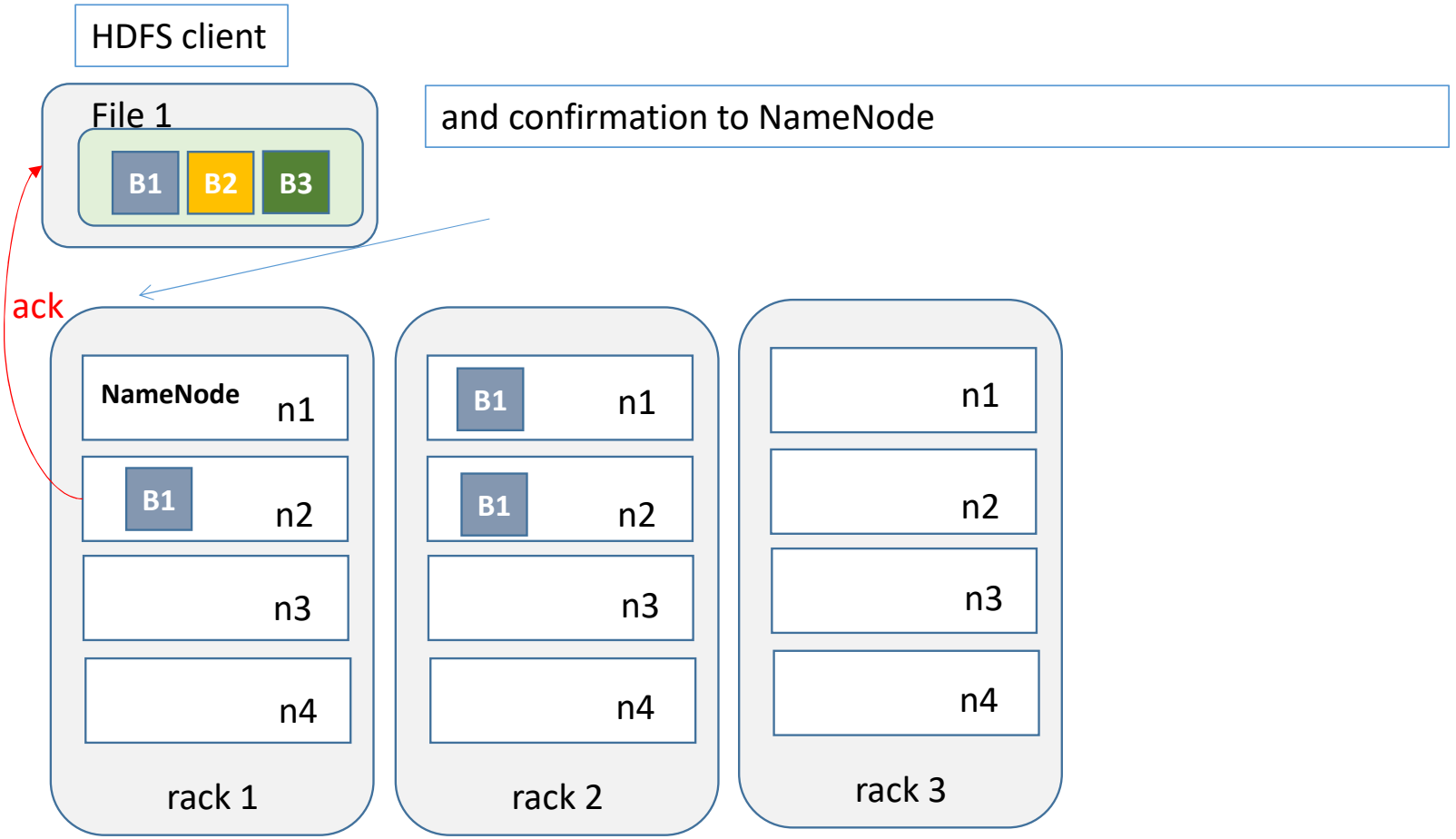
Hadoop distributed file system (HDFS)

A

- Hadoop Distributed File System (HDFS)



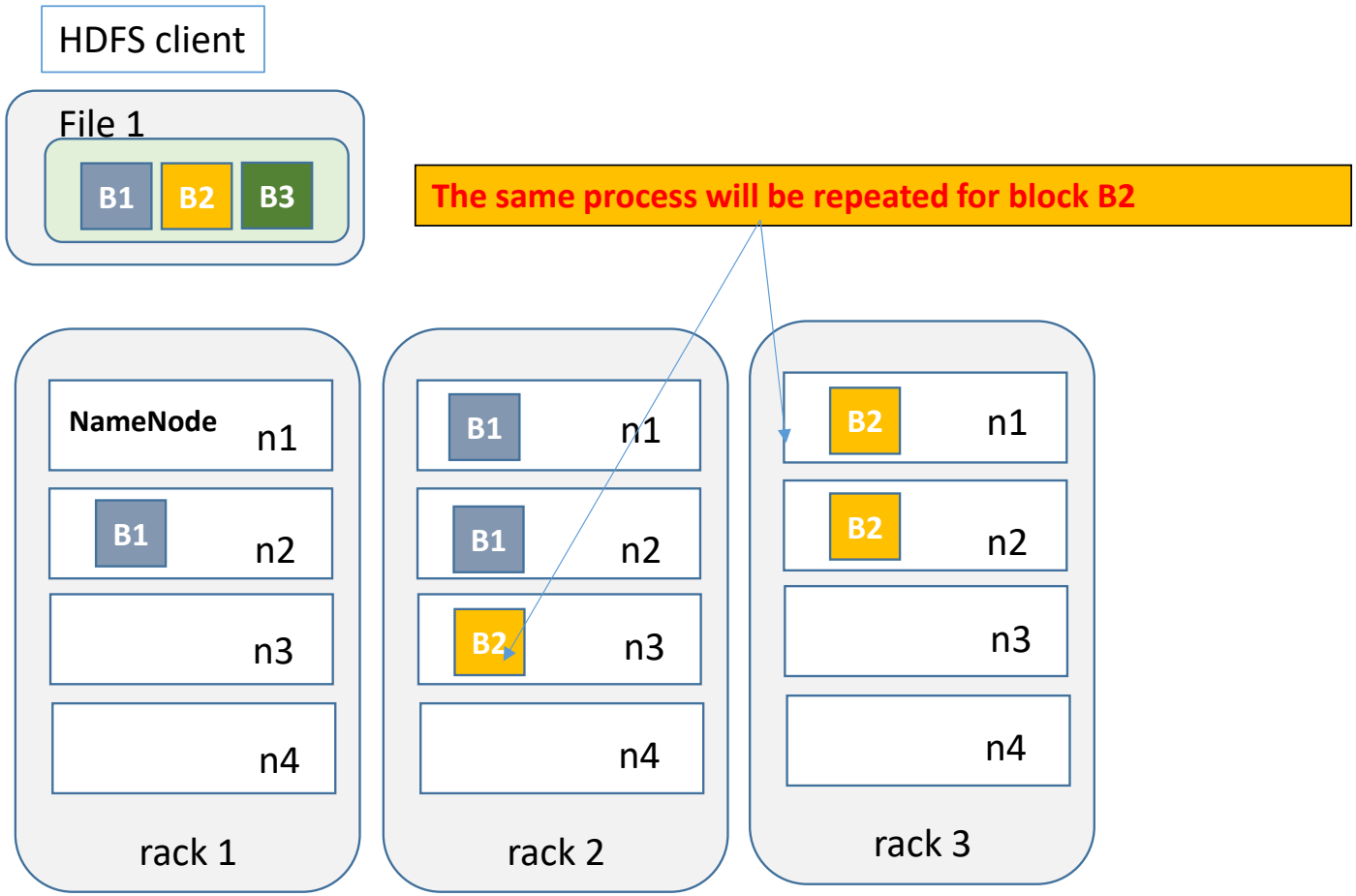
Saving a file to HDFS is done as follows:



Hadoop distributed file system (HDFS)

A

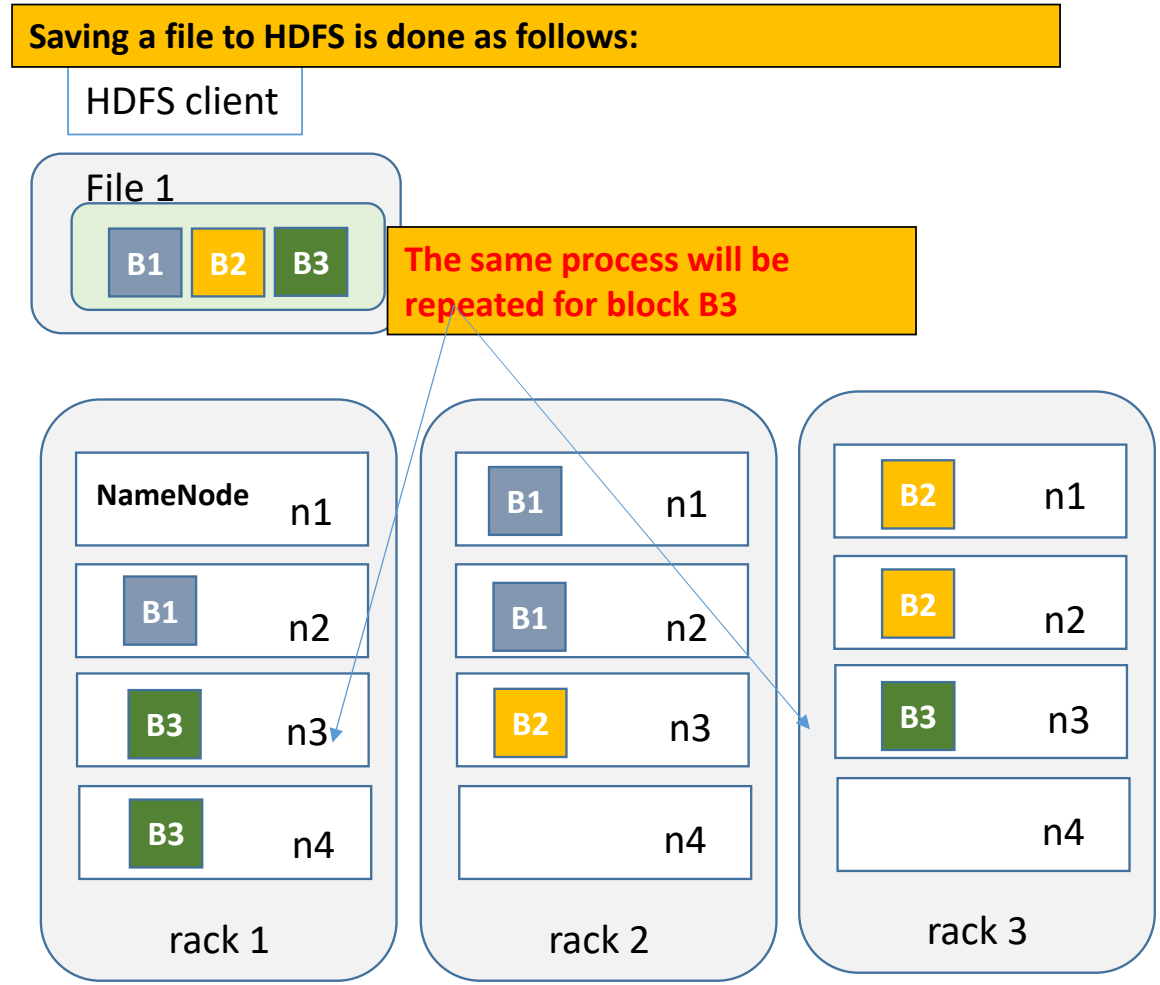
- Hadoop Distributed File System (HDFS)



Hadoop distributed file system (HDFS)

A

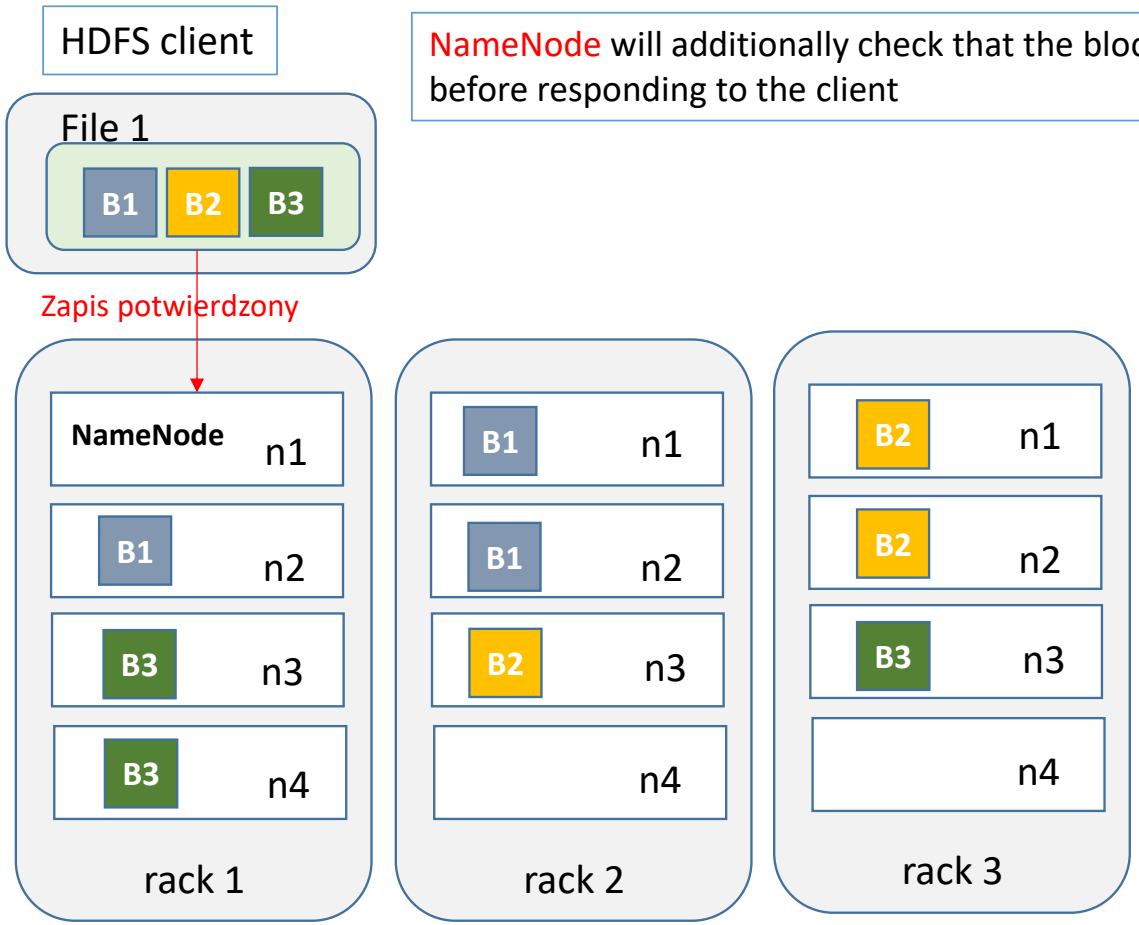
- Hadoop Distributed File System (HDFS)



Hadoop distributed file system (HDFS)

A

- Hadoop Distributed File System (HDFS)



NameNode will additionally check that the blocks are replicated correctly before responding to the client

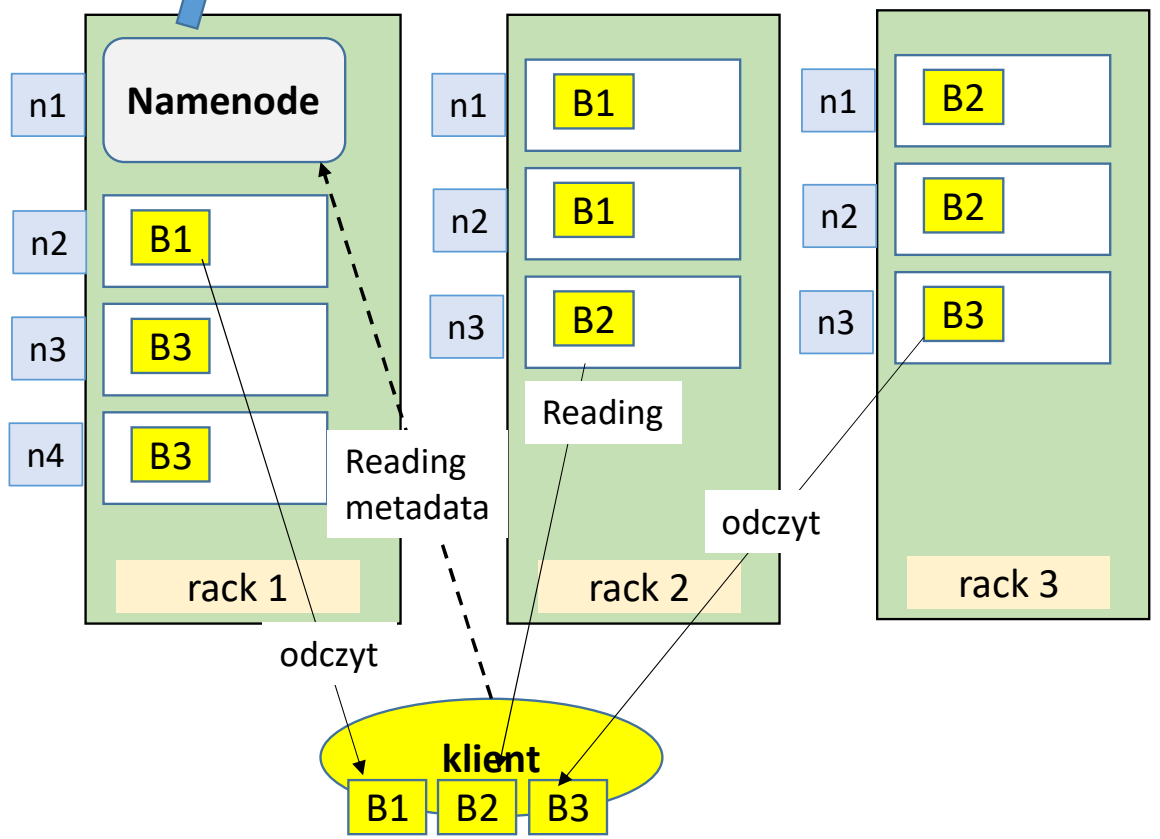
Hadoop distributed file system (HDFS)

A

- Hadoop Distributed File System (HDFS)

stores Metadata (e.g.about names, replicas, etc.)

HDFS architecture



Hadoop distributed file system (HDFS)



A

- Hadoop Distributed File System (HDFS)

HDFS może być obsługiwany:

przez: interfejs Java API

przez: interfejs wiersza polecenia (HDFS Command line interface).
Komendy takie zaczynają się od hadoop (lub „hdfs", spacji dfs (co określa zasoby plików klastra)) i "fs".

polecenia powłoki **systemu plików HDFS** - zaczynają się od kreski.

Polecenia powłoki systemu plików:

wyzwalane są następująco:

hadoop fs <args>

hdfs dfs <args>

Przykład:
polecenie do prezentacji katalogu "Ls" jest wspólnym poleceniem UNIX i jest poprzedzona myślnikiem.
Podobnie jak w systemach UNIX, można podać ścieżkę jako argument.
W przykładzie ścieżka jest bieżącym katalogiem, reprezentowanym przez pojedynczą kropkę.

Przykład:

Prezentacja zawartości katalogu głównego w systemie hdfs:

hadoop fs -ls

lub

hdfs dfs -ls /

Hadoop distributed file system (HDFS)



A

- Hadoop Distributed File System (HDFS)

HDFS Command line interface:

komendy powłoki systemu plików HDFS mogą przyjmować ścieżki jako argumenty.

- **FS shell commands take paths URIs as argument**
  - URI format: scheme://authority/path
- **Scheme:**
  - For the local filesystem, the scheme is *file*
  - For HDFS, the scheme is *hdfs*

Ścieżki te można wyrazić w postaci jednolitych identyfikatorów URI (Uniform Resource Identifier).
Format URI składa się z schematu, authority i ścieżki.

obsługiwane są różne systemy. Lokalny system plików ma schemat "file". HDFS ma system zwany "hdfs".

```
hadoop fs –copyFromLocal
        file://myfile.txt
        hdfs://localhost/user/keith/myfile.txt
```

- **Scheme and authority are optional**
  - Defaults are taken from configuration file core-site.xml

Hadoop distributed file system (HDFS)



A

- Hadoop Distributed File System (HDFS)

HDFS Command line interface:

- **Many POSIX-like commands**
  - cat, chgrp, chmod, chown, cp, du, ls, mkdir, mv, rm, stat, tail
- **Some HDFS-specific commands**
  - copyFromLocal, copyToLocal, get, getmerge, put, setrep

HDFS obsługuje wiele poleceń typu POSIX.
System HDFS nie jest w pełni zgodnym systemem plików POSIX (Portable operating system interface for UNIX), ale obsługuje wiele poleceń.
Polecenia HDFS są w większości łatwo rozpoznawanymi poleceniami UNIX, takimi jak cat i chmod.
Istnieje również kilka poleceń specyficznych dla HDFS, takich jak copyFromLocal.

POSIX (Portable Operating System Interface for Unix) – przenośny interfejs dla systemu operacyjnego Unix.
POSIX to odpowiedź na próby standaryzacji różnych odmian systemu operacyjnego Unix.

Hadoop distributed file system (HDFS)



A

- Hadoop Distributed File System (HDFS)

HDFS - szczegółowe polecenia:

- **copyFromLocal / put**
  - Copy files from the local file system into fs

```
hadoop fs -copyFromLocal <localsrc> .. <dst>
```

Or

```
hadoop fs -put <localsrc> .. <dst>
```

Przeanalizujmy kilka z nich:

copy**From**Local i **put** to dwie komendy specyficzne dla HDFS, które robią to samo → kopiują pliki z lokalnego systemu plików do lokalizacji w systemie plików HDFS. Ich przeciwieństwem jest polecenie copy**To**Local, które można również nazwać „weź”

Hadoop distributed file system (HDFS)



A

- Hadoop Distributed File System (HDFS)

HDFS - szczegółowe polecenia:

- **copyToLocal / get**
  - Copy files from fs into the local file system

```
hadoop fs -copyToLocal [-ignorecrc] [-crc]
                                <src> <localdst>
```

Or

```
hadoop fs -get [-ignorecrc] [-crc]
                                <src> <localdst>
```

Polecenie copy**To**Local (**get**) kopiuje pliki z systemu plików HDFS określonego przez użytkownika do lokalnego systemu plików.

Hadoop distributed file system (HDFS)



A

- Hadoop Distributed File System (HDFS)

HDFS - szczegółowe polecenia:

- **getMerge**
  - Get all the files in the directories that match the source file pattern
  - Merge and sort them to only one file on local fs
  - <src> is kept

```
hadoop fs -getmerge <src> <localdst>
```

getmerge - to udoskonalona forma pobierania, która może łączyć pliki z wielu lokalizacji w pojedynczy plik lokalny.

Hadoop distributed file system (HDFS)



A

- Hadoop Distributed File System (HDFS)

HDFS - szczegółowe polecenia:

- **setRep**
  - Set the replication level of a file.
  - The -R flag requests a recursive change of replication level for an entire tree.
  - If -w is specified, waits until new replication level is achieved.

```
hadoop fs -setrep [-R] [-w] <rep> <path/file>
```

setrep - pozwala na zastąpienie domyślnego poziomu replikacji na poziom określony przez użytkownika.
Można to zrobić dla jednego pliku lub, z opcją -R, dla całego drzewa.

Polecenie to zwraca natychmiast nowy poziom replikacji.
Jeśli chcesz, aby polecenie założyło blokadę do czasu wykonania zadania, przekaż opcję -w.

Hadoop distributed file system (HDFS)

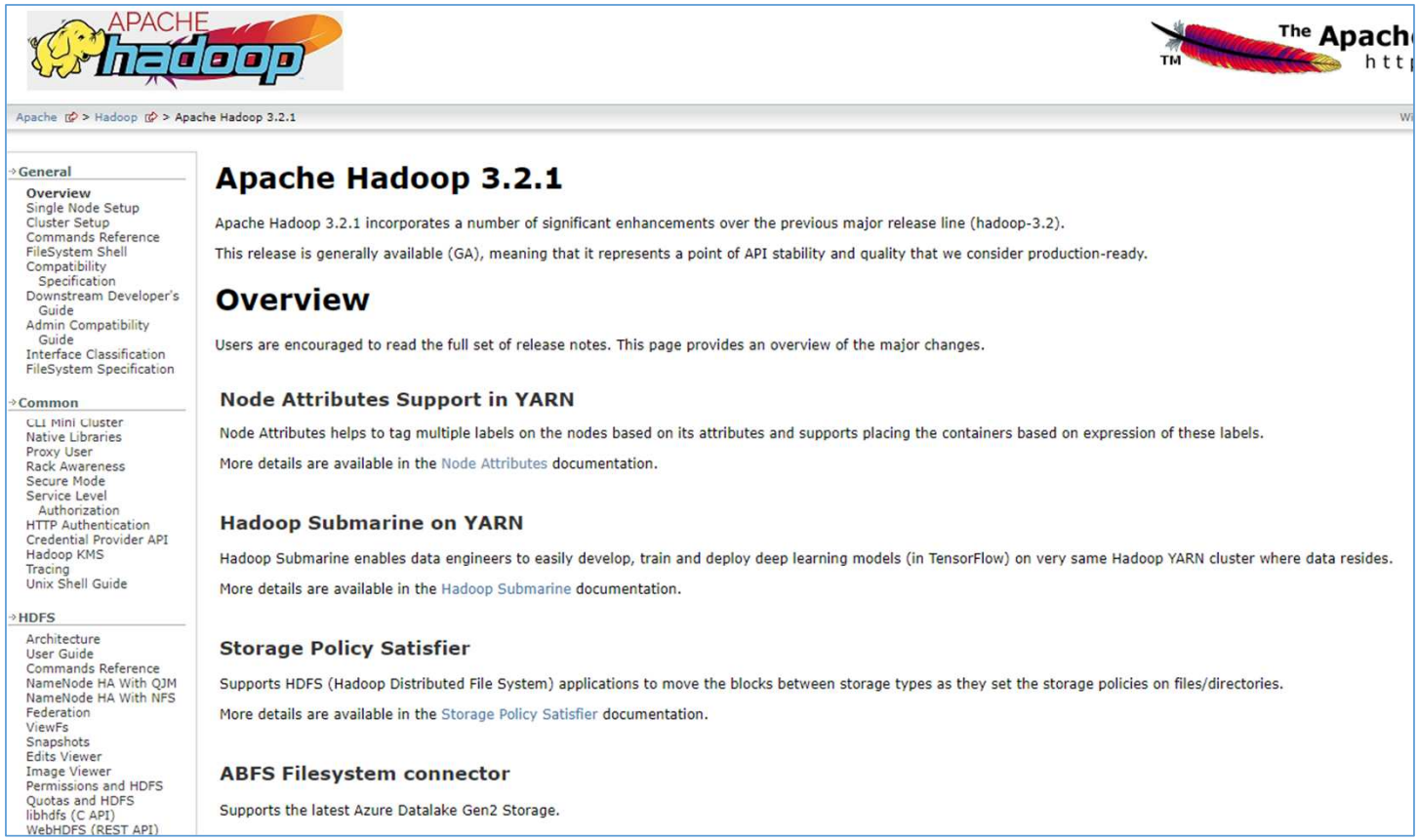


A

Przykłady komend podane są np.:



http://hadoop.apache.org/docs/stable/





Creating an Apache Hadoop cluster...

... in Azure HDInsight using the Azure portal

At the WEB site you can find

Description: What is Apache Hadoop in Azure HDInsight?

https://docs.microsoft.com/pl-pl/azure/hdinsight/hadoop/apache-hadoop-introduction

... How to create an Apache Hadoop cluster in Azure HDInsight using the Azure portal

https://docs.microsoft.com/pl-pl/azure/hdinsight/hadoop/apache-hadoop-linux-create-cluster-get-started-portal

How to create **an Apache Hadoop cluster in Azure HDInsight using the Azure portal**



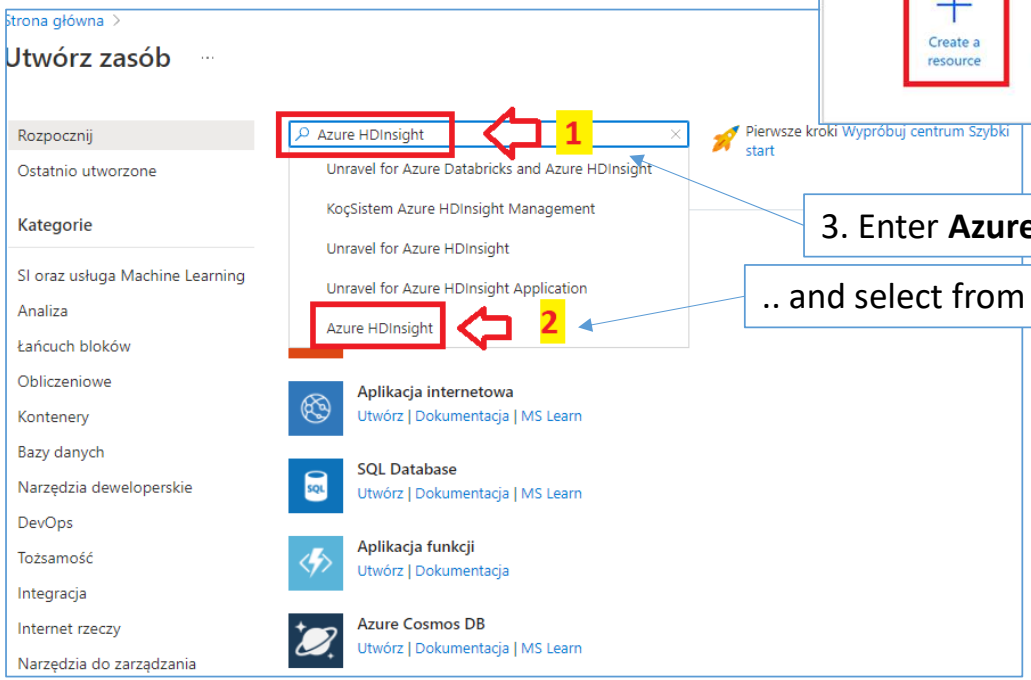
Procedure:

1. Log in to Azure portal.

https://portal.azure.com/#home

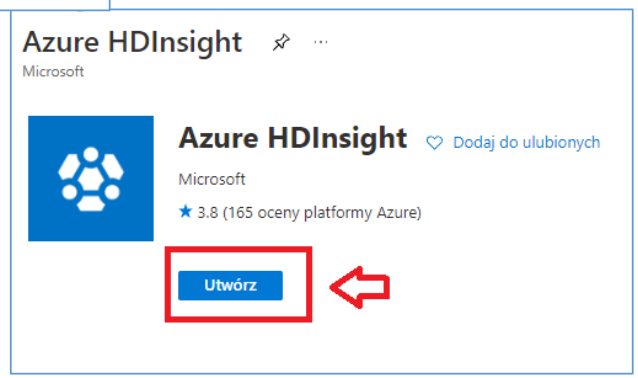
2. Select an item in the top menu → **Create a resource**





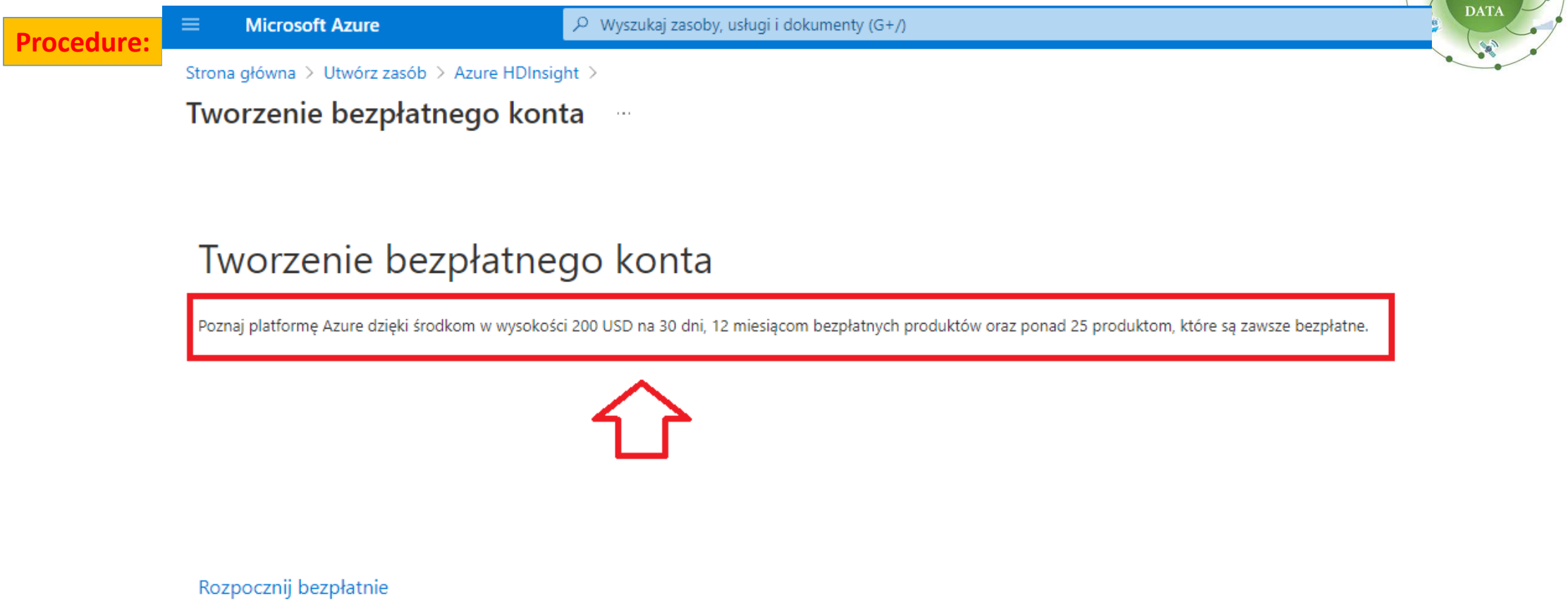
3. Enter **Azure HDInsight** → because there is no resource on the list

.. and select from the list at the bottom: **Azure HDInsight**



How to create an Apache Hadoop cluster in Azure HDInsight using the Azure portal

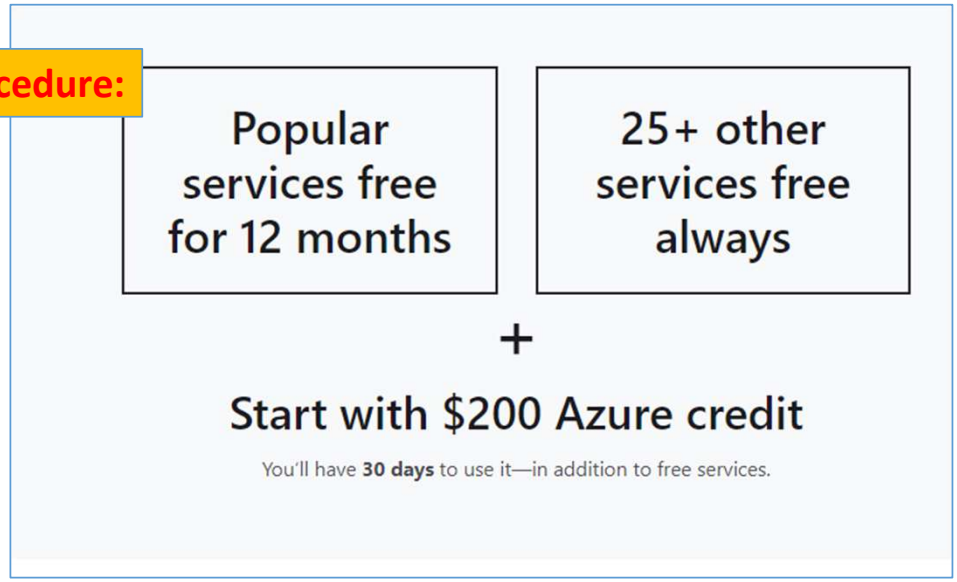
Procedure:

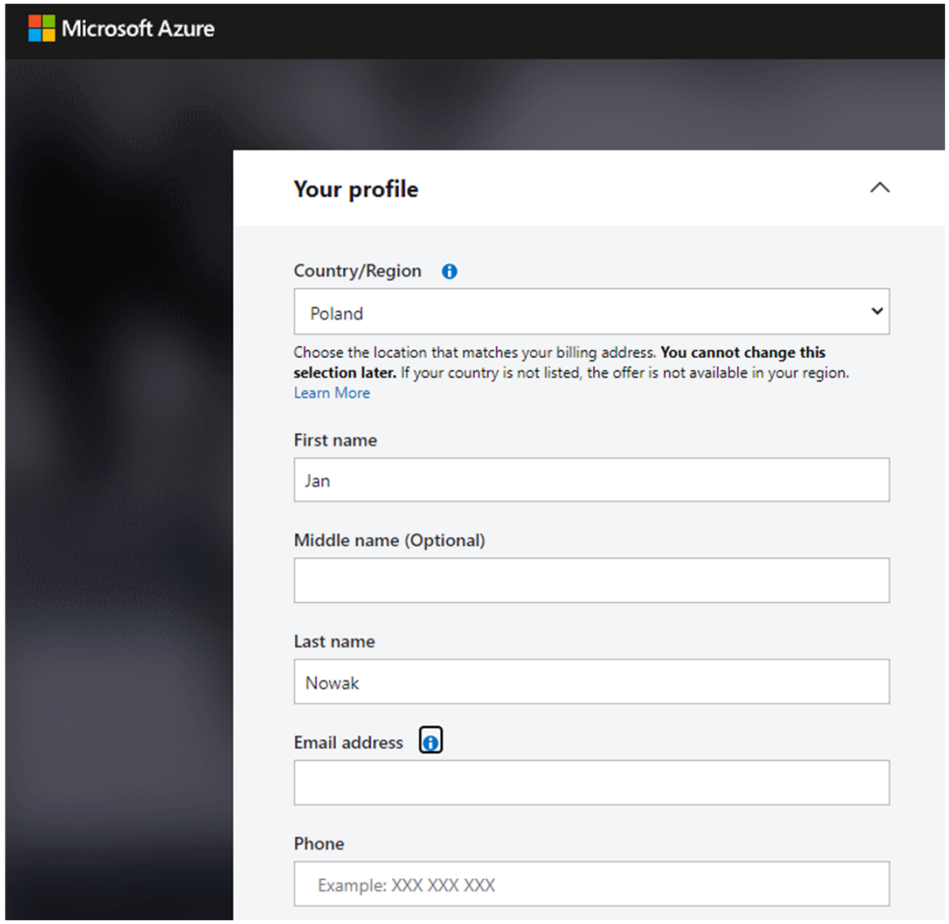


How to create an Apache Hadoop cluster in Azure HDInsight using the Azure portal



Procedure:

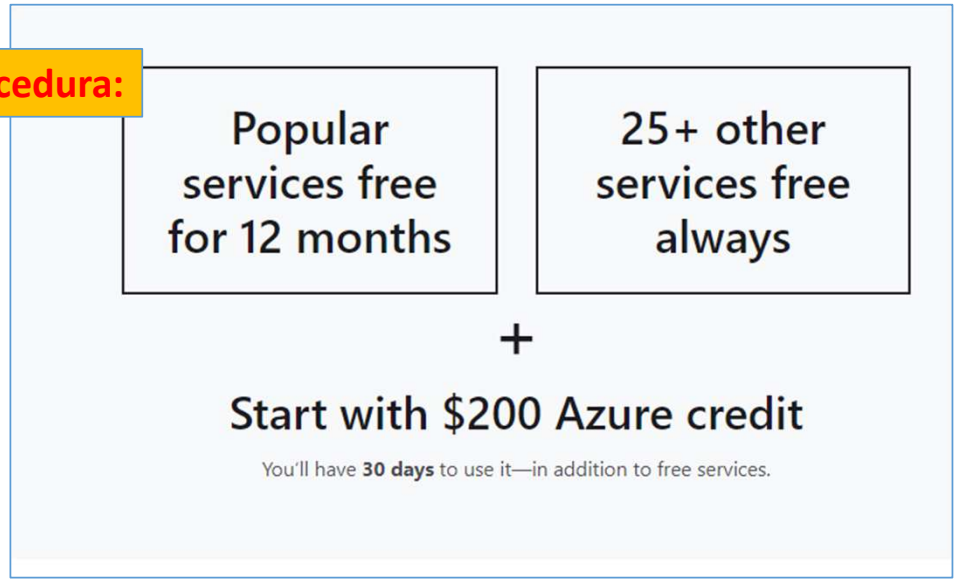


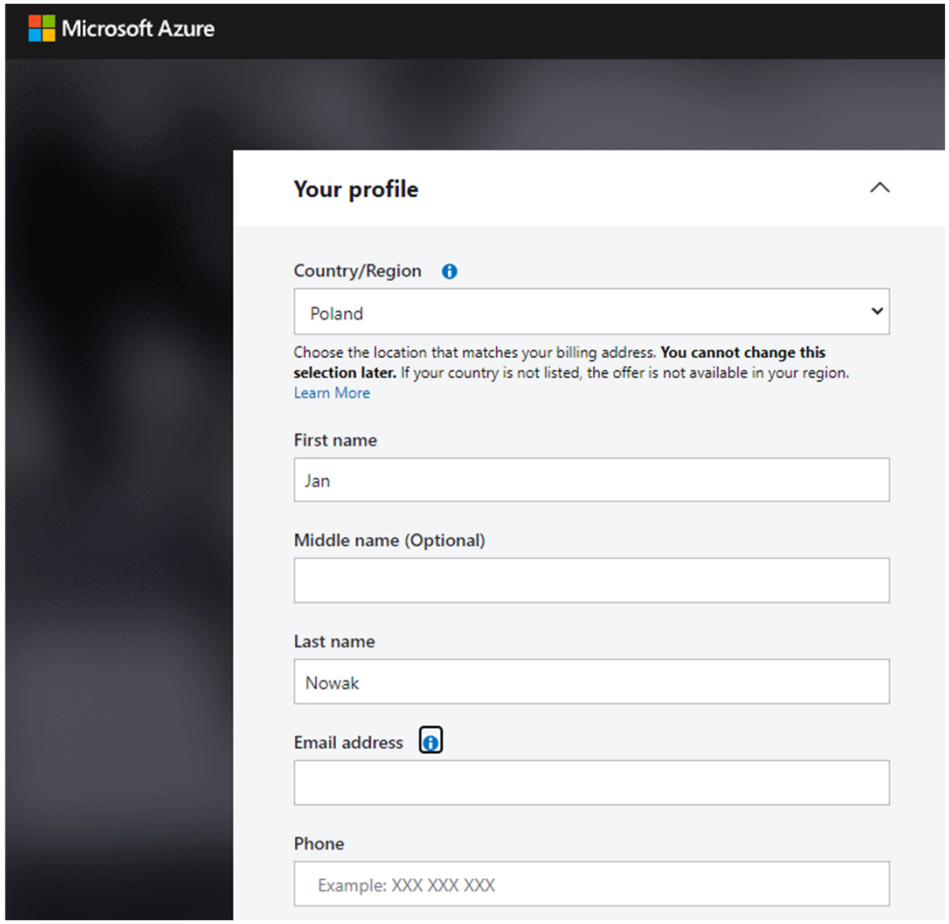


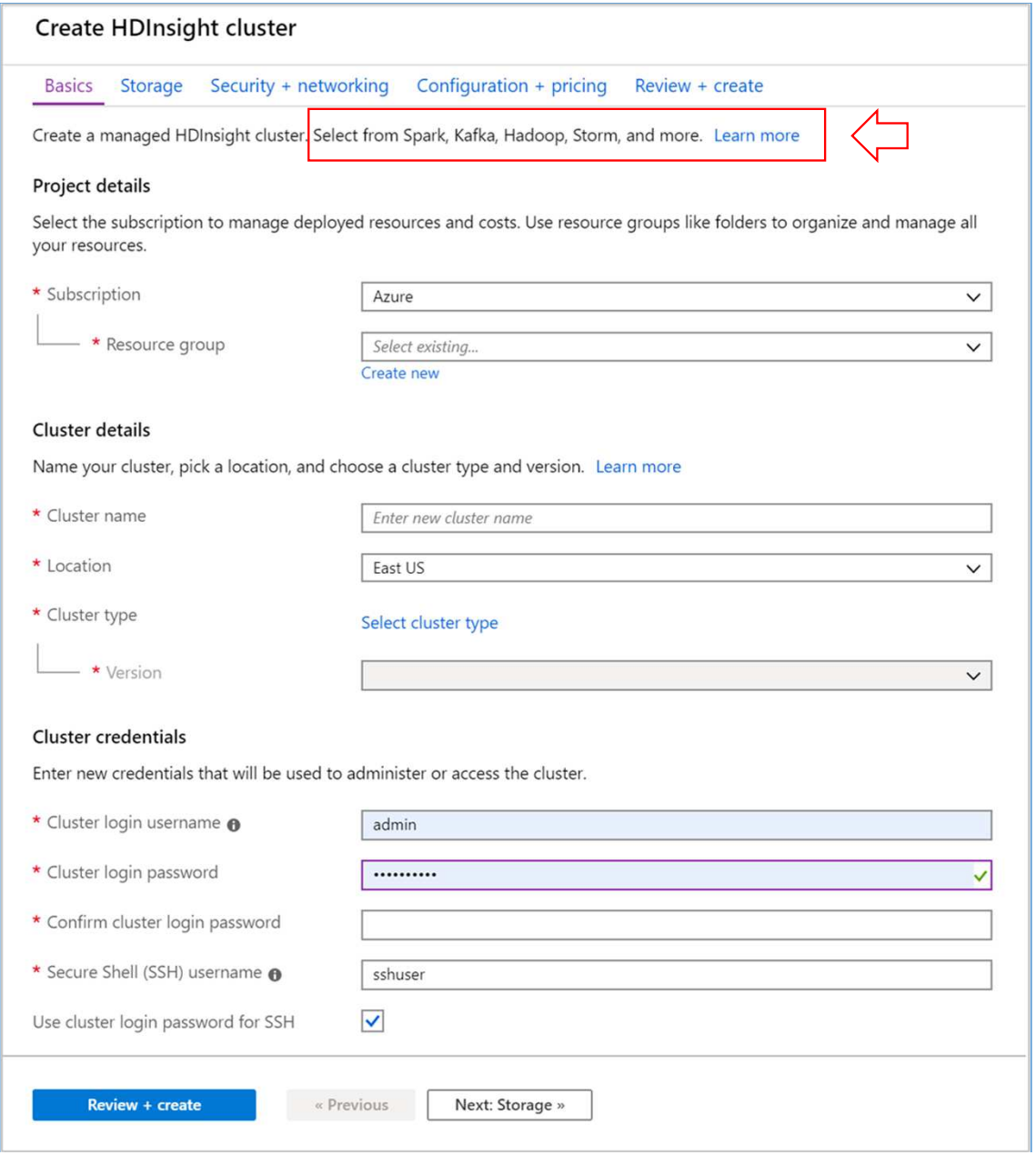
Tworzenie klastra Apache Hadoop w usłudze Azure HDInsight przy użyciu Azure Portal

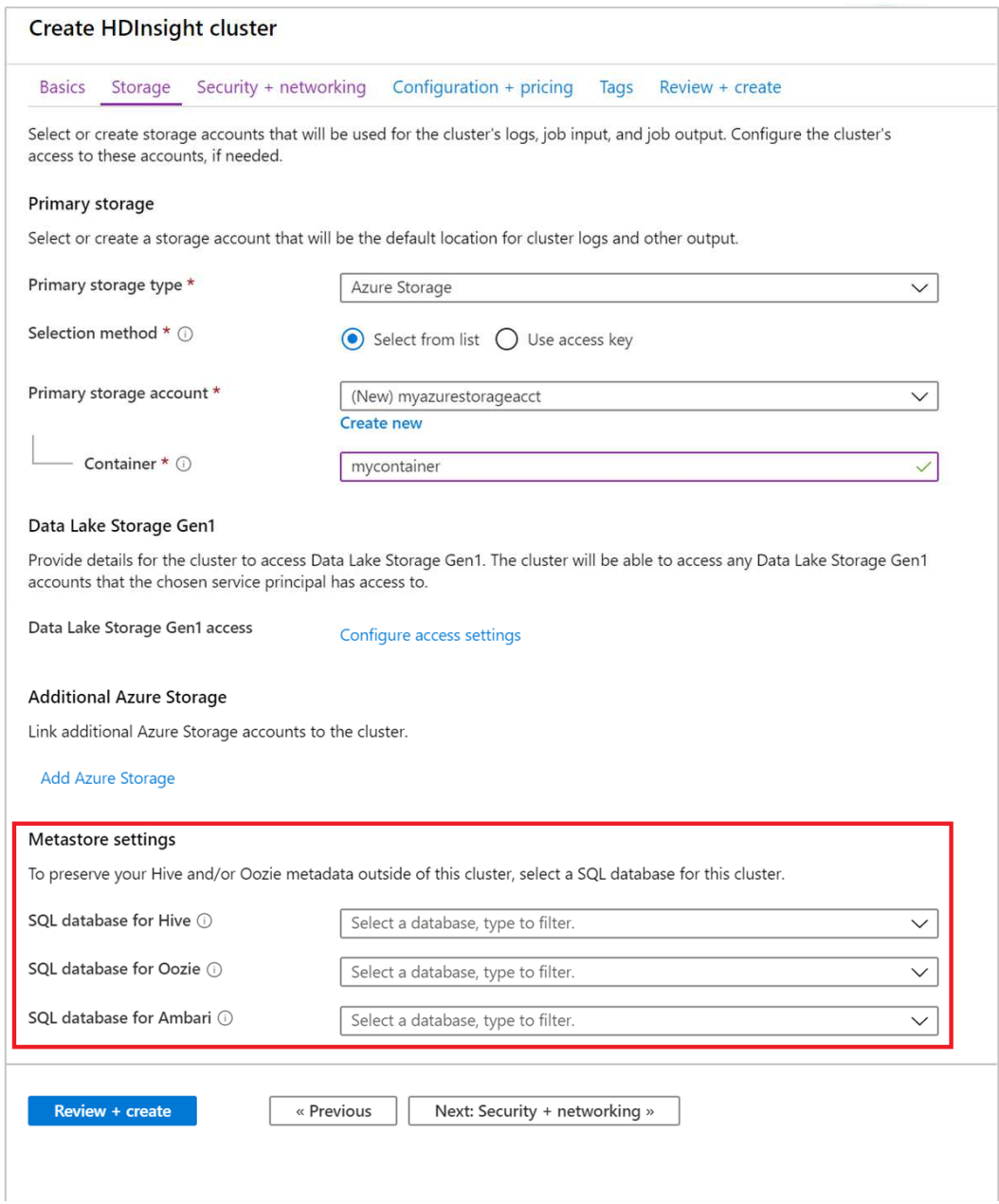


Procedura:

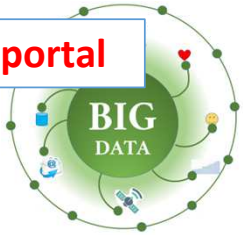





Create HDInsight cluster
Basics
Storage
Security + networking
Configuration + pricing
Review + create
Create a managed HDInsight cluster. Select from Spark, Kafka, Hadoop, Storm, and more. Learn more
Project details
Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.
* Subscription
Azure
* Resource group
Select existing...
Create new
Cluster details
Name your cluster, pick a location, and choose a cluster type and version. Learn more
* Cluster name
Enter new cluster name
* Location
East US
* Cluster type
Select cluster type
* Version
Cluster credentials
Enter new credentials that will be used to administer or access the cluster.
* Cluster login username
admin
* Cluster login password
* Confirm cluster login password
* Secure Shell (SSH) username
sshuser
Use cluster login password for SSH
Review + create
« Previous
Next: Storage »


Create HDInsight cluster
Basics
Storage
Security + networking
Configuration + pricing
Tags
Review + create
Select or create storage accounts that will be used for the cluster's logs, job input, and job output. Configure the cluster's access to these accounts, if needed.
Primary storage
Select or create a storage account that will be the default location for cluster logs and other output.
Primary storage type *
Azure Storage
Selection method *
Select from list
Use access key
Primary storage account *
(New) myazurestorageacct
Create new
Container *
mycontainer
Data Lake Storage Gen1
Provide details for the cluster to access Data Lake Storage Gen1. The cluster will be able to access any Data Lake Storage Gen1 accounts that the chosen service principal has access to.
Data Lake Storage Gen1 access
Configure access settings
Additional Azure Storage
Link additional Azure Storage accounts to the cluster.
Add Azure Storage
Metastore settings
To preserve your Hive and/or Oozie metadata outside of this cluster, select a SQL database for this cluster.
SQL database for Hive
Select a database, type to filter.
SQL database for Oozie
Select a database, type to filter.
SQL database for Ambari
Select a database, type to filter.
Review + create
« Previous
Next: Security + networking »

How to create an Apache Hadoop cluster in Azure HDInsight using the Azure portal



Create HDInsight cluster

Validation succeeded.

Basics | Storage | Security + networking | Configuration + pricing | Review + create

Hadoop 2.7.3 (HDI 3.6)

<price> USD Total estimated cost/hour

This estimate does not include subscription discounts or costs related to storage, networking, or data transfer.

Node usage fees are charged for as long as the cluster exists.

Billing starts when a cluster is created and stops when a cluster is deleted. **Clusters cannot be paused.**

Basics

Subscription	Azure
Resource group	(new) myResourceGroup
Location	East US
Cluster name	(new) myHadoop
Cluster type	Hadoop 2.7.3 (HDI 3.6)
Cluster login username	admin
Secure Shell (SSH) username	sshuser
Use cluster login password for SSH	Enabled

Storage

Primary storage type	Azure Storage
Primary storage account	(new) myhadoophdistorage
Container	myhadoop-2019-09-27t22-00-07-427z
Additional Azure storage	None
Data Lake Storage Gen1 access	Disabled

Cluster configuration

Head	2 nodes, D12 v2 (4 Cores, 28 GB RAM)
Worker	4 nodes, D4 v2 (8 Cores, 28 GB RAM)

Create | « Previous | Next | Download a template for automation

How to create an Apache Hadoop cluster in Azure HDInsight using the Azure portal



Region: Germany North | Currency: United States – Dollar ($) USD | Display pricing by: Hour

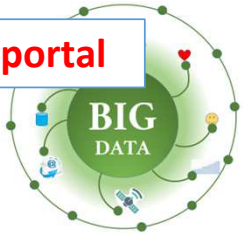
https://azure.microsoft.com/en-us/pricing/details/hdinsight/

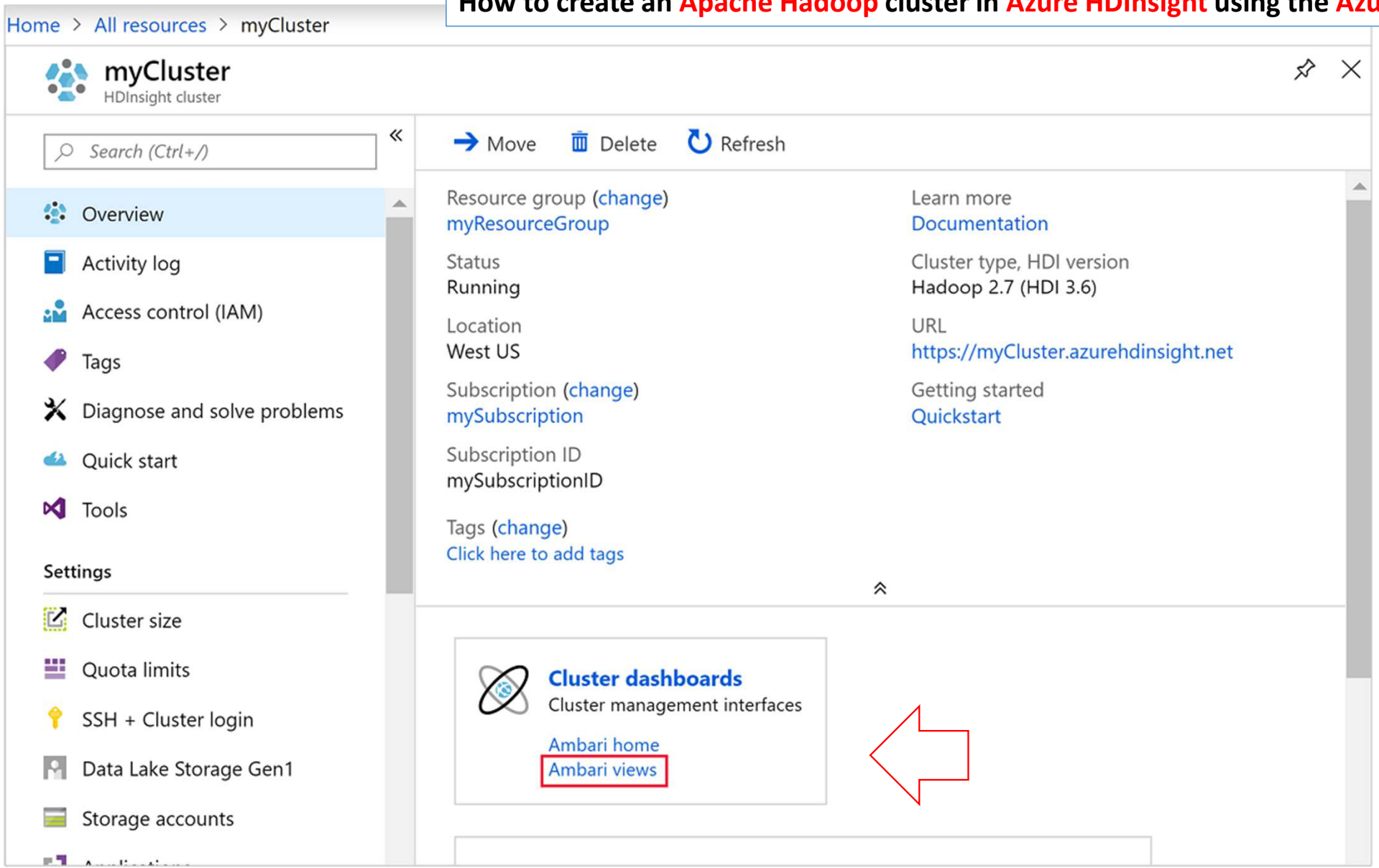
Memory Optimized nodes for HDInsight

Instance	vCPU(s)	RAM	OS	HDInsight Price	Total Price*
E2 v3	2	16 GB	$0.198/hour	$0.049/hour	$0.247/hour
E4 v3	4	32 GB	$0.395/hour	$0.099/hour	$0.494/hour
E8 v3	8	64 GB	$0.79/hour	$0.198/hour	$0.988/hour
E16 v3	16	128 GB	$1.581/hour	$0.395/hour	$1.976/hour
E20 v3	20	160 GB	$1.976/hour	$0.494/hour	$2.47/hour
E32 v3	32	256 GB	$3.162/hour	$0.789/hour	$3.951/hour
E64i v3	64	432 GB	$5.691/hour	$1.579/hour	$7.27/hour
E64 v3	64	432 GB	$5.691/hour	$1.579/hour	$7.27/hour

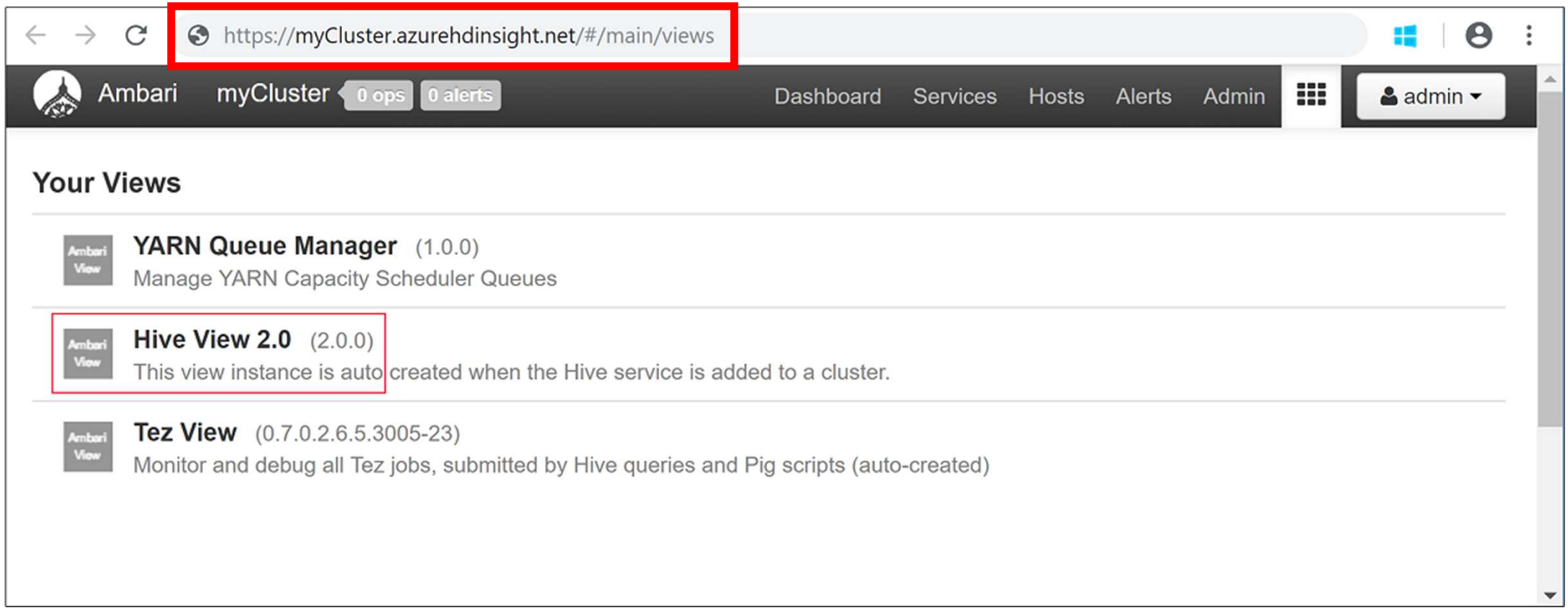
*Customers will continue to see one line item on their bill for Total Price

How to create an Apache Hadoop cluster in Azure HDInsight using the Azure portal

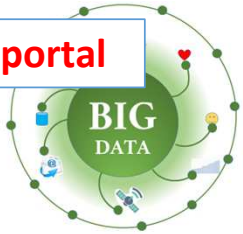


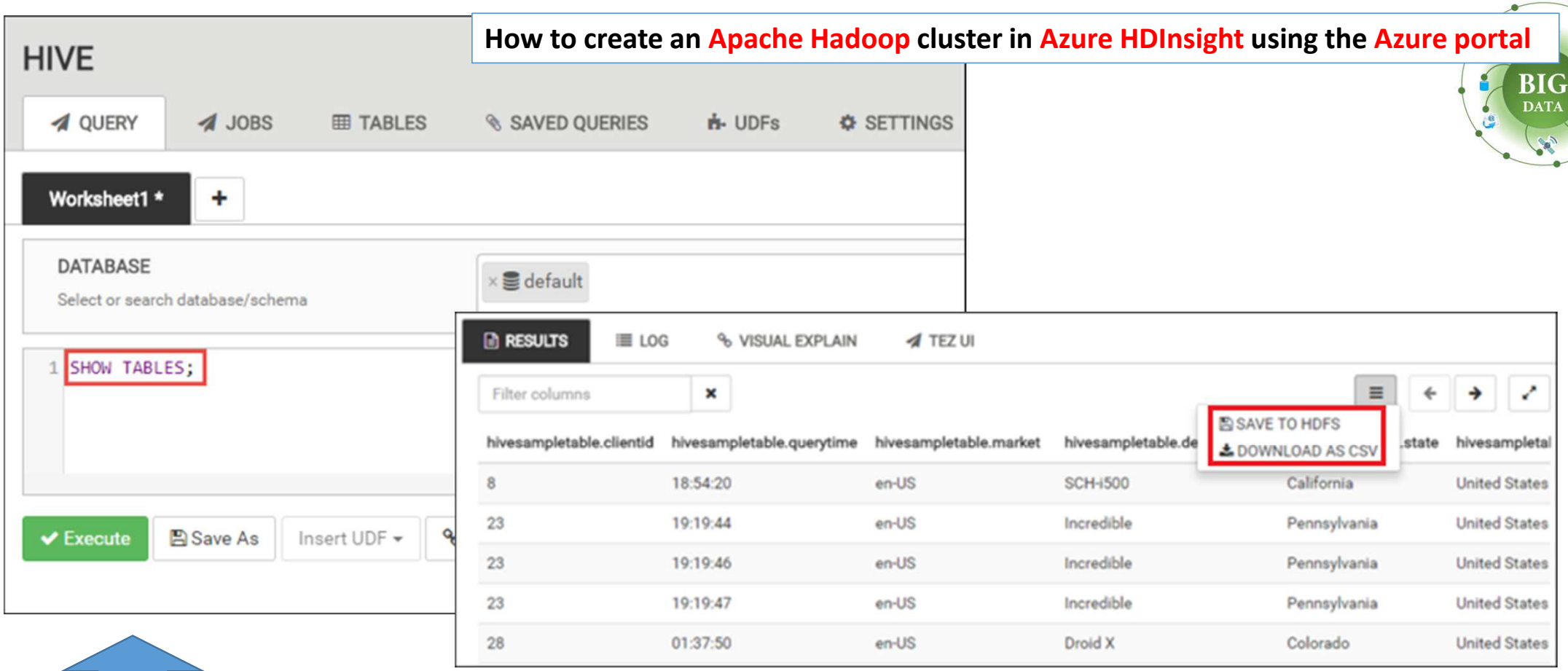


How to create an Apache Hadoop cluster in Azure HDInsight using the Azure portal



How to create an Apache Hadoop cluster in Azure HDInsight using the Azure portal





Uwaga !!!!

Widok Apache Hive nie jest dostępny w usłudze HDInsight 4.0.

Tak samo jak na maszynie Cloudera Hortonworks 3.0 (Uwaga wymagane 48GB RAM) → nowy interfejs **Ambari dla Hive** → bazuje na interfejsie terminalowym **Beeline**

What is Apache Hive and HiveQL on Azure HDInsight?

Apache Hive → is a data storage system for Apache Hadoop.

Hive lets you:

- query, and
- analyze your data.



Hive queries → are written in **HiveQL**, which is a query language **similar** to SQL.

How to use Hive?

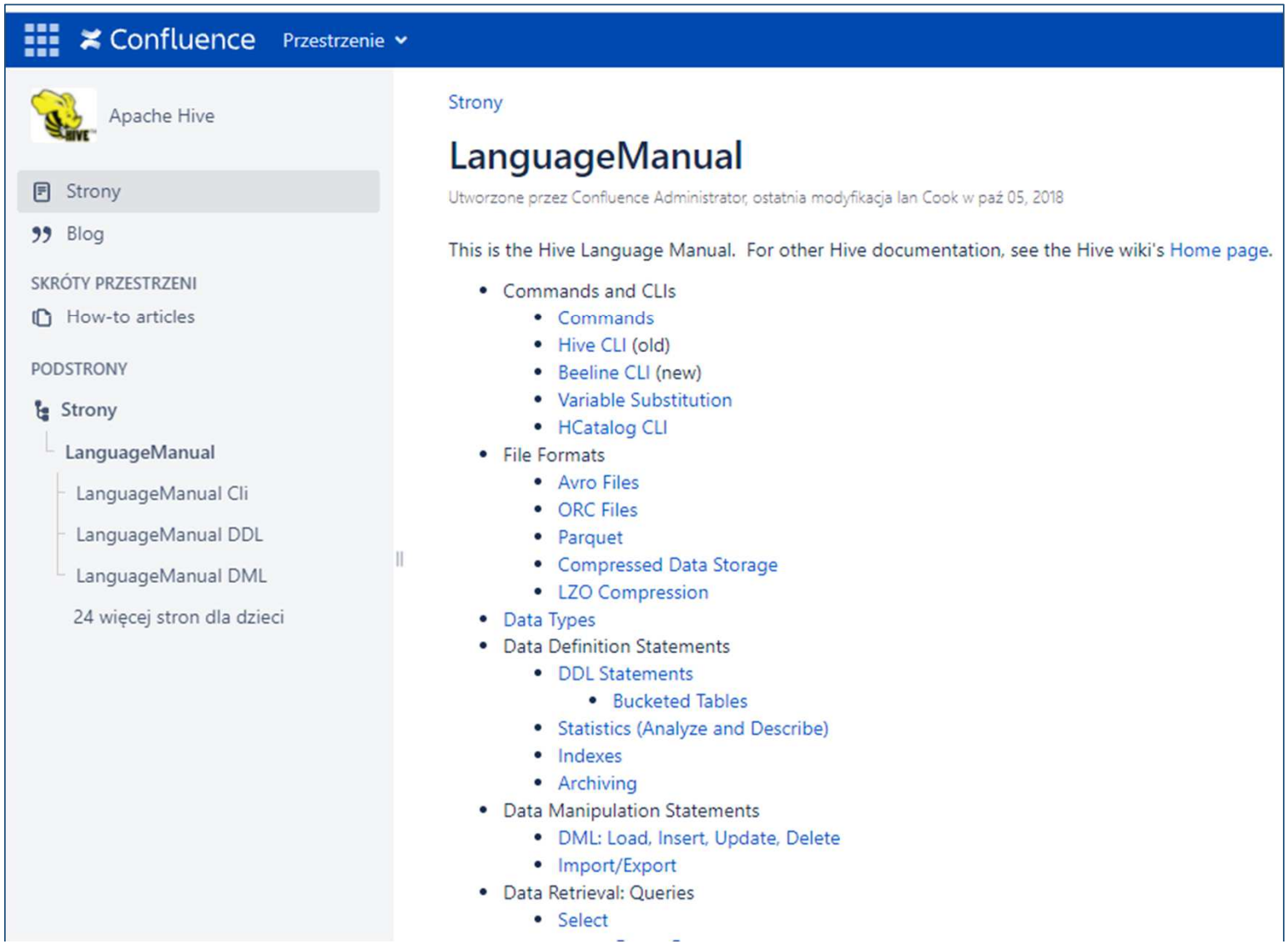
Skorzystaj z poniższej tabeli, aby poznać różne sposoby korzystania z technologii Hive z hdInsight:

Użyj tej metody, **jeśli chcesz...**	... zapytania interakcyjne	... przetwarzanie wsadowe	... **z tego** systemu operacyjnego klienta
Narzędzia HDInsight dla Visual Studio Code	✓	✓	Linux, Unix, Mac OS X lub Windows
Narzędzia HDInsight dla Visual Studio	✓	✓	Windows
widok programu Hive	✓	✓	Dowolne (oparte na przeglądarce)
Klient usługi Beeline	✓	✓	Linux, Unix, Mac OS X lub Windows
Interfejs API REST		✓	Linux, Unix, Mac OS X lub Windows
Windows PowerShell		✓	Windows

Informacje o języku HiveQL

https://cwiki.apache.org/confluence/display/Hive/LanguageManual



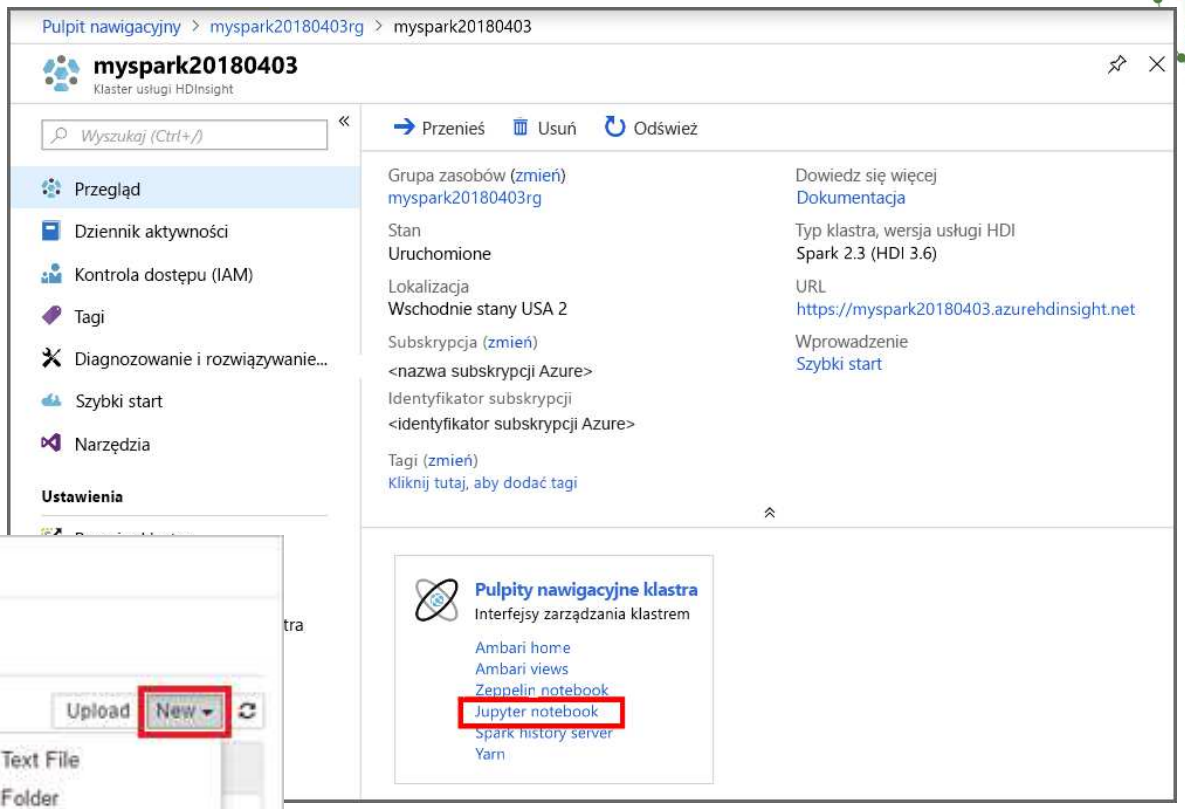


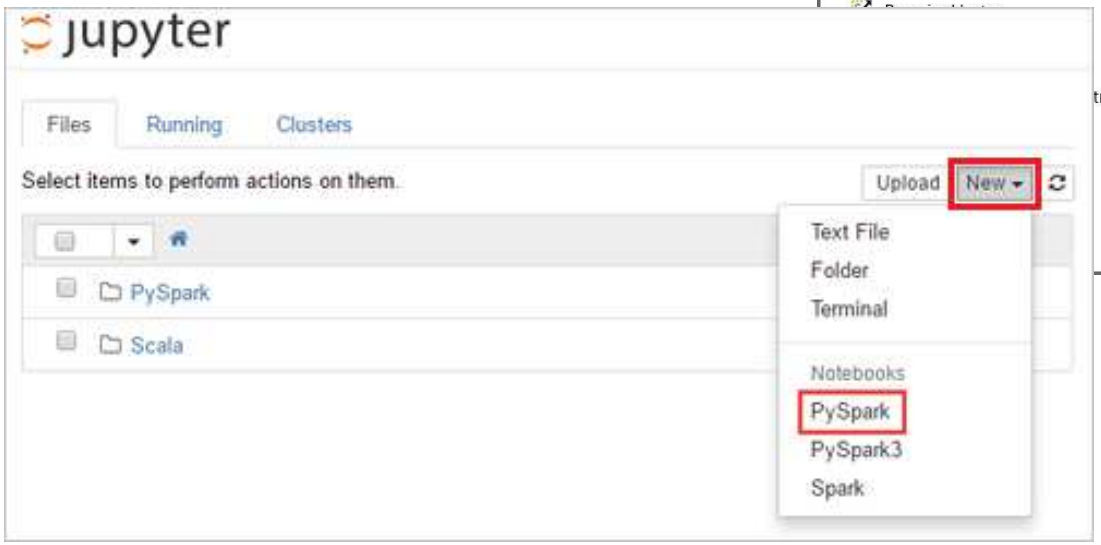


Apache Spark for cluster in Azure HDInsight

Apache Spark for cluster in Azure HDInsight

Tworzenie pliku notesu **Jupyter Notebook**





Hadoop

Hive → is a module developed by **Facebook** (Meta Platforms) that runs Hadoop in a data warehouse along with a SQL dialect called **HiveQL**

data processing → **via SQL dialect,** → **HiveQL** → **jest językiem deklaratywnym,**

meaning it describes the desired outcome (goal) and Hive calculates how to build the data flow to achieve it.

(unlike Pig where you had to build a data flow).

Also unlike Pig (where a schema is required), it is not limited to a single schema.

Like SQL, HiveQL is a stand-alone relational language, but not a Turing-complete language.

It can be extended by a UDF, fulfilling these Turing assumptions.

Hadoop can handle unstructured data, semi-structured data, and highly structured data. Hive is focused on structured data and uses Hadoop to handle large amounts of data provided.

Hadoop

Pig

developed at **Yahoo** Research in 2006 and commissioned by the Apache Software Foundation in 2007.

Pig language

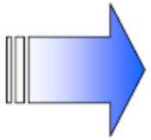
- **.. is called Pig Latin**
- **is the language of data flow**
- **- is a type of language in which elements (nodes) are combined together in a program;**
- **- each node transforms data that is used by subsequent elements and instructions.**

Pig może działać na złożonych strukturach danych, nawet tych, które mogą mieć wiele poziomów zagnieżdżania;

The motto for the PIG is: → **" Pigs eat everything "**

so Pig - accepts any input format

Agenda



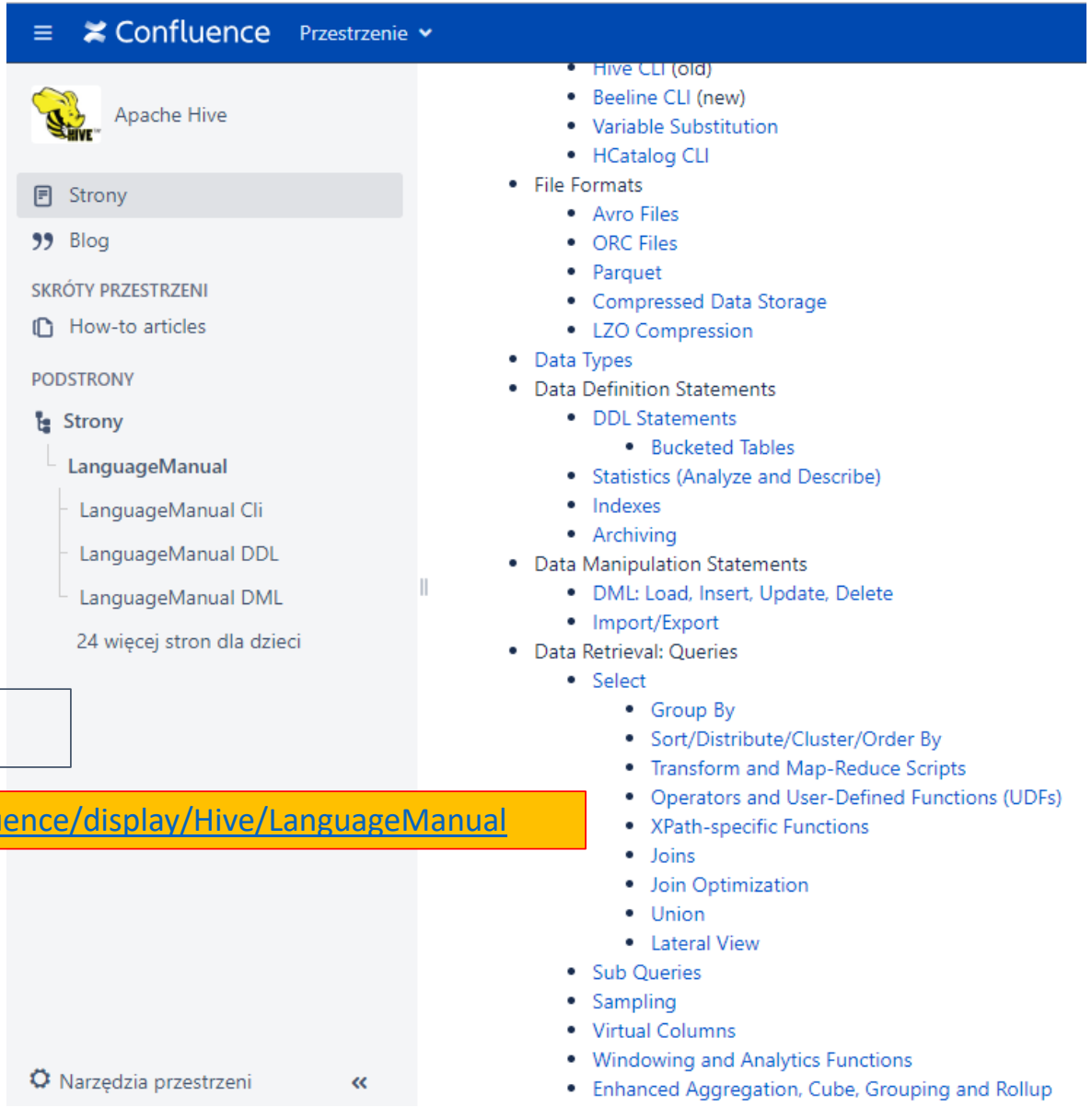
- Overview
- Pig
- **Hive**
- Jaql

Hive - is the technology that transforms Hadoop into a data warehouse, using the SQL dialect (HiveQL) in which it can be queried.

Hive

Hive documentation page:

https://cwiki.apache.org/confluence/display/Hive/LanguageManual



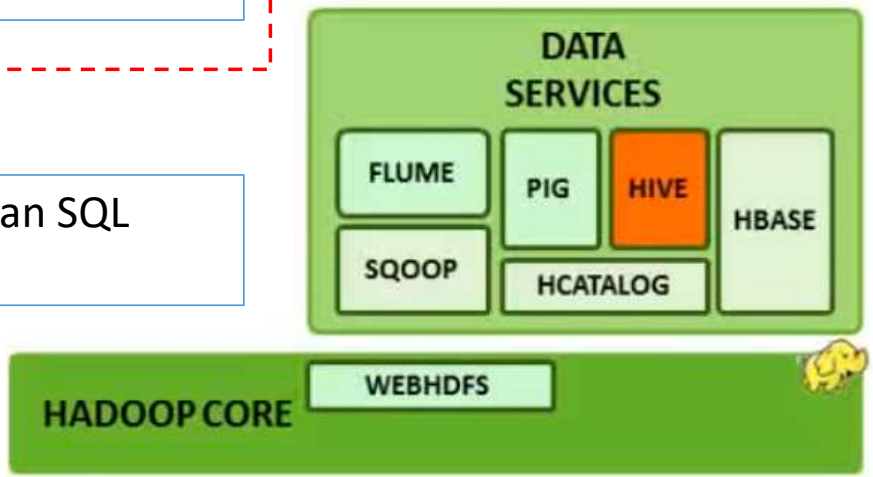
Hive

What is Hive?

is a data warehouse-like layer based on Hadoop

allows you to define a structure for unstructured Big Data

simplifies analysis and querying using an SQL scripting language called HiveQL



Hive is Not…

…a relational database

- Hive uses a database to store metadata, but the data that Hive processes is stored in HDFS

…designed for on-line transaction processing

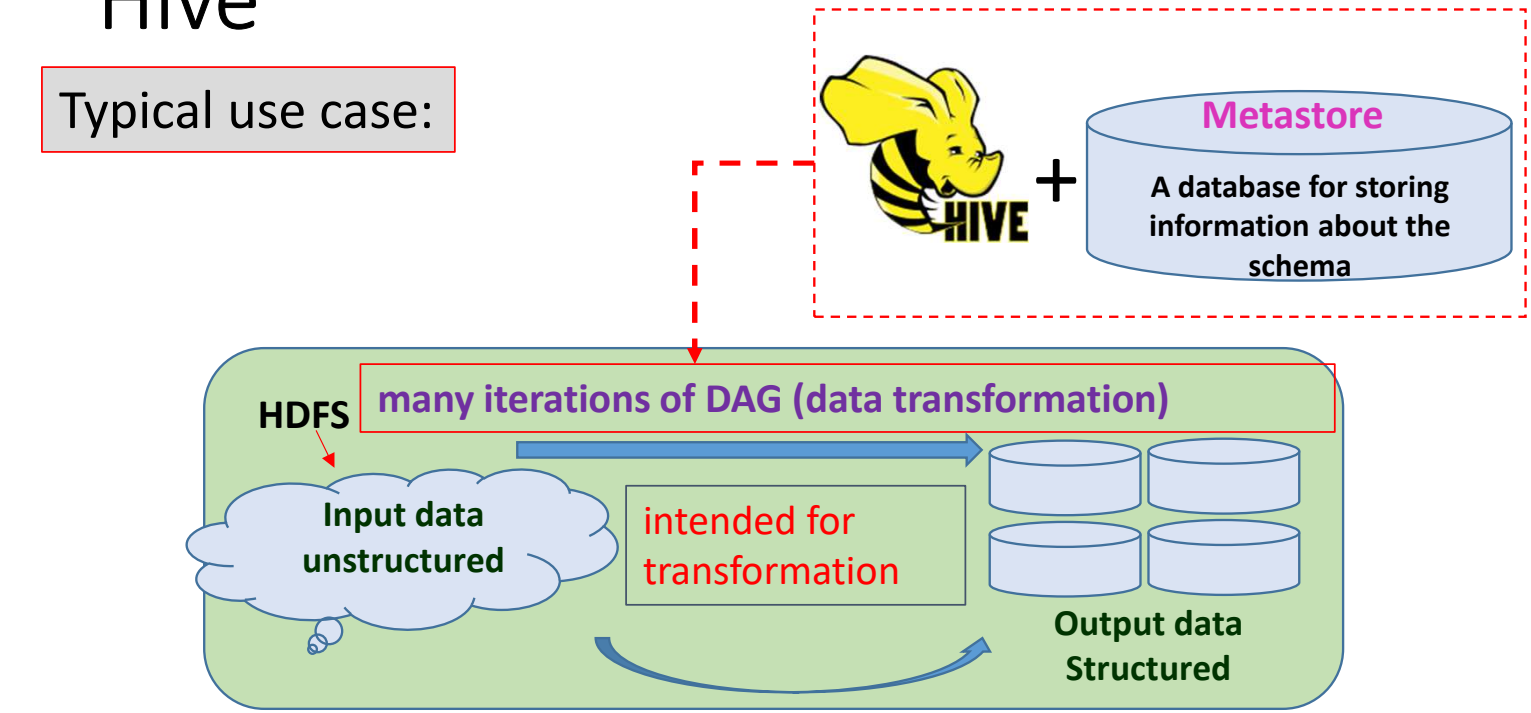
- Hive runs on Hadoop (a batch-processing system where jobs can have high latency with substantial overhead)
- Therefore, latency for Hive queries is generally high (even for small jobs)

…suited for real-time queries and row-level updates

- Hive is best used for batch jobs over large sets of immutable data (such as web logs)

Hive

Typical use case:



Supports uses like:

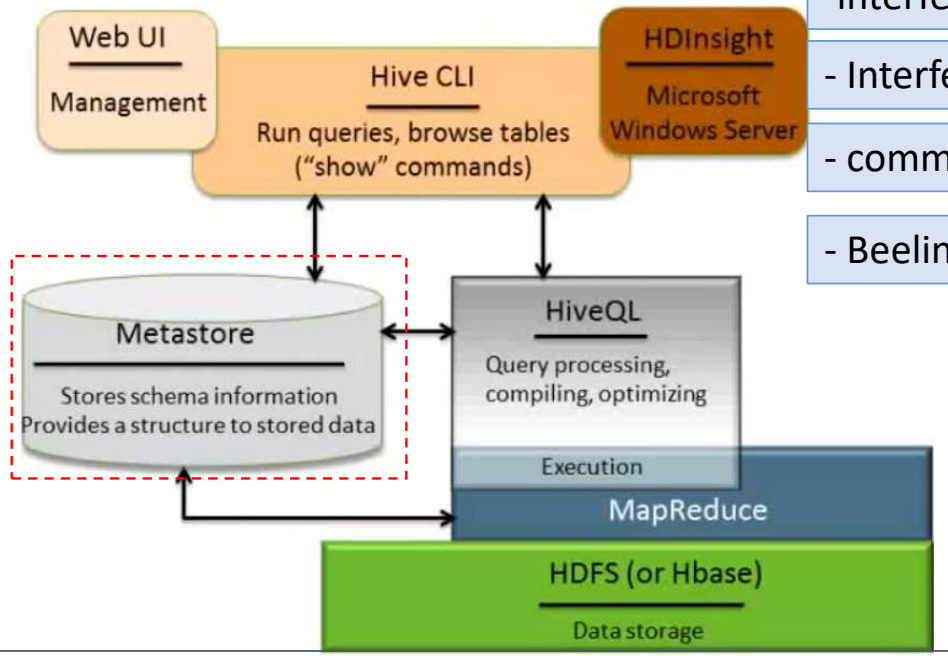
- Ad-hoc queries
- aggregations
- data analysis

QL can also be extended with custom scalar user functions (UDFs), aggregate functions (UDAFs) and tabular functions (UDTFs)

Hive Architecture Basics

Praca w środowisku Hive możliwa jest przy użyciu jednej z następujących metod:

System for querying and managing structured data



-interfejsu JDBC/ODB

- Interfejsu Web UI

- command line interface (CLI) (old)

- Beeline – Command Line Shell (CLI)

Interfejsy dodtępu - mogą być użyte do wydawania zapytań SQL lub DDL. Hive jest przeznaczony do wyszukiwania danych strukturalnych i wykorzystuje wiele koncepcji z relacyjnych baz danych (np. tabele, wiersze, kolumny, schematy itp.). Na potrzeby tych obiektów baza danych Hive przechowuje informacje o schemacie w tzw. "**metastore**".

„**Metastore**" - to komponent Hive, który zwykle jest przechowywany w relacyjnej bazie danych, domyślnie Apache Derby, choć może to być dowolna relacyjna baza danych. Składuje on informacje o schemacie danych i zapewnia strukturę do składowania danych

Na końcu - wszystko jest konwertowane na zadania DAG lub MapReduce, które jest przetwarzane przez klaser Hadoop.

Hadoop

Running Hive

Można współdziałać z Hive przy użyciu jednej z następujących metod:

- **Hive Shell**
  - **Interactive**

    ```
    hive
    ```
  - **Script**

    ```
    hive -f myscript
    ```
  - **Inline**

    ```
    hive -e 'SELECT * FROM mytable'
    ```

Istnieją trzy sposoby uruchamiania Hive z CLI:

- można uruchomić interaktywnią powłokę **hive** bez polecenia.

- można uruchomić **skrypt** Hive, przekazując atrybut **-f** do polecenia **hive** wraz ze ścieżką do pliku skryptu.

- można uruchomić program Hive jako jedno polecenie, przekazując opcję **-e** do polecenia **hive**, a następnie program **Hive** w cudzysłowie.

Hive services

hive --service *servicename*

where *servicename* can be:

- hiveserver
  server for Thrift, JDBC, ODBC clients
- hwi
  web interface
- jar
  hadoop jar with Hive jars in classpath
- metastore
  out of process metastore

Hive Metastore - przechowuje metadane o strukturze danych

Hive obsługuje także usługi uruchamiania z polecenia Hive:

- można uruchomić usługę Hive przez sterownik ODBC, lub JDBC przez przekazanie do serwisu komendy Hive ,z użyciem słowa **hiveserver**.

- Istnieje również interfejs WWW do utworzenia zapytań

- można również użyć usługi Hive, aby uruchomić polecenie hadoop z opcją jar

Hive - Metastore

- **Stores Hive metadata**
- **Configurations**
  - **Embedded**

    in-process metastore, in-process database
  - **Local**

    in-process metastore, out-of-process database
  - **Remote**

    out-of-process metastore, out-of-process database

Hive Metastore - stores Hive metadata.

there are three **Metastore** configurations available:

Embedded - runs **Metastore** code in the same process with Hive,

Local - stores the **Metastore** code with the process, but moves the database to a separate process that the Metastore communicates with.

Remote - is to move the **Metastore** itself out of the process. It is useful if you plan to share Metastore with other users.

Hive – Hive allows to define a schema for your data sets

Hive – has a read-oriented schema (Hive – Schema-On-Read)



Hive – has a read-oriented schema (Hive – **Schema-On-Read**)

…in most databases, the schema is used **when data is loaded** into the database.

or in other words, most database records require a schema definition.

Data loaded into the database must comply with the schema.

Hive's approach is to defer applying the schema until you are trying to read the data.

This has both advantages and disadvantages:



The benefits are **that data loading is faster** and **there is the flexibility** to use multiple schemas for the same data.

The downside is that **queries are slower**

Hive Query Language - HiveQL

HiveQL functionalities:

Hive Query Language or **HiveQL** is a dialect of SQL.

- uses known concepts from databases (tables, rows, columns and schemas)
- it does not fully support the SQL92 specification and is similar to the MySQL dialect
- można wykonać instrukcję INSERT OVERWRITE TABLE ale nie można wykonywać UPDATE lub DELETE.

- no transactions and indexes.
- there is no support for correlated subqueries, materialized views, stored procedures.

Supports multi-table inserts

- enables access to Big Data through tables

Converts SQL queries into DAG (TEZ) or Map - Reduce jobs

- the user does not need to know anything about DAG or Map - Reduce operations

Supports user script plugins for executing DAG and Map Reduce scripts

Like Pig - Hive has both simple types such as INT and STRING and complex types ARRAY, MAP and STRUCT.

HiveQL

ceil
ceiling
coalesce
collect_list
collect_set
compute_stats
concat
concat_ws
context_ngrams
conv
corr
cos
count
covar_pop
covar_samp
crc32
create_union
cume_dist
current_database
current_date
current_timestamp
current_user
date_add
date_format
date_sub
datediff
day
dayofmonth
decode
degrees
dense_rank
div

The command : SHOW FUNCTIONS

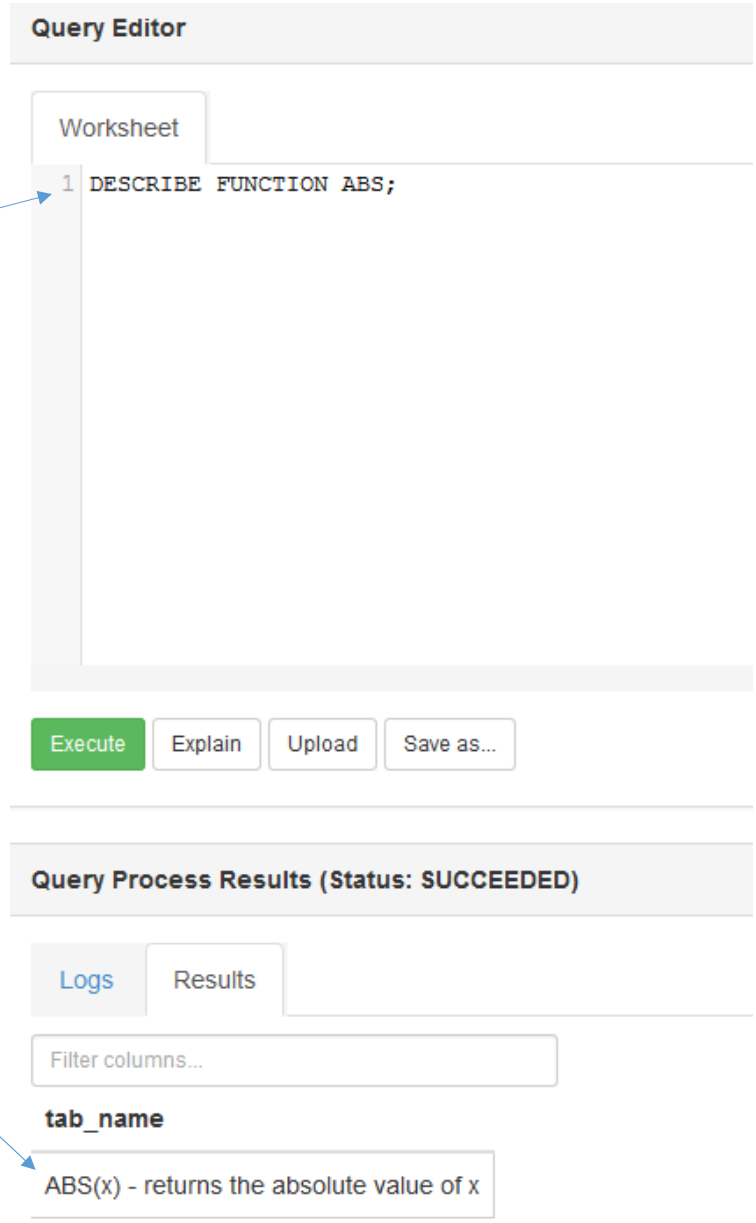


tan
to_date
to_unix_timestamp
to_utc_timestamp
translate
trim
trunc
ucase
unbase64
unhex

xpath_float
xpath_int
xpath_long
xpath_number
xpath_short
xpath_string
year
|
~

HiveQL

Information about individual features can be found by typing: DESCRIBE FUNCTION, and then the name of the function in the Hive shell.



HIVE tabels:

What are HIVE tables?:

- A Hive table consists of:
  - Data: typically a file or group of files in HDFS
  - Schema: in the form of metadata stored in a relational database
- Schema and data are separate
  - A schema can be defined for existing data
  - Data can be added or removed independently
  - Hive can be "pointed" at existing data
- You have to define a schema if you have existing data in HDFS that you want to use in Hive

Hive - tabele

Like other SQL databases, Hive uses tables.

There are two types of tables you can create:

- internal tables - when loading a file into a managed table - Hive moves the file to its data containers. When the DROP statement is run (i.e. the resource is deleted) - both data and metadata will be deleted.

- external tables - it's metadata for data. Deleting a table only deletes the metadata. The data stays where it is.

Hive supports and uses tables of the type:

Apache ORC - is a fast columnar storage format for workloads in Hadoop.

Link do opisu: https://orc.apache.org/

https://cwiki.apache.org/confluence/display/Hive/LanguageManual+ORC

The **O**ptimized **R**ow **C**olumnar (new project Apache ORC) - is a file format that provides a very efficient way of storing data for the Hive system. It is designed to overcome the limitations of other Hive file formats.

To create a table using the ORC file format:

```
CREATE TABLE <tablename> ... STORED AS ORC ...
```

Hive

Advantages of storing in ORC format:

- single data file for each task, which reduces the load on the **NameNode**
- supports new data types: datetime, decimal i typów złożonych (struct, list, map i union)
- has lightweight indexes stored in the file
- allows you to skip groups of rows that do not pass predicate filtering
- allows search operations in a given row
- there is block mode compression based on data type
- there is dictionary encoding for string columns
- allows for simultaneous parallel reading of the same file using separate RecordReaders
- allows splitting files without scanning tags
- metadata stored using Protocol Buffers, which allow adding and removing fields

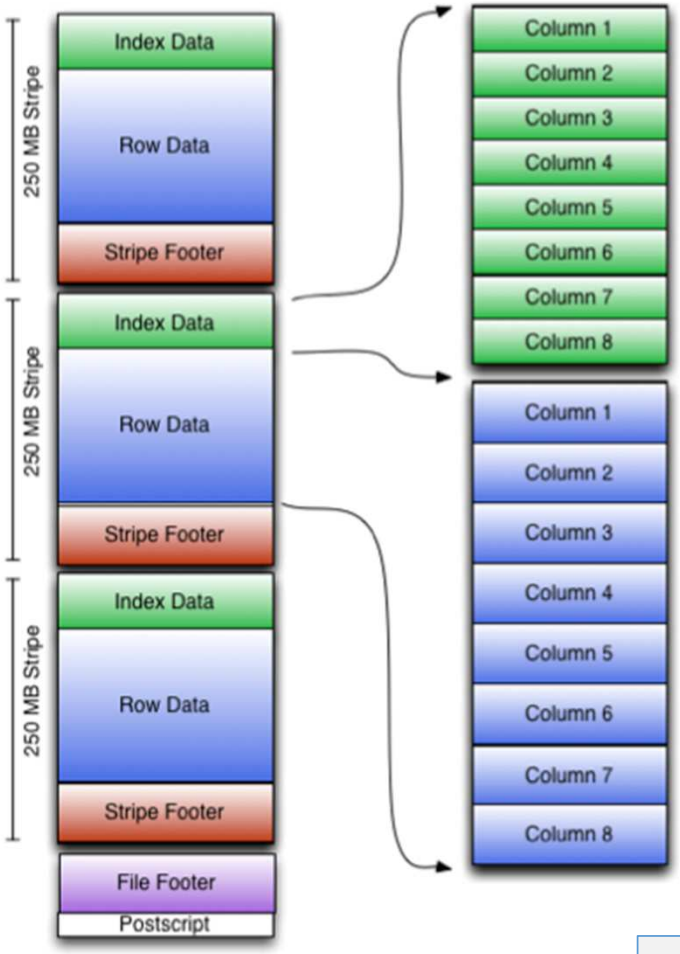
Hive

Struktura pliku ORC:

Plik ORC zawiera grupy danych wiersza tzw: „paski”, wraz z informacjami pomocniczymi w stopce pliku. Na końcu pliku postscript posiada parametry kompresji i rozmiar skompresowanej stopki.

Domyślny rozmiar paska wynosi 250 MB. Duże rozmiary pasków umożliwiają duże i wydajne odczyty z HDFS.

Stopka pliku zawiera listę pasków w pliku, liczbę wierszy na pasek i typ danych każdej kolumny. Zawiera również liczbę agregatów na poziomie kolumny, min, maks i sumę.



www.hive.com

Hadoop

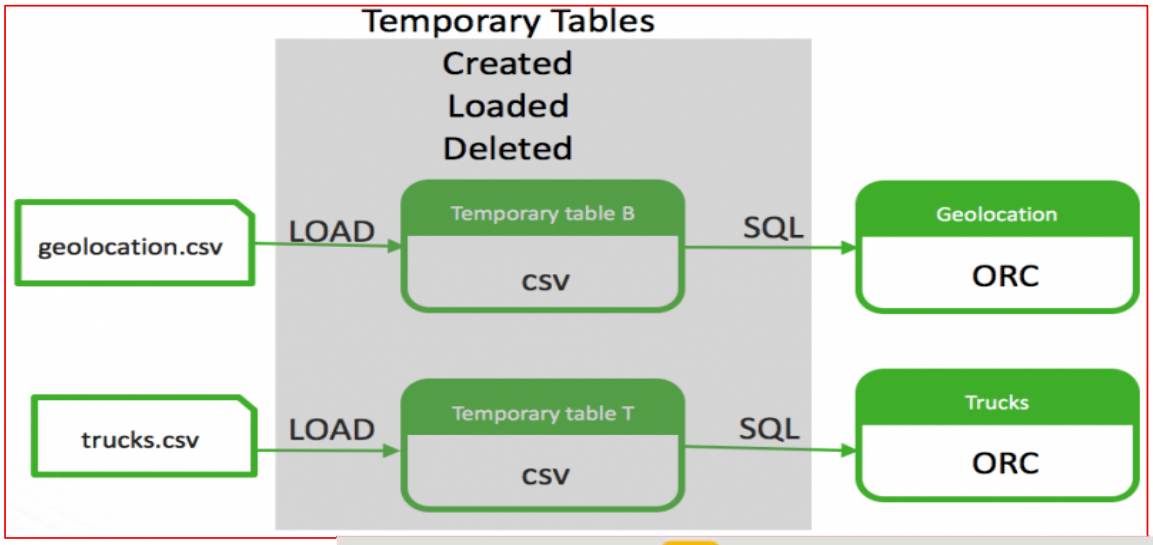
Defining a Table

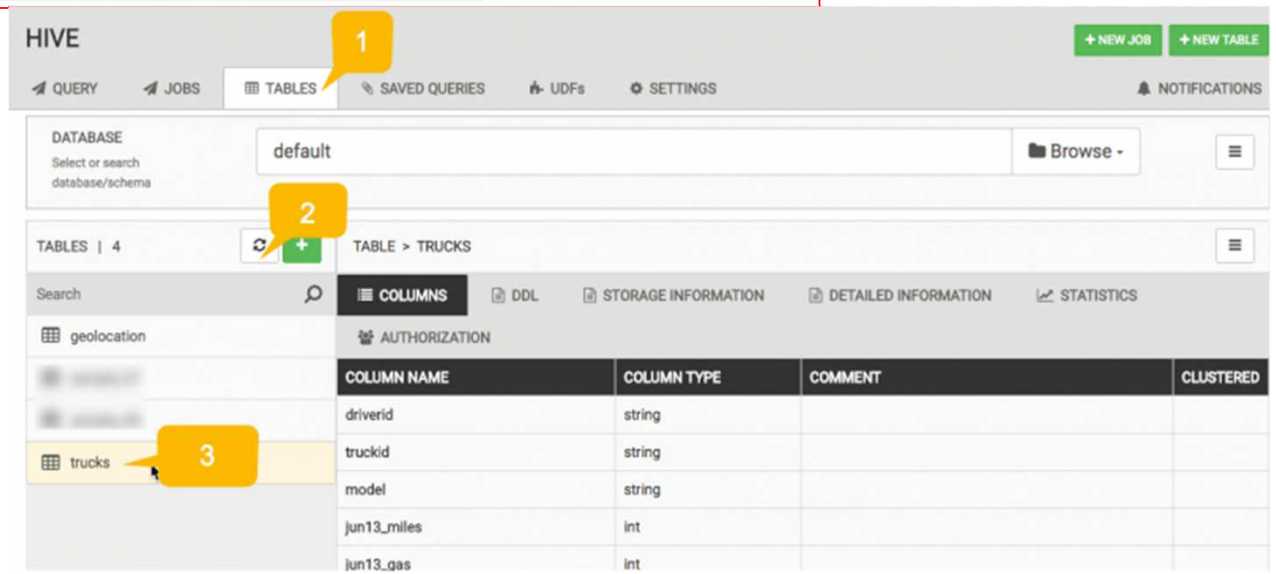
```
hive> CREATE TABLE mytable (name
string, age int)
   ROW FORMAT DELIMITED
   FIELDS TERMINATED BY ';'
   STORED AS TEXTFILE;
```

Each row is comma delimited text

HiveQL statements are terminated with a semicolon

Hive


Temporary Tables
Created
Loaded
Deleted
geolocation.csv
LOAD
Temporary table B
csv
SQL
Geolocation
ORC
trucks.csv
LOAD
Temporary table T
csv
SQL
Trucks
ORC


HIVE
1
+ NEW JOB
+ NEW TABLE
QUERY
JOBS
TABLES
SAVED QUERIES
UDFs
SETTINGS
NOTIFICATIONS
DATABASE
Select or search database/schema
default
Browse
2
TABLES | 4
TABLE > TRUCKS
Search
COLUMNS
DDL
STORAGE INFORMATION
DETAILED INFORMATION
STATISTICS
AUTHORIZATION
geolocation
trucks
3
COLUMN NAME
COLUMN TYPE
COMMENT
CLUSTERED
driverid
string
truckid
string
model
string
jun13_miles
int
jun13_gas
int

Hive

Hive – Partycjonowanie:

- może przyspieszyć niektóre zapytania

- dzieli dane na podstawie kolumny partycji

  można dzielić dane na podstawie wartości kolumny.

-podczas tworzenia tabeli wykorzystuje się klauzulę PARTITION BY

Podczas tworzenia tabeli można określić klauzulę PARTITION BY w celu określenia kolumny służącej do podziału danych.

- podczas ładowania danych wykorzystuje się klauzulę PARTITION

Następnie, podczas ładowania danych, określa się klauzulą PARTITION, która partycja jest ładowana.

Managing Tables

Operation	Command Syntax
See current tables	`hive> SHOW TABLES`
Check the schema	`hive> DESCRIBE mytable;`
Change the table name	`hive> ALTER TABLE mytable RENAME to mt;`
Add a column	`hive> ALTER TABLE mytable ADD COLUMNS (mycol STRING);`
Drop a partition	`hive> ALTER TABLE mytable DROP PARTITION (age=17)`

Hive

Other Key HiveQL Operations

Loading Data

- Use LOAD DATA to import data into a Hive table

```
hive> LOAD DATA LOCAL INPATH 'input/mydata/data.txt'
INTO TABLE myTable;
```

Indicate data path

- The files are not modified by Hive – they are loaded in as is
- Use the word OVERWRITE to write over a file of the same name
- Hive warehouse default location is:
  - `/hive/warehouse`
- Hive can read all of the files in a particular directory
- The schema is checked when the data is queried
  - If a row does not match the schema, it will be read as null

Loading Data Examples

Wskazywany jest lokalny system plików

```
LOAD DATA LOCAL INPATH
'/tmp/customers.csv' OVERWRITE INTO
TABLE customers;
```

Wskazywany jest lokalny system HDFS

```
LOAD DATA INPATH
'/user/train/customers.csv'
OVERWRITE INTO TABLE customers;
```

```
INSERT INTO birthdays SELECT
firstName, lastName, birthday FROM
customers WHERE birthday IS NOT
NULL;
```

INSERT

do zapełniania danymi

- Use INSERT statement to populate data into a table from another Hive table
- Since query results are usually large it is best to use an INSERT clause to tell Hive where to store your query
- Example: creating a table and inserting data into it

```
hive> CREATE TABLE age_count  (name string, age int);
hive> INSERT OVERWRITE TABLE age_count
        SELECT age, COUNT(age)
        FROM mytable;
```

INSERT OVERWRITE

- OVERWRITE is used to replace the data in the table, otherwise the data is appended to the table.
  - Append happens by adding files to the directory holding the table data

```
INSERT OVERWRITE newtable SELECT * FROM mytable;
```

- Can write to a directory in HDFS

Zapisywanie bezpośrednio do HDFS

```
INSERT OVERWRITE DIRECTORY '/hdfs/myresult' SELECT
* FROM mytable;
```

- Can write to a local directory

Zapisywanie do lokalnego systemy plików

```
INSERT OVERWRITE LOCAL DIRECTORY...
```

Performing Queries

- SELECT
  - Simple query example:

```
SELECT * FROM mytable;
```

- Supports the following:
  - WHERE clause ← wspierana jest klauzula warunkowa (operacja map)
  - UNION ALL and DISTINCT
  - GROUP BY and HAVING
  - LIMIT clause
    - Rows returned are chosen at random
  - Can use REGEX Column Specification

```
SELECT '(ds|hr)?+.+' FROM sales;
```

Performing Queries Examples

- `SELECT * FROM customers;`
- `SELECT COUNT(1) FROM customers;`
- `SELECT firstName, lastName, address, zip FROM customers WHERE orderID > 0 GROUP BY zip;`
- `SELECT customers.*, orders.* FROM customers JOIN orders ON (customers.customerID = orders.customerID);`
- `SELECT customers.*, orders.* FROM customers LEFT OUTER JOIN orders ON (customers.customerID = orders.customerID);`

Subqueries

- Hive supports subqueries only in the FROM clause
- For example:

UWAGA !!!
Podzapytania dostępne są po słowie FROM

```
SELECT total FROM
    (SELECT c1 + c2 AS total FROM
mytable) my_query;
```

Must name subquery

- The columns in the subquery SELECT list are available in the outer query

Hive – Inner Join

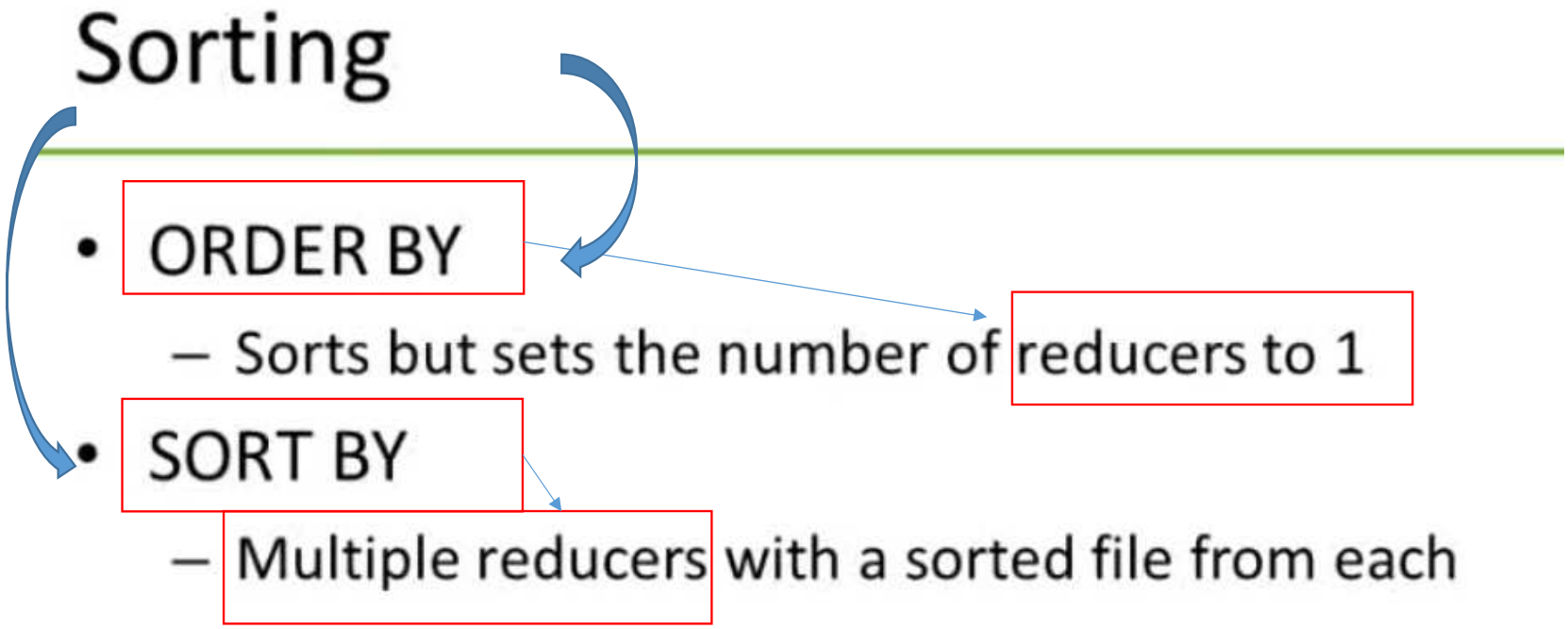
Sprzężenie INNER JOIN – część wspólna dwóch tabel (zwraca wiersze, które mają ze sobą część wspólną

```
SELECT * FROM students;
    Steve     2.8
    Raman     3.2
    Mary      3.9
SELECT * FROM grades;
    2.8       B
    3.2       B+
    3.9       A
 SELECT students.*, grades.*
   FROM students JOIN grades ON
(students.grade = grades.grade)
       Steve     2.8  2.8   B
       Raman     3.2  3.2   B+
       Mary      3.9  3.9   A
```

Hive – external join

- Allows for the finding of rows with non-matches in the tables being joined
- Three types of Outer Joins:
  - LEFT OUTER JOIN
    - Returns a row for every row in the first table entered
  - RIGHT OUTER JOIN
    - Returns a row for every row in the second table entered
  - FULL OUTER JOIN
    - Returns a row for every row from both tables

Hive - sorting



Hive Tutorial Overview

Przykład sekwencji zdarzeń:

Load files into HDP

- Load batting.csv and master.csv into your sandbox

Create table

- Create table called **temp_batting** (empty at this point)

Load data into Hive/HDFS

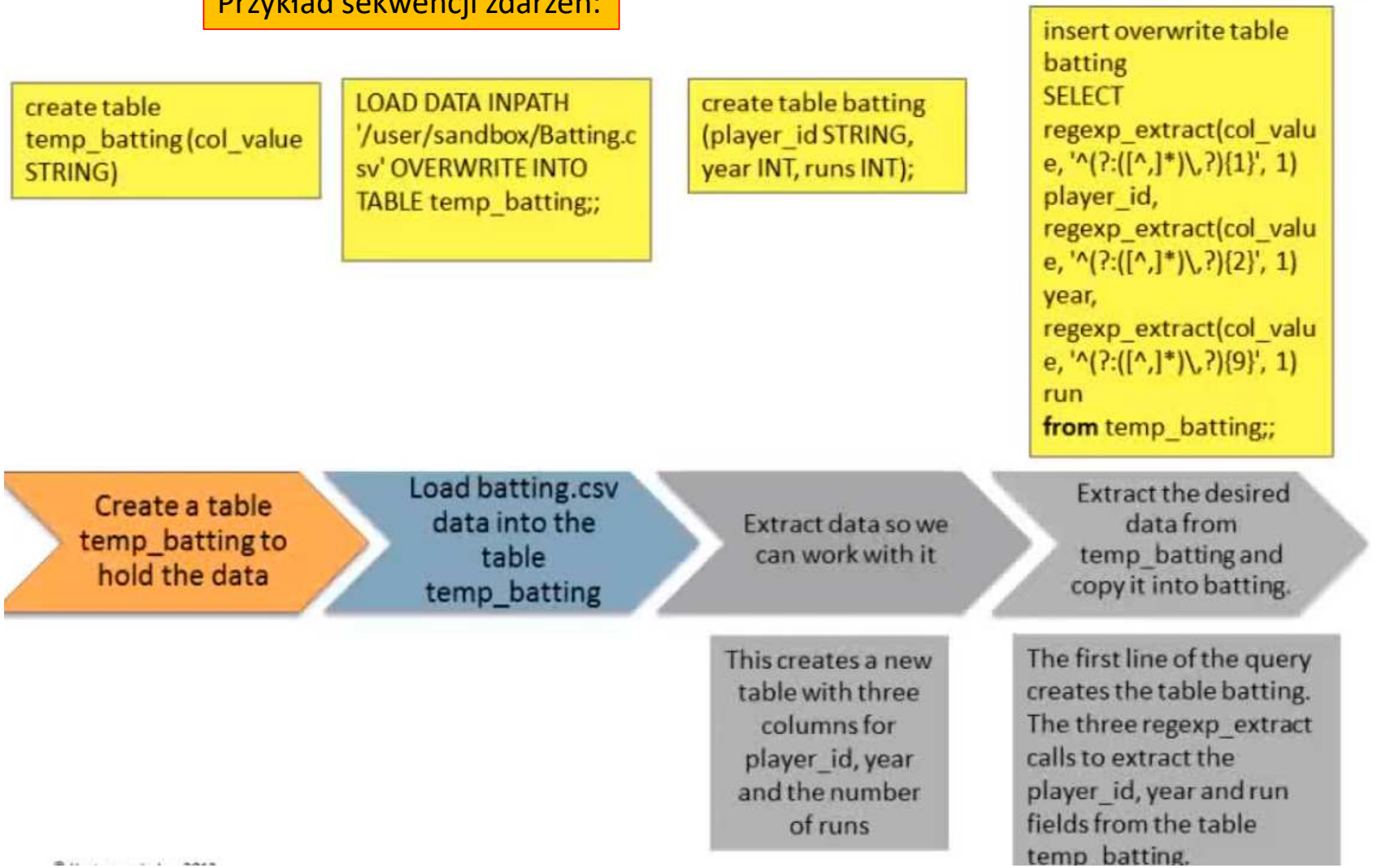
- Load the data file Batting.csv into the table temp_batting

Query Data

- Extract data to find player with highest runs for the year (1871-2011)
- Extend script to translate player ID field into first and last names of the players

Hive Tutorial Steps

Przykład sekwencji zdarzeń:



Hive Tutorial Steps (Cont.)

```
SELECT year, max(runs) FROM
batting GROUP BY year
```

```
SELECT a.year, a.player_id, a.runs from
batting a
JOIN (SELECT year, max(runs) runs FROM
batting GROUP BY year ) b
ON (a.year = b.year AND a.runs = b.runs) ;
```

Group data by year

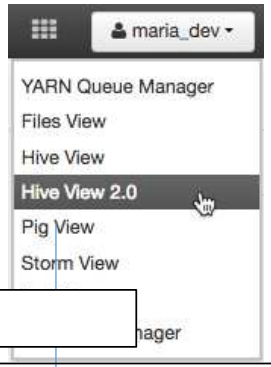
Get the player id

	year	_c1
0	NULL	NULL
1	1871	66
2	1872	94
3	1873	125
4	1874	91
5	1875	115
6	1876	126
7	1877	68

	year	player_id	runs
0	1871	barnero01	66
1	1872	eggleda01	94
2	1873	barnero01	125
3	1874	mcveyca01	91
4	1875	barnero01	115
5	1876	barnero01	126
6	1877	orourji01	68
7	1878	highadi01	60

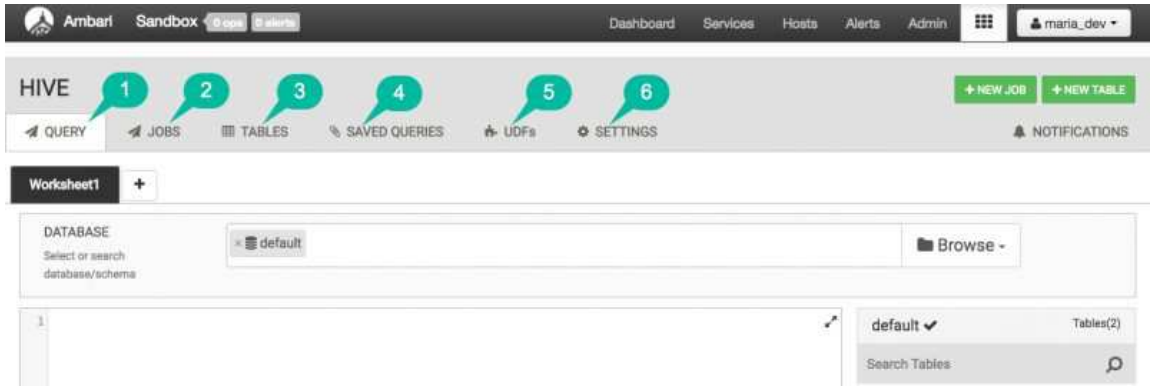
AMBARI HIVE VIEW 2.0 ON SANDBOX

Zapewnia interaktywny interfejs do Hive 2.
Możemy tworzyć, edytować, zapisywać i uruchamiać zapytania, a Hive używaja MapReduce lub zadań Tez.



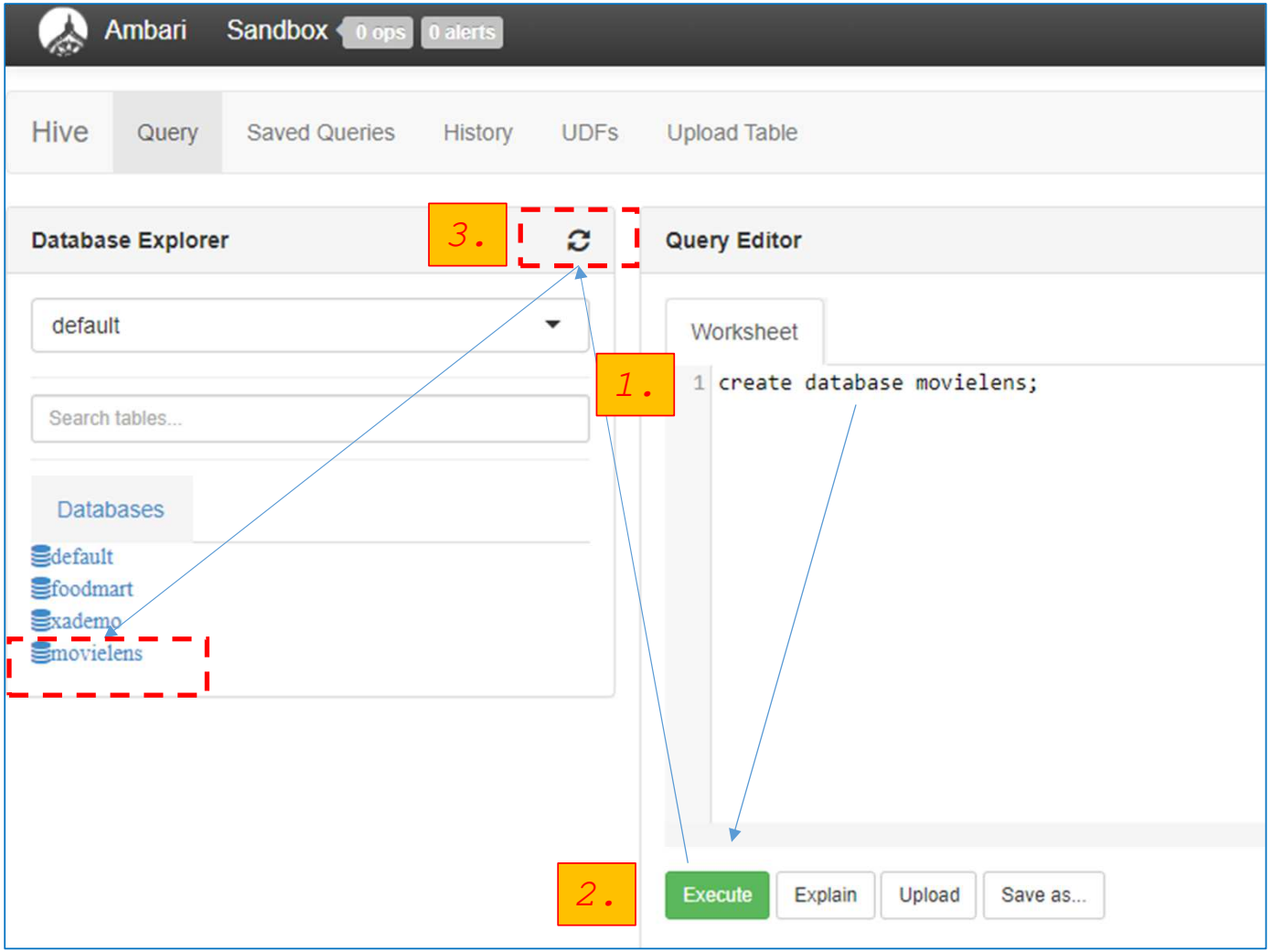
W Hive View 2.0 znajduje się 6 kart:

1. **QUERY**: - podstawowy interfejs do zapisywania, edycji i wykonywania nowych instrukcji SQL.
2. **JOBS**: - pozwala zobaczyć przeszłe i aktualnie uruchomione zapytania.
3. **TABLES**: stanowi jedno centralne miejsce do przeglądania, tworzenia, usuwania i zarządzania tabelami.
4. **SAVED QUERIES**: Wyświetl zapytania zapisane przez bieżącego użytkownika.
5. **UDFs**: Funkcje zdefiniowane przez użytkownika (UDF) można dodawać do zapytań, wskazując na plik JAR w systemie HDFS i wskazując ścieżkę klasy Java, która zawiera definicję UDF. Po dodaniu UDF w programie Query Editor pojawia się przycisk Wstaw UDF, który umożliwia dodanie UDF do zapytania.
6. **SETTINGS**: Umożliwia modyfikację ustawień mających wpływ na zapytania wykonane w widoku Hive.

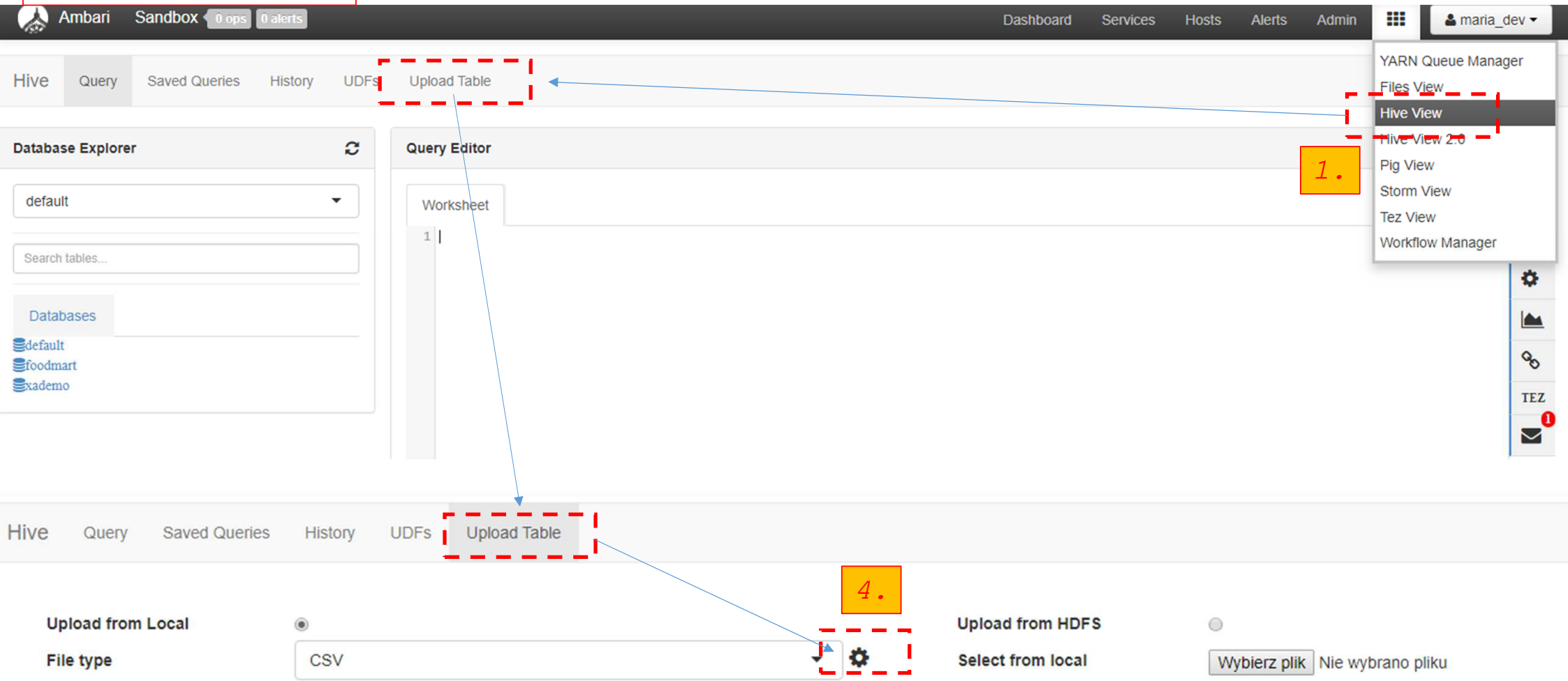


Apache Hive - examples

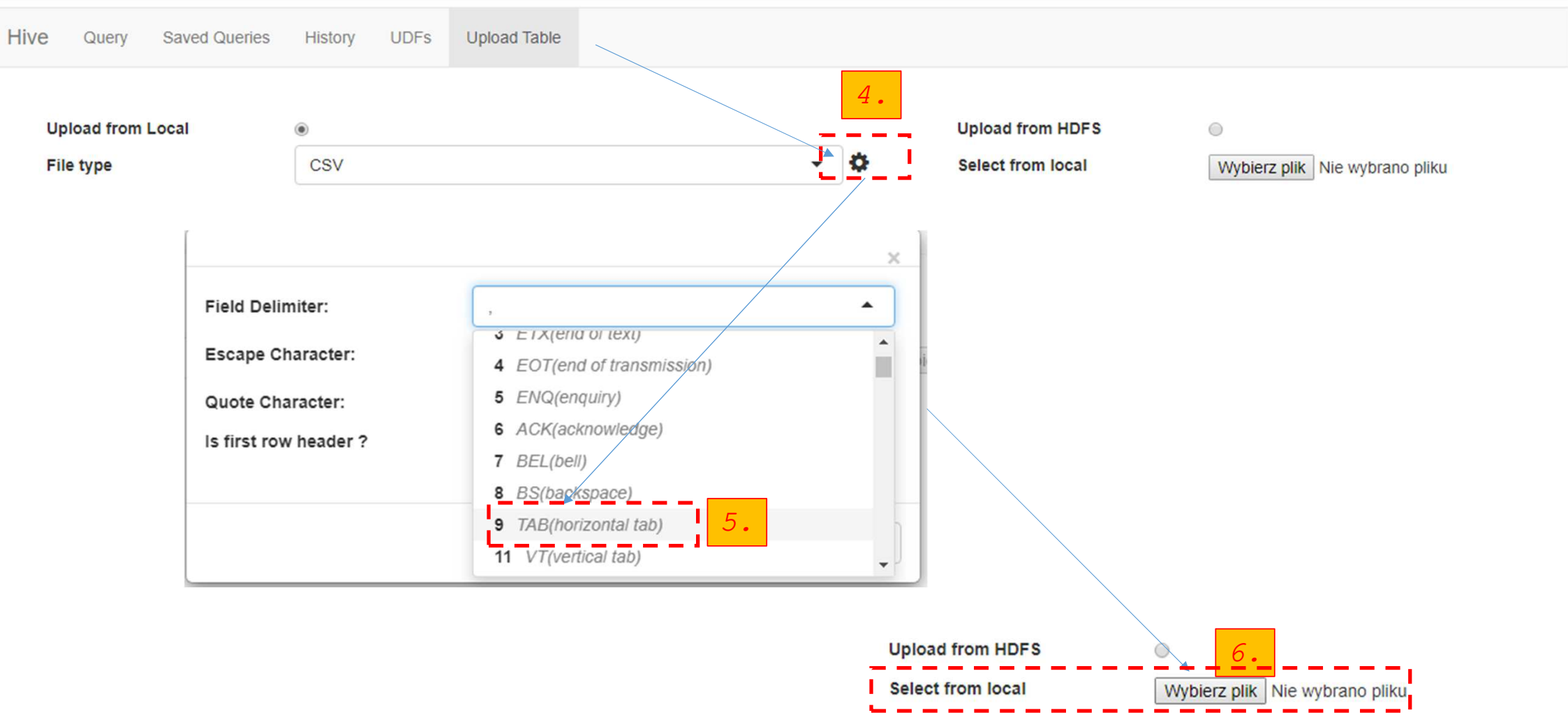
Film data analysis in Hive: ⇨ defining the database: ⇨ create database movielens;



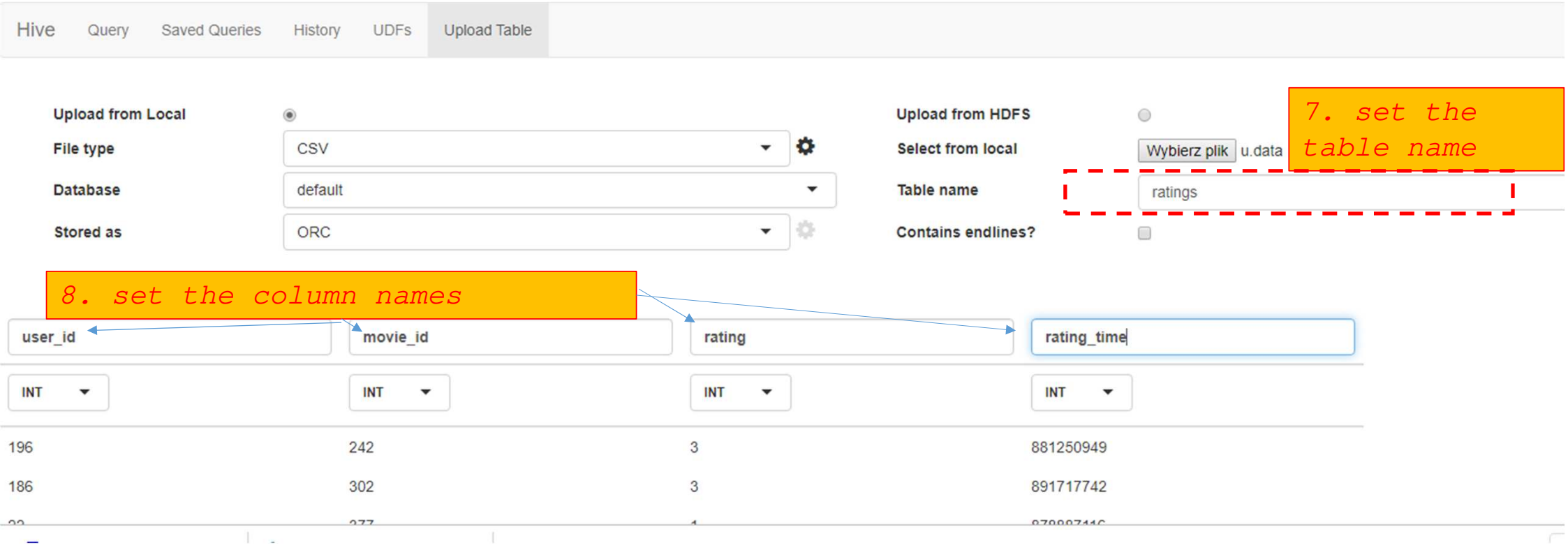
Film data analysis in Hive:



Film data analysis in Hive:



Film data analysis in Hive:



… and confirm with the button: **upload**

Similarly, we import **movie** names:

File: u.item



9. set the table name

Upload from Local

File type: CSV

Database: default

Stored as: ORC

Upload from HDFS

Select from local: Wybierz plik u.item

Table name: movie_names

Contains endlines?

10. set the column names

movie_id	name	produce_date	column4	column5
INT	STRING	STRING	STRING	STRING
1	Toy Story (1995)	01-Jan-1995		http://us.imdb.com/M/title-exact?Toy%20Story%20(1995)
2	GoldenEye (1995)	01-Jan-1995		http://us.imdb.com/M/title-exact?

.. and confirm with the button: **upload**

ATTENTION!!!

how to set the engine for MR tasks:

The Tez engine runs 10x faster than classic MR tasks (because it creates a DAG) - however, for an existing BLADE virtual environment it may not work better with a disk array controller!!!



how to set the engine for MR tasks:

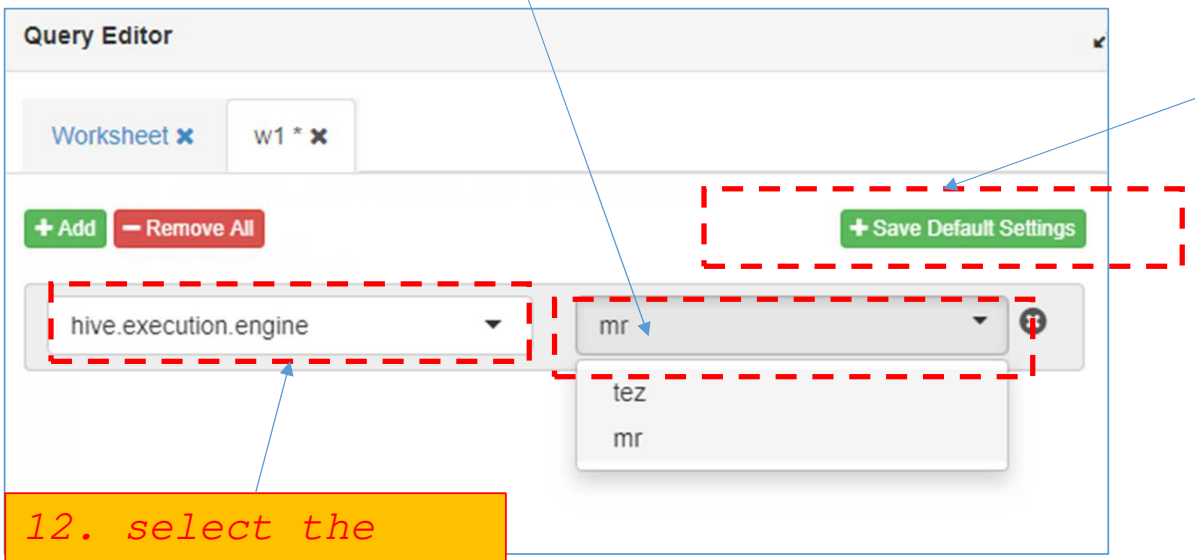
ATTENTION!!!

The Tez engine runs 10x faster than classic MR tasks (because it creates a DAG) - however, for an existing BLADE virtual environment it may not work better with a disk array controller!!!

13. choose mr option

MANDATORY!!!!

14. save the settings



12. select the option for hive processing engine

We write quiries:

Count how many votes were cast (give the number of individual votes cast) for a given rating (for the rating range from 1 to 5)?

or otherwise:
Are people more likely to give positive or negative ratings to movies?

We write quiries:

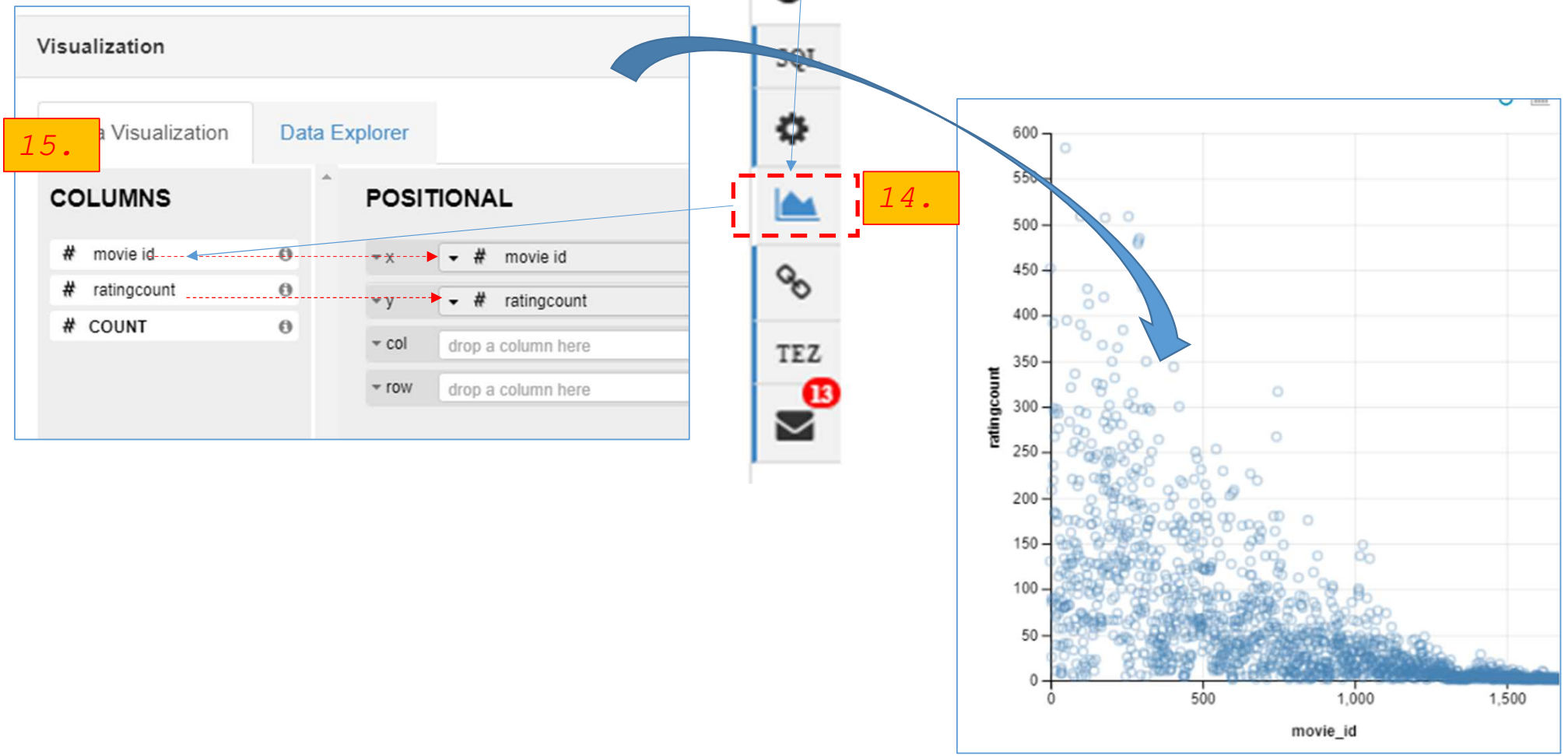
What was the most popular movie - i.e. with the most ratings?

…otherwise …
…. count the number of ratings for each movie?
…. What movie do people go to the cinema to see most often?

We write quiries:

.. next visualize the obtained results on a chart:

Results:



We write quiries:

Results:

Execute | Explain | Upload | Save as...

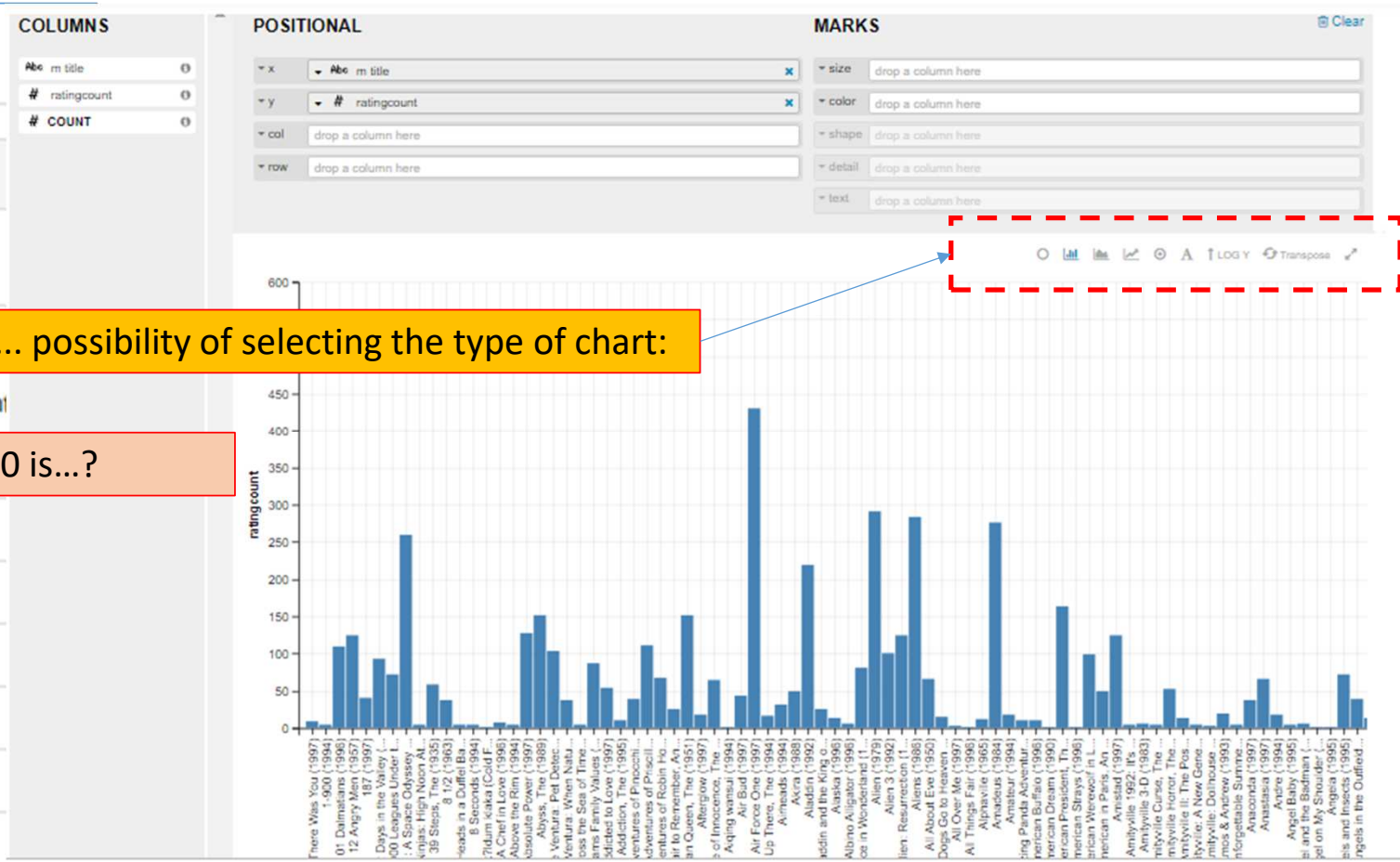
Query Process Results (Status: SUCCEEDED)

Logs | Results

Filter columns...

m.title	ratingcount
Star Wars (1977)	
Contact (1997)	509
Fargo (1996)	508
Return of the Jedi (1983)	507
Liar Liar (1997)	485
English Patient, The (1996)	481
Scream (1996)	478

Movie with ID=50 is…?



... possibility of selecting the type of chart:

We write quiries:

Which of the movies had the least number of ratings?

We write quiries:

Zadanie:

what movie titles have the highest average rating of all ratings?

... additionally check whether these data are reliable based on numerous ratings of a given film

We write quiries:

…. what movie titles have the highest average rating of all ratings?

Results :

Aiqing wansui (1994)

Star Kid (1997)

Query Process Results (Status: SUCCEEDED)

Logs | Results

Filter columns...

m.name	ratingavg	ratingcount
Aiqing wansui (1994)	5.0	1
Star Kid (1997)	5.0	3
Someone Else's America (1995)	5.0	1
Saint of Fort Washington, The (1993)	5.0	2
Great Day in Harlem, A (1994)	5.0	1
Santa with Muscles (1996)	5.0	2
Prefontaine (1997)	5.0	3
They Made Me a Criminal (1939)	5.0	1
Marlene Dietrich: Shadow and Light (1996)	5.0	1
Entertaining Angels: The Dorothy Day Story (1996)	5.0	1
Pather Panchali (1955)	4.625	8

… are these data reliable based on the numerous ratings of a given film

The answer is… : **NO**!!!

…too few ratings

We write quiries:

....what movie titles have the highest average rating of all ratings for which the number of ratings is >10?

Homework:

…What is the oldest movie with the highest average rating (i.e. above 4 stars) and provide the name(s) of the movie(s).

ATTENTION!!!

--In the Hive environment, convert the date so that analysis is possible.

In this query, you should sort by date: → (… oldest movie) → Let's analyze the description: → **README.txt**

… and analyze the data themselves:

SELECT * FROM ratings LIMIT 100;

ratings.user_id	ratings.movie_id	ratings.rating	ratings.rating_time
196	242	3	881250949
186	302	3	891717742
22	377	1	878887116
244	51	2	880606923

Conclusion: the date must be converted to a form that can be sorted

Description of the date operation function:

- https://www.educba.com/date-functions-in-hive/
- https://cwiki.apache.org/confluence/display/hive/languagemanual+udf#LanguageManualUDF-DateFunctions
- https://sqlandhadoop.com/hive-date-functions-all-possible-date-operations/

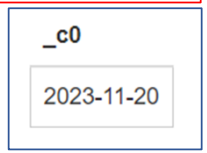
Description of the date operation function: https://cwiki.apache.org/confluence/display/hive/languagemanual+udf#LanguageManualUDF-DateFunctions

Return Type	Name(Signature)	Description
timestamp	from_utc_timestamp({*any primitive type*} ts, string timezone)	Converts a timestamp* in UTC to a given timezone (as of Hive 0.8.0). * timestamp is a primitive type, including timestamp/date, tinyint/smallint/int/bigint, float/double and decimal. Fractional values are considered as seconds. Integer values are considered as milliseconds. For example, from_utc_timestamp(2592000.0,'PST'), from_utc_timestamp(2592000000,'PST') and from_utc_timestamp(timestamp '1970-01-30 16:00:00','PST') all return the timestamp 1970-01-30 08:00:00.
timestamp	to_utc_timestamp({*any primitive type*} ts, string timezone)	Converts a timestamp* in a given timezone to UTC (as of Hive 0.8.0). * timestamp is a primitive type, including timestamp/date, tinyint/smallint/int/bigint, float/double and decimal. Fractional values are considered as seconds. Integer values are considered as milliseconds. For example, to_utc_timestamp(2592000.0,'PST'), to_utc_timestamp(2592000000,'PST') and to_utc_timestamp(timestamp '1970-01-30 16:00:00','PST') all return the timestamp 1970-01-31 00:00:00.
timestamp	current_timestamp	Returns the current timestamp at the start of query evaluation (as of Hive 1.2.0). All calls of current_timestamp within the same query return the same value.
string	from_unixtime(bigint unixtime[, string pattern])	Converts a number of seconds since epoch (1970-01-01 00:00:00 UTC) to a string representing the timestamp of that moment in the current time zone(using config "hive.local.time.zone") using the specified pattern. If the pattern is missing the default is used ('uuuu-MM-dd HH:mm:ss' or yyyy-MM-dd HH:mm:ss'). Example: from_unixtime(0)=1970-01-01 00:00:00 (hive.local.time.zone=Etc/GMT) As of Hive 4.0.0 (HIVE-25576), the "hive.datetime.formatter" property can be used to control the underlying formatter implementation, and as a consequence the accepted patterns and their behavior. Prior versions always used https://docs.oracle.com/javase/8/docs/api/java/text/SimpleDateFormat.html as the underlying formatter.
string	add_months(string start_date, int num_months, output_date_format)	Returns the date that is num_months after start_date (as of Hive 1.1.0). start_date is a string, date or timestamp. num_months is an integer. If start_date is the last day of the month or if the resulting month has fewer days than the day component of start_date, then the result is the last day of the resulting month. Otherwise, the result has the same day component as start_date. The default output format is 'yyyy-MM-dd'. Before Hive 4.0.0, the time part of the date is ignored. As of Hive 4.0.0, add_months supports an optional argument output_date_format, which accepts a String that represents a valid date format for the output. This allows to retain the time format in the output. For example : add_months('2009-08-31', 1) returns '2009-09-30'. add_months('2017-12-31 14:15:16', 2, 'YYYY-MM-dd HH:mm:ss') returns '2018-02-28 14:15:16'.
string	last_day(string date)	Returns the last day of the month which the date belongs to (as of Hive 1.1.0). date is a string in the format 'yyyy-MM-dd HH:mm:ss' or 'yyyy-MM-dd'. The time part of date is ignored.

Description of the date operation function: https://cwiki.apache.org/confluence/display/hive/languagemanual+udf#LanguageManualUDF-DateFunctions

current_date(): This function returns the current system date alone. It does not include the time part.

```
select current_date();
```



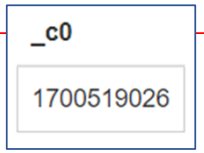
current_timestamp(): This function returns the current system time and date in a complete format.

```
select current_timestamp() DateTime;
```



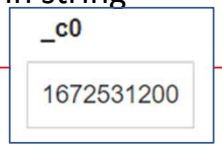
unix_timestamp(): This function uses the default time zone of the Unix epoch and returns a converted time format of the number of seconds between the Unix epoch and the specified date in string format.

```
select unix_timestamp();
```



unix_timestamp(str date): This function is used to convert ‘yyyy-MM-dd HH:mm: ss’ date format into normal Unix timestamp. It returns a converted time format of the number of seconds between the Unix epoch and the specified date in string format. And, it returns 0 on failure.

```
select unix_timestamp('2023-01-01 00:00:00');
```



unix_timestamp(str date, str pattern): This function is used to convert normal data type into another data type which can be specified in string pattern. It returns a converted time format of the number of seconds between the Unix epoch and the specified date in string format. And, it returns 0 on failure.

```
select unix_timestamp('2023-01-01 00:00:00', 'yyyy-MM-dd');
```



Description of the date operation function: https://cwiki.apache.org/confluence/display/hive/languagemanual+udf#LanguageManualUDF-DateFunctions

from_unixtime(bigint number of seconds [, str format]):

This function, from Unix epoch, converts the given number of seconds and returns the date in 'yyyy-MM-dd HH:mm: ss' format.

```
select from_unixtime(unix_timestamp());
```

_c0
2023-11-20 22:25:39

from_utc_timestamp(str date, time zone):

This function converts the specified date format in string format to the specified time zone in the second part of the expression. This function is used for the time zone conversion of the UTC time zone to another time zone.

```
select from_utc_timestamp('1990-01-01 09:00:00', 'JST');
```

_c0
1990-01-01 18:00:00.0

to_utc_timestamp(str date, time zone):

This function converts the specified data format in a string format in the time zone specified in the second part to the UTC time zone. This function is used for the time zone conversion of another time zone to the UTC time zone.

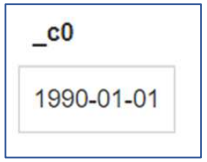
```
select to_utc_timestamp ('1990-02-13 00:00:00','America/Denver');
```

_c0
1990-02-13 07:00:00.0

Description of the date operation function: https://cwiki.apache.org/confluence/display/hive/languagemanual+udf#LanguageManualUDF-DateFunctions

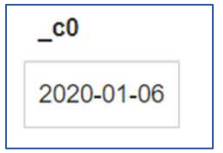
to_date(str timestamp): This function is used to return only the date part of the specified timestamp in standard date format 'yyyy-MM-dd'

```
select to_date ('1990-01-01 01:57:30');
```



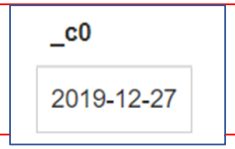
date_add(str date, int number of days): This function is used to add the specified number of days to the given date and return the final added date.

```
select date_add ('2020-01-01', 5);
```



date_sub(str date, int number of days): This function is used to subtract the specified number of days from the given date and return the final subtracted date.

```
select date_sub ('2020-01-01', 5);
```



date_diff(str date 1, str date 2): This function is used to find the difference between two specified dates and returns the difference in the number of days.

```
select date_diff('2020-01-01','2019-01-01');
```

We write quiries:

… What is the oldest movie with the highest average rating (i.e. above 4 stars) and provide the name(s) of the movie(s).

This function uses the default time zone of the Unix epoch and returns a converted time format of the number of seconds between the Unix epoch and the specified date in string format.

This function, from Unix epoch, converts the given number of seconds and returns the date in 'yyyy-MM-dd HH:mm: ss' format.

This function is used to return only the date part of the specified timestamp in standard date format 'yyyy-MM-dd'.

Question: why do we aggregate based on the film's release date?

Answer: **because there are many ratings for the same movie**

Thin Man, The (1934)
It Happened One Night (1934)

Query Process Results (Status: SUCCEEDED)

Logs | Results

Filter columns...

ratingavg	mov_id	name	mdate
4.15	493	Thin Man, The (1934)	1934-01-01
4.012345679012346	604	It Happened One Night (1934)	1934-01-01
4.0508474576271185	615	39 Steps, The (1935)	1935-01-01
4.0476190476190474	1203	Top Hat (1935)	1935-01-01
37037037037037	613	My Man Godfrey (1936)	1936-01-01
57971014492754	633	Christmas Carol, A (1938)	1938-01-01
4.0772357723577235	132	Wizard of Oz, The (1939)	1939-01-01

… are these data reliable based on the numerous ratings of a given film

The answer is…

… let's say yes

Homework:

.. Then verify whether it is the oldest video using the Hive framework.

Homework:

What genres of films were watched most often by women and which by men?

We write quiries:

First --> create table: USERS:

Results:

```
CREATE TABLE users (user_id INT, age INT, gender STRING, occupation STRING,
zip_code INT) ROW FORMAT DELIMITED STORED AS TEXTFILE;
```

and import user.csv

We write quiries:

What genres of films were watched most often by women and which by men?

Results:

Step: 3

we add Lateral View columns

```
CREATE VIEW IF NOT EXISTS TopCategory
as
SELECT us.gender,
                    SUM(m.unknown) Unknown,'Unknown' Unk1, SUM(m.action) Action, 'Action' Act1, SUM(m.adventure) Adventure, 'Adventure' Adv1,
SUM(m.animation) Animation, 'Animation' Ani1,
                    SUM(m.childrens) Childrens, 'Childrens' Chi1,
                    SUM(m.comedy) Comedy, 'Comedy' Com1, SUM(m.crime) Crime, 'Crime' Cri1, SUM(m.documentary) Documentary,'Documentary'
Doc1, SUM(m.drama) Drama,'Drama' Dra1, SUM(m.fantasy) Fantasy, 'Fantasy' Fan1,
                    SUM(m.filmnoir) Film_noir,'Film_noir' Fil1,
                    SUM(m.horror) Horror, 'Horror' Hor1, SUM(m.musical) Musical, 'Musical' Mus1, SUM(m.mystery) Mystery, 'Mystery' Mys1,
SUM(m.romance) Romance, 'Romance' Rom1, SUM(m.scifi) Sci_fi, 'Sci_fi' Sci1,
                    SUM(m.thriller) Thriller, 'Thriller' Thr1,
                    SUM(m.war) War, 'War' War1, SUM(m.western) Western, 'Western' Wes1
          FROM users us
          INNER JOIN ratings r
          ON us.user_id = r.user_id
          INNER JOIN movies m
          ON r.movie_id = m.movie_id
          GROUP BY us.gender;
SELECT tc.gender, tf.key, tf.value  FROM TopCategory tc
lateral VIEW explode(map(Unk1,Unknown, Act1,Action, Adv1,Adventure, Ani1,Animation, Chi1,Childrens, Com1,Comedy, Cri1,Crime, Doc1,Documentary,
Dra1,Drama, Fan1,Fantasy,
Fil1,Film_noir, Hor1,Horror, Mus1,Musical, Mys1,Mystery, Rom1,Romance, Sci1,Sci_fi, Thr1,Thriller, War1,War, Wes1,Western))
tf AS key,value;
```

```
CREATE VIEW IF NOT EXISTS TopCategory
as
SELECT us.gender,
                    SUM(m.unknown) Unknown,'Unknown' Unk1, SUM(m.action) Action, 'Action' Act1, SUM(m.adventure) Adventure, 'Adventure' Adv1, SUM(m.animation)
Animation, 'Animation' Ani1,
                    SUM(m.childrens) Childrens, 'Childrens' Chi1,
                    SUM(m.comedy) Comedy, 'Comedy' Com1, SUM(m.crime) Crime, 'Crime' Cri1, SUM(m.documentary) Documentary,'Documentary' Doc1, SUM(m.drama)
Drama,'Drama' Dra1, SUM(m.fantasy) Fantasy, 'Fantasy' Fan1,
                    SUM(m.filmnoir) Film_noir,'Film_noir' Fil1,
                    SUM(m.horror) Horror, 'Horror' Hor1, SUM(m.musical) Musical, 'Musical' Mus1, SUM(m.mystery) Mystery, 'Mystery' Mys1, SUM(m.romance) Romance,
'Romance' Rom1, SUM(m.scifi) Sci_fi, 'Sci_fi' Sci1,
                    SUM(m.thriller) Thriller, 'Thriller' Thr1,
                    SUM(m.war) War, 'War' War1, SUM(m.western) Western, 'Western' Wes1
          FROM users us
          INNER JOIN ratings r
          ON us.user_id = r.user_id
          INNER JOIN movies m
          ON r.movie_id = m.movie_id
          GROUP BY us.gender;
SELECT u1.*
FROM (
SELECT tc.gender, tf.key, tf.value  FROM TopCategory tc
lateral VIEW explode(map(Unk1,Unknown, Act1,Action, Adv1,Adventure, Ani1,Animation, Chi1,Childrens, Com1,Comedy, Cri1,Crime, Doc1,Documentary, Dra1,Drama, Fan1,Fantasy,
Fil1,Film_noir, Hor1,Horror, Mus1,Musical, Mys1,Mystery, Rom1,Romance, Sci1,Sci_fi, Thr1,Thriller, War1,War, Wes1,Western))
tf AS key,value
where tc.gender = 'F'
order by tf.value desc limit 1) u1
union all
SELECT u2.*
FROM (SELECT tc.gender, tf.key, tf.value  FROM TopCategory tc
lateral VIEW explode(map(Unk1,Unknown, Act1,Action, Adv1,Adventure, Ani1,Animation, Chi1,Childrens, Com1,Comedy, Cri1,Crime, Doc1,Documentary, Dra1,Drama, Fan1,Fantasy,
Fil1,Film_noir, Hor1,Horror, Mus1,Musical, Mys1,Mystery, Rom1,Romance, Sci1,Sci_fi, Thr1,Thriller, War1,War, Wes1,Western))
tf AS key,value
where tc.gender = 'M'
order by tf.value desc limit 1) u2;
```

_u1.gender	_u1.key	_u1.value
F	Drama	11008
M	Drama	28887

Step: 4

we use UNION ALL

We write quiries:

What genres of films were watched most often by women and which by men?

Results:

Step: 2

we add Lateral View columns

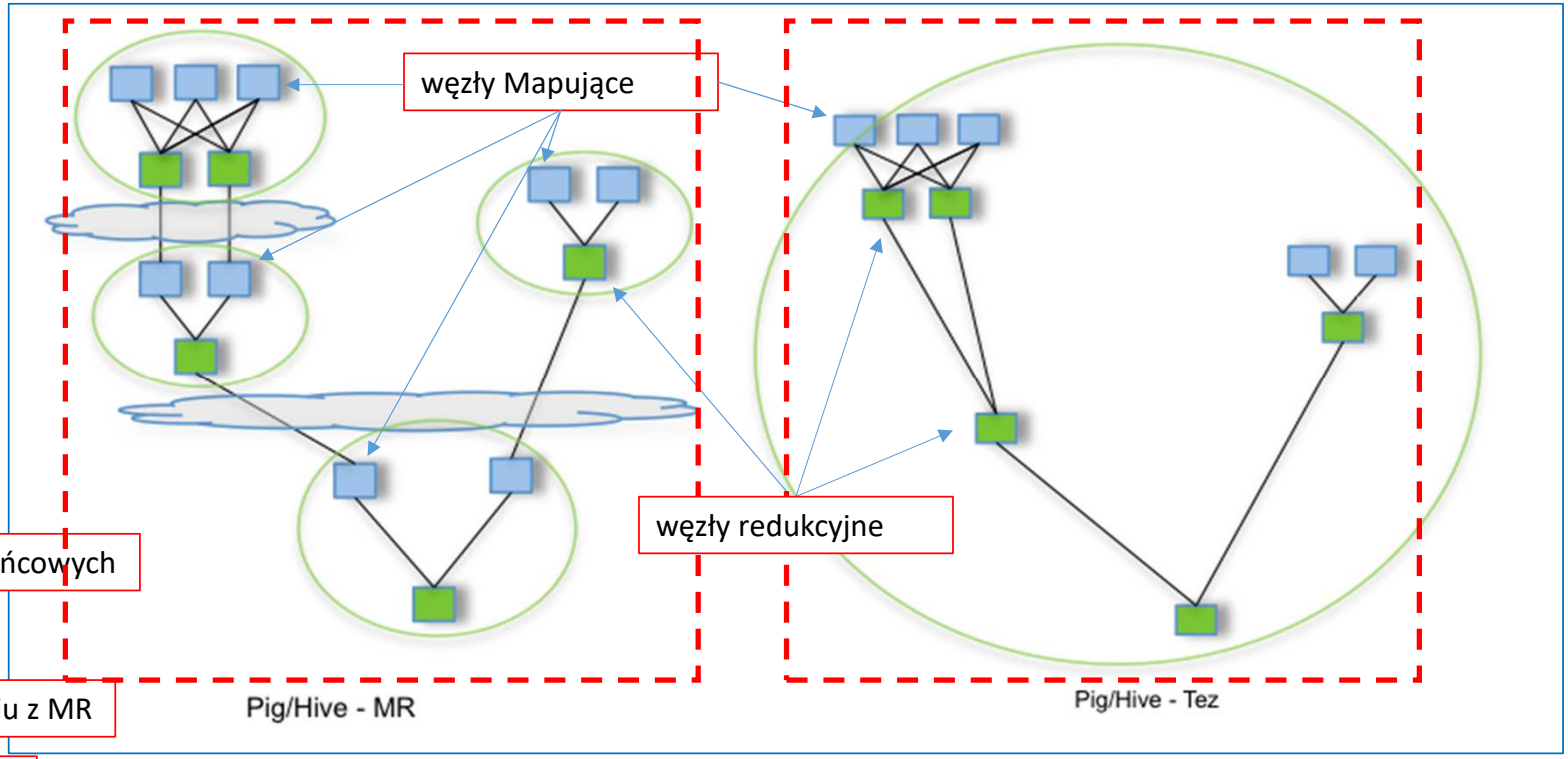
```
CREATE VIEW IF NOT EXISTS TopCategory
as
SELECT us.gender,
                    SUM(m.unknown) Unknown,'Unknown' Unk1, SUM(m.action) Action, 'Action' Act1, SUM(m.adventure) Adventure, 'Adventure' Adv1,
SUM(m.animation) Animation, 'Animation' Ani1,
                    SUM(m.childrens) Childrens, 'Childrens' Chi1,
                    SUM(m.comedy) Comedy, 'Comedy' Com1, SUM(m.crime) Crime, 'Crime' Cri1, SUM(m.documentary) Documentary,'Documentary'
Doc1, SUM(m.drama) Drama,'Drama' Dra1, SUM(m.fantasy) Fantasy, 'Fantasy' Fan1,
                    SUM(m.filmnoir) Film_noir,'Film_noir' Fil1,
                    SUM(m.horror) Horror, 'Horror' Hor1, SUM(m.musical) Musical, 'Musical' Mus1, SUM(m.mystery) Mystery, 'Mystery' Mys1,
SUM(m.romance) Romance, 'Romance' Rom1, SUM(m.scifi) Sci_fi, 'Sci_fi' Sci1,
                    SUM(m.thriller) Thriller, 'Thriller' Thr1,
                    SUM(m.war) War, 'War' War1, SUM(m.western) Western, 'Western' Wes1
          FROM users us
          INNER JOIN ratings r
          ON us.user_id = r.user_id
          INNER JOIN movies m
          ON r.movie_id = m.movie_id
          GROUP BY us.gender;
SELECT tc.gender, tf.key, tf.value  FROM TopCategory tc
lateral VIEW explode(map(Unk1,Unknown, Act1,Action, Adv1,Adventure, Ani1,Animation, Chi1,Childrens, Com1,Comedy, Cri1,Crime, Doc1,Documentary,
Dra1,Drama, Fan1,Fantasy,
Fil1,Film_noir, Hor1,Horror, Mus1,Musical, Mys1,Mystery, Rom1,Romance, Sci1,Sci_fi, Thr1,Thriller, War1,War, Wes1,Western))
tf AS key,value;
```

Silnik TEZ

https://tez.apache.org/

Apache TEZ pozwala na zbudowanie szkieletu, który pozwoli na złożenie ukierunkowanego acyklicznego wykresu zadań (DAG - directed-acyclic-graph) przetwarzania danych.

Pozwalając projektom takim jak Apache Hive i Apache Pig na uruchamianie złożonych zadań DAG



2 główne motywy projektowe dla Tez to:

- zwiększanie możliwości użytkowników końcowych
- wydajność wykonania:
  - wzrost wydajności w porównaniu z MR
  - optymalne zarządzanie zasobami
  - dynamiczne decyzje dotyczące fizycznego przepływu danych

Silnik TEZ

https://tez.apache.org/

Dashboard | Services | Hosts | Alerts | Admin | maria_dev

- YARN Queue Manager
- Files View
- Hive View
- Hive View 2.0
- Pig View
- Storm View
- Tez View
- Workflow Manager

Upload Table

Query Editor

Worksheet ✖ | Worksheet ✖

```
1 FROM (
2 SELECT m.NAME, AVG(r. rating) as RatingAVG,  COUNT(r.movie_id) as RatingCount
3 from ratings r inner join movies m on r.movie_id=m.movie_id
4 GROUP BY m.NAME
5 ) a
6 select a.name, a.RatingAVG, a.RatingCount
7 WHERE a.RatingCount>10
8 order by a.RatingAVG
9 DESC;
```

TEZ Home / All Queries — Version 0.7.0.2.6.0.3-8

Hive Queries | All DAGs

Last refreshed at 24 Nov 2020 22:36:07 — Refresh

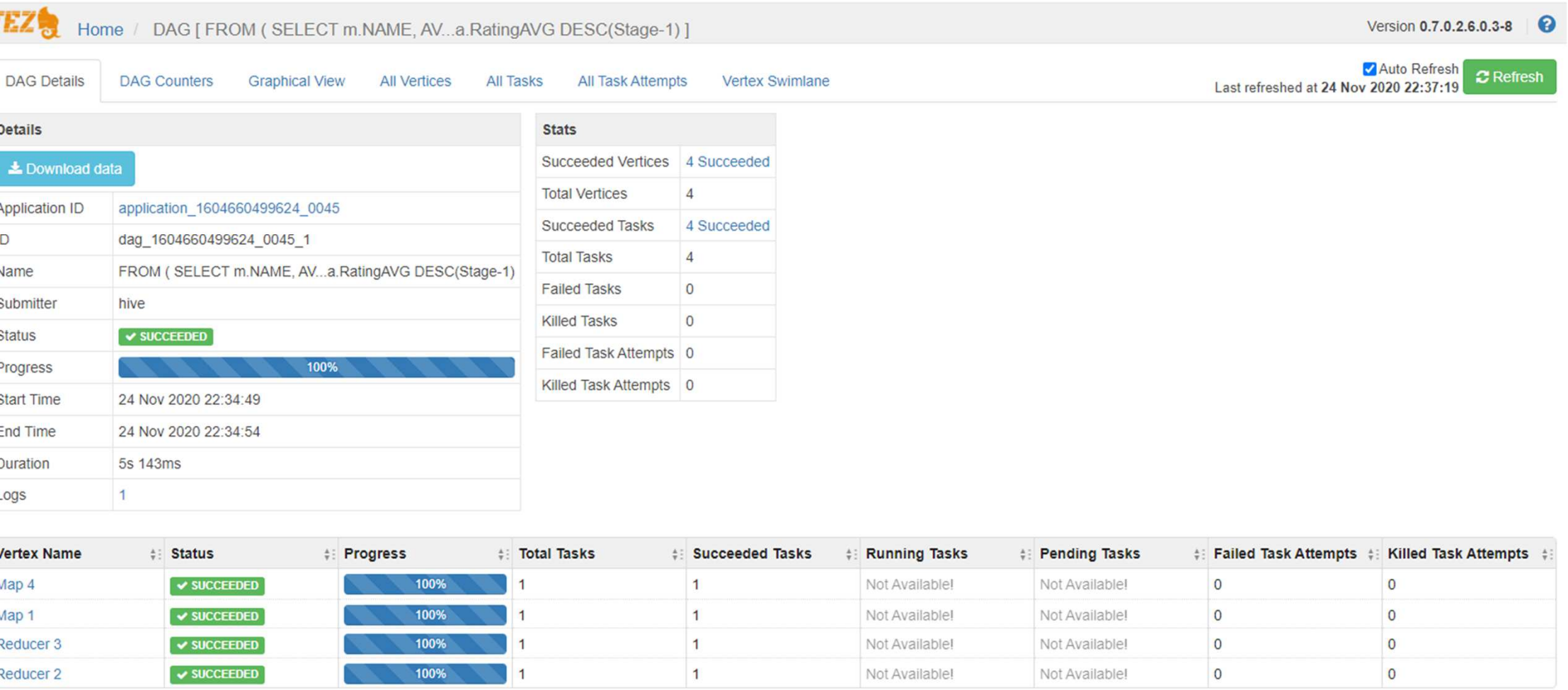
Query ID:	User:	DAG ID:	Tables Read:	Tables Written:	App ID:	Queue:	Execution Mode:
Search...	Search...	Search...	Search...	Search...	Search...	Search...	Search...

1 2 — 10 Rows

Query ID	User	Status	Query	DAG ID	Tables Read	Tables Written	LLAP App ID	Start Time
hive_20201124213445_17ce6fe5-5...	maria_dev	SUCCEEDED	FROM (SELECT m...	dag_1604660499624_0045_1	default.ratings,defa...	None	Not Available!	24 Nov 2020 22:3
hive_20201124210917_42608b36-c...	maria_dev	SUCCEEDED	FROM (SELECT m...	dag_1604660499624_0044_1	default.ratings,defa...	None	Not Available!	24 Nov 2020 22:0
hive_20201120151028_ecf508d1-d...	maria_dev	SUCCEEDED	select m.name, AV...	dag_1604660499624_0043_1	default.noofrs,defau...	None	Not Available!	20 Nov 2020 16:1
hive_20201120150930_2c151420-b...	maria_dev	SUCCEEDED	SELECT m.name, A...	dag_1604660499624_0042_1	default.ratings,defa...	None	Not Available!	20 Nov 2020 16:0
hive_20201120145607_fd68069f-aa...	maria_dev	SUCCEEDED	select m.name, AV...	dag_1604660499624_0041_1	default.noofrs,defau...	None	Not Available!	20 Nov 2020 15:5
hive_20201120145308_afbf023c-64...	maria_dev	SUCCEEDED	select m.name, AV...	dag_1604660499624_0040_1	default.noofrs,defau...	None	Not Available!	20 Nov 2020 15:5
hive_20201120145158_fb0f8a8f-ba...	maria_dev	SUCCEEDED	select m.name, AV...	dag_1604660499624_0039_1	default.noofrs,defau...	None	Not Available!	20 Nov 2020 15:5
hive_20201120145104_bf9a200e-4...	maria_dev	SUCCEEDED	select m.name, AV...	dag_1604660499624_0038_1	default.noofrs,defau...	None	Not Available!	20 Nov 2020 15:5
hive_20201120144453_afafae36-c0...	maria_dev	SUCCEEDED	select m.name, no...	dag_1604660499624_0037_1	default.noofrs,defau...	None	Not Available!	20 Nov 2020 15:4
hive_20201120144323_a035393a-4...	maria_dev	SUCCEEDED	select m.name, no...	dag_1604660499624_0036_2	default.noofrs,defau...	None	Not Available!	20 Nov 2020 15:4

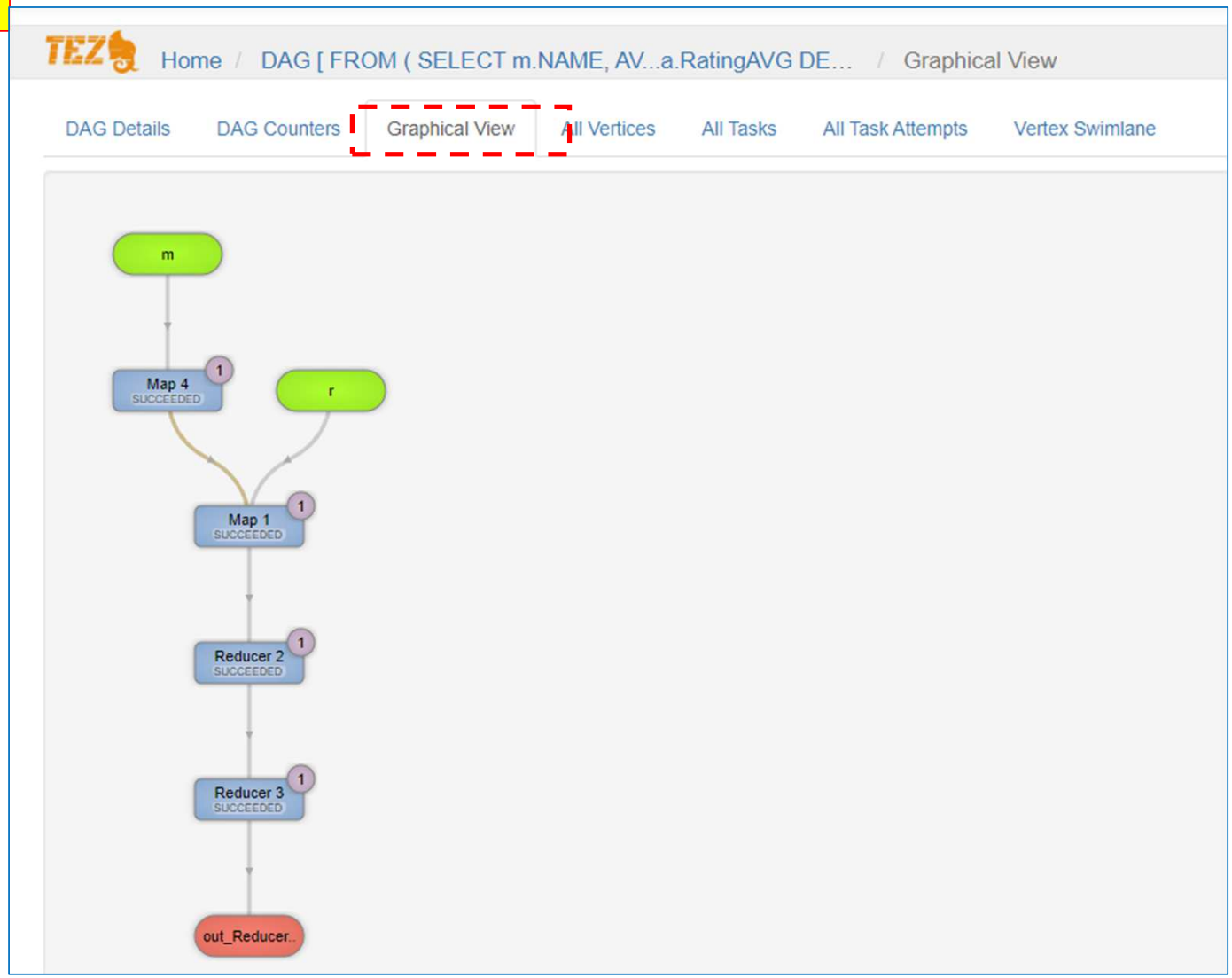
Silnik TEZ

https://tez.apache.org/



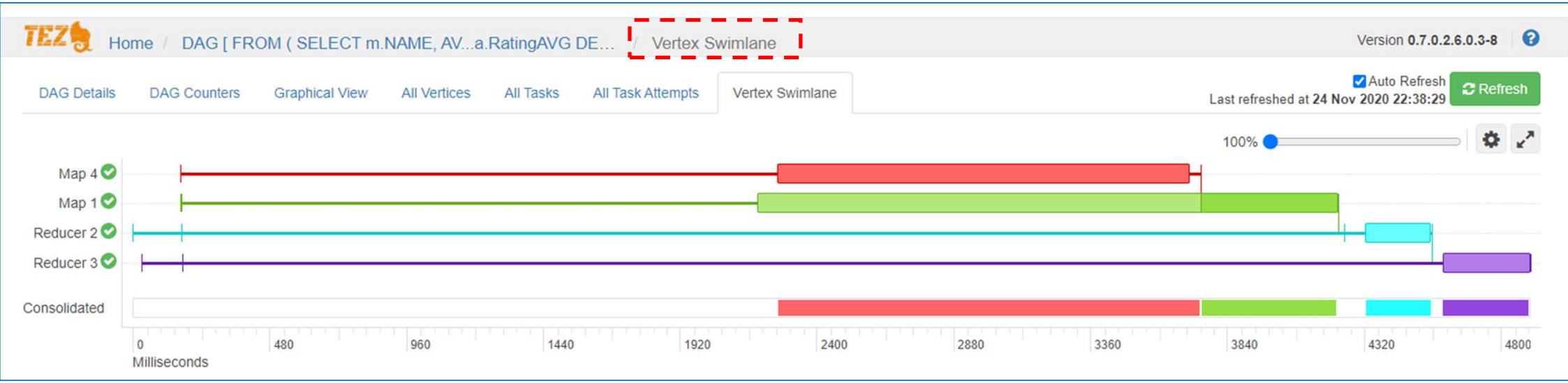
Silnik TEZ

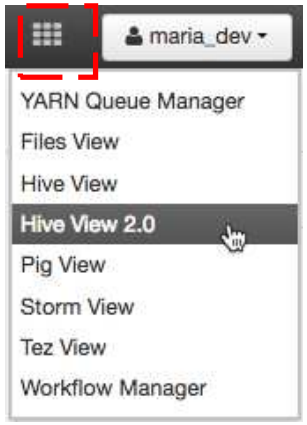
https://tez.apache.org/

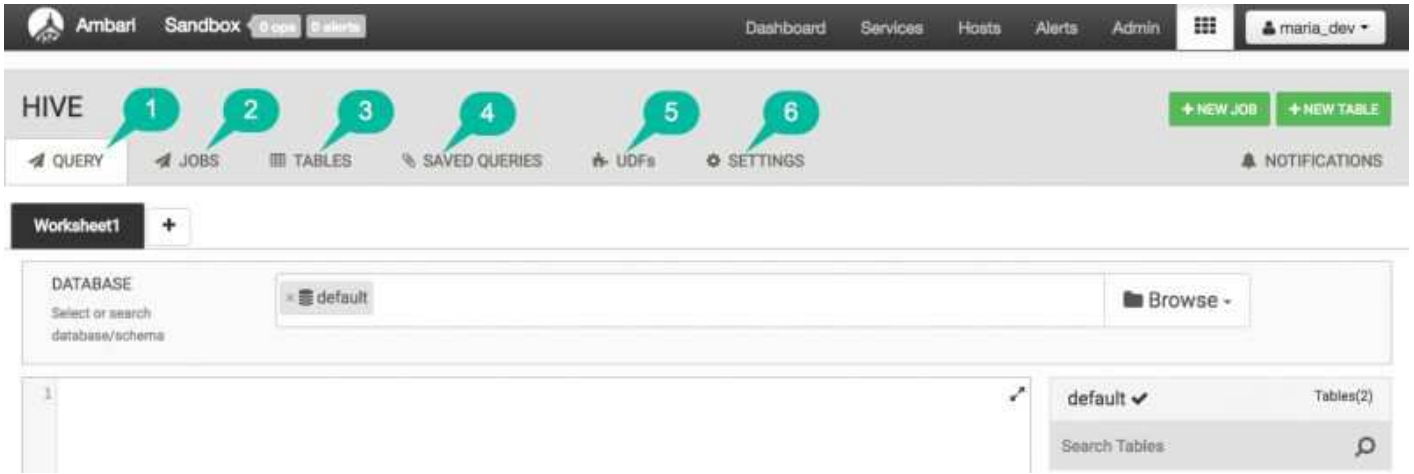


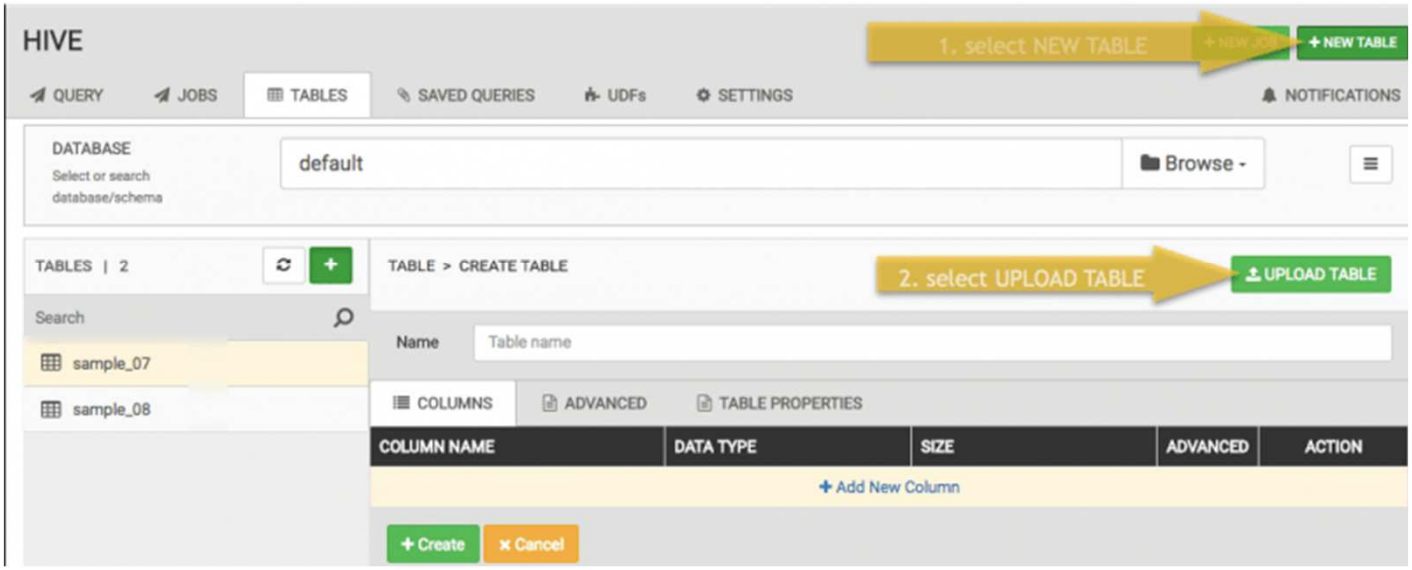
Silnik TEZ

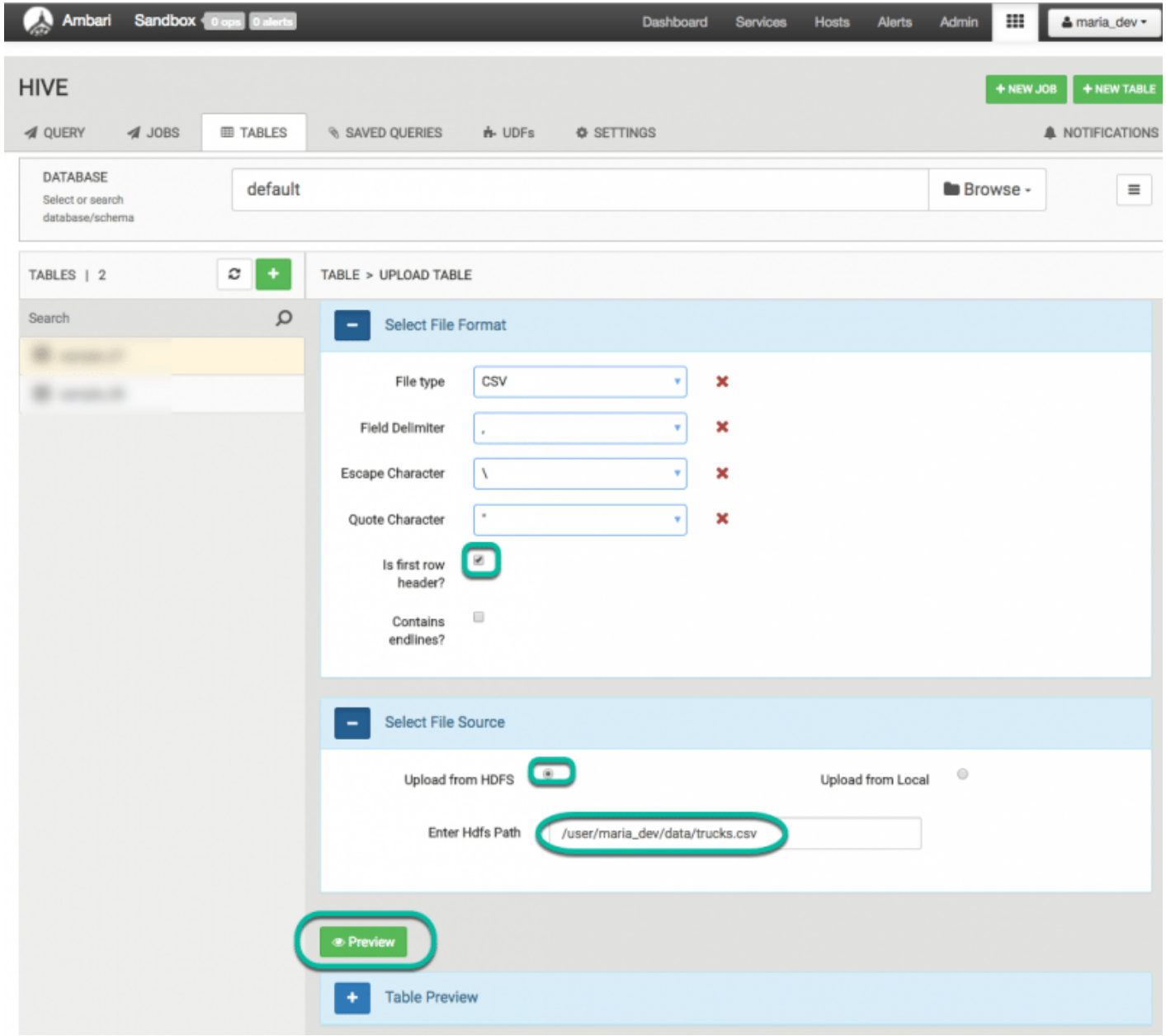
https://tez.apache.org/

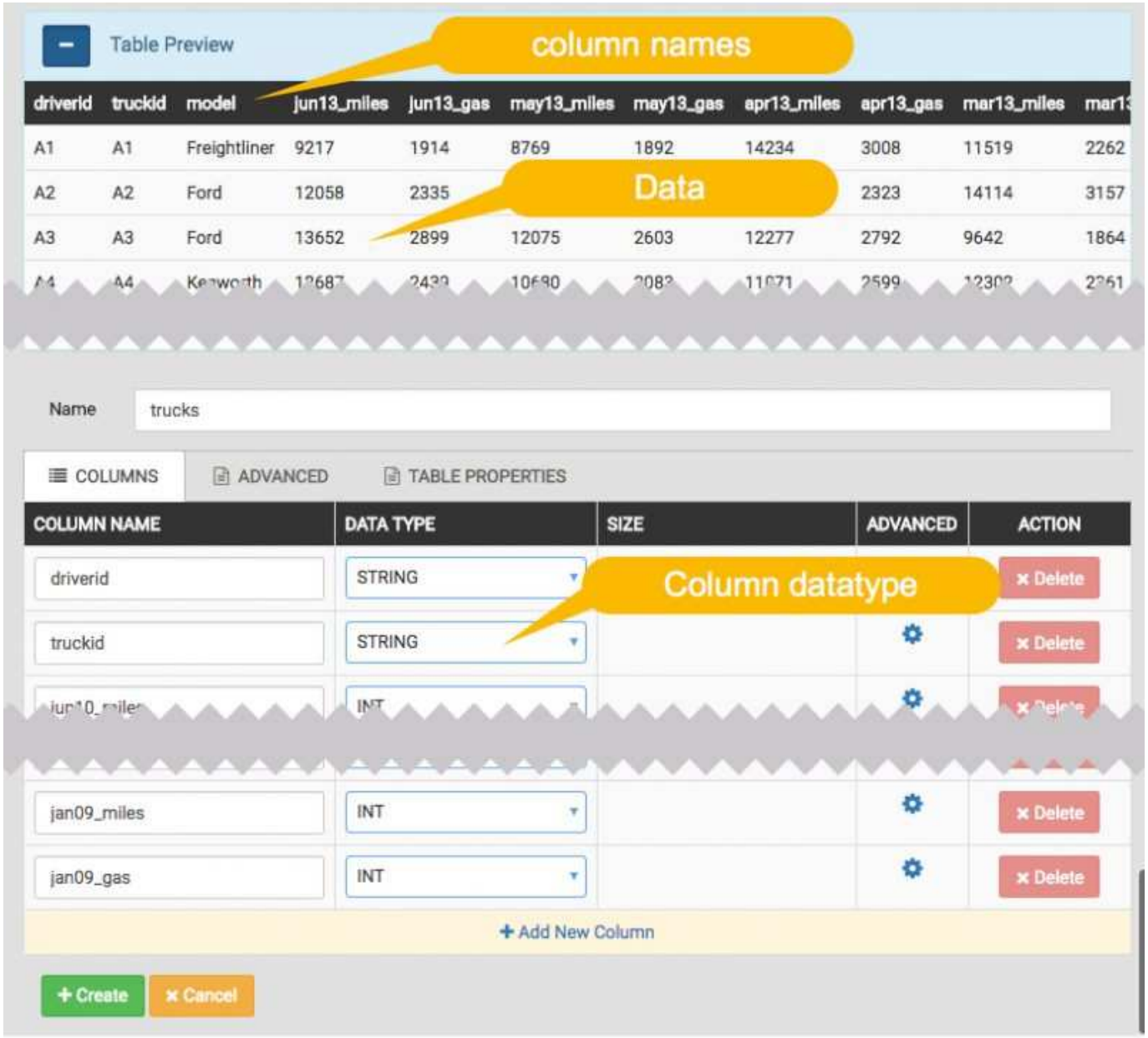


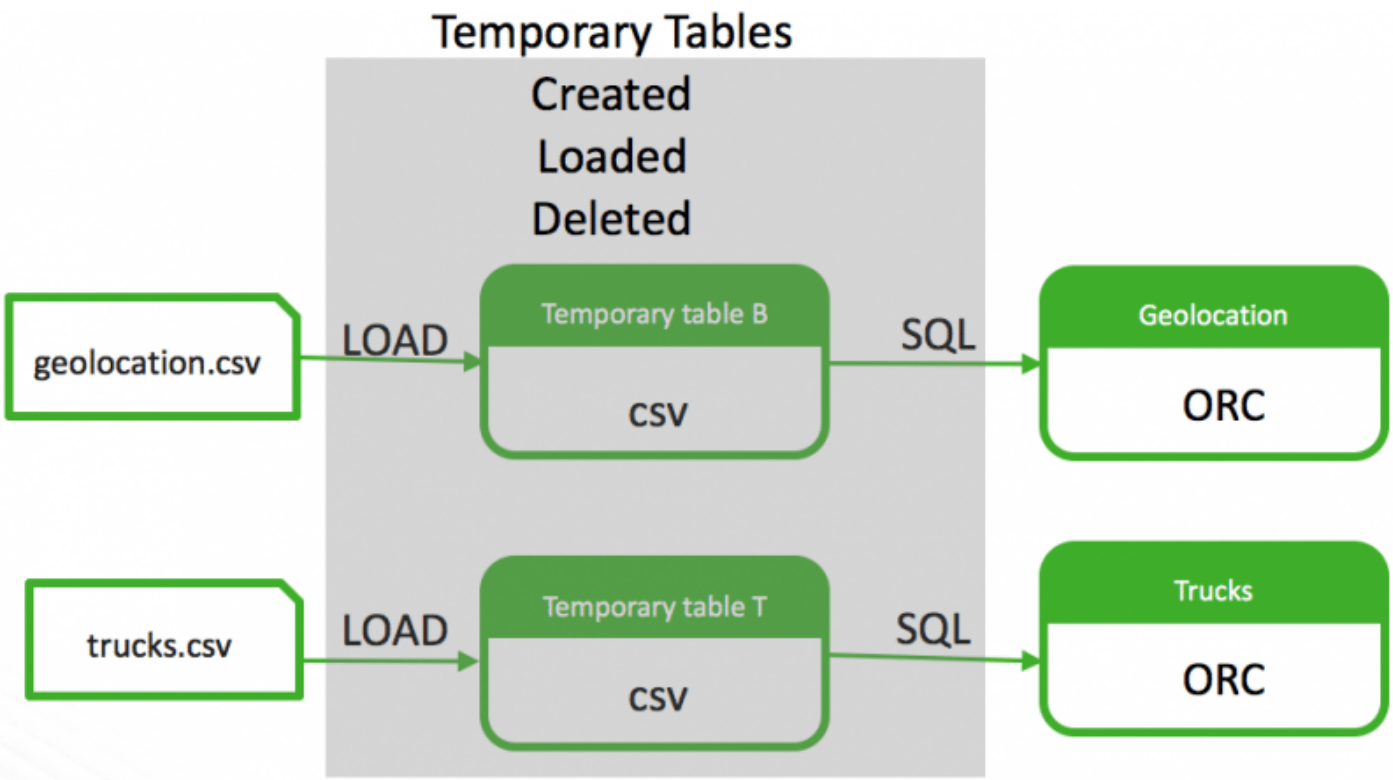

maria_dev
YARN Queue Manager
Files View
Hive View
Hive View 2.0
Pig View
Storm View
Tez View
Workflow Manager


Ambari
Sandbox
Dashboard
Services
Hosts
Alerts
Admin
maria_dev
HIVE
1
2
3
4
5
6
QUERY
JOBS
TABLES
SAVED QUERIES
UDFs
SETTINGS
+ NEW JOB
+ NEW TABLE
NOTIFICATIONS
Worksheet1
DATABASE
Select or search database/schema
default
Browse
default
Tables(2)
Search Tables


HIVE
1. select NEW TABLE
+ NEW TABLE
QUERY
JOBS
TABLES
SAVED QUERIES
UDFs
SETTINGS
NOTIFICATIONS
DATABASE
Select or search database/schema
default
Browse
TABLES | 2
TABLE > CREATE TABLE
2. select UPLOAD TABLE
UPLOAD TABLE
Search
sample_07
sample_08
Name
Table name
COLUMNS
ADVANCED
TABLE PROPERTIES
COLUMN NAME
DATA TYPE
SIZE
ADVANCED
ACTION
+ Add New Column
+ Create
× Cancel


Ambari
Sandbox
0 ops
0 alerts
Dashboard
Services
Hosts
Alerts
Admin
maria_dev
HIVE
+ NEW JOB
+ NEW TABLE
QUERY
JOBS
TABLES
SAVED QUERIES
UDFs
SETTINGS
NOTIFICATIONS
DATABASE
Select or search database/schema
default
Browse
TABLES | 2
TABLE > UPLOAD TABLE
Search
Select File Format
File type
CSV
Field Delimiter
,
Escape Character
\
Quote Character
"
Is first row header?
Contains endlines?
Select File Source
Upload from HDFS
Upload from Local
Enter Hdfs Path
/user/maria_dev/data/trucks.csv
Preview
Table Preview


Table Preview
column names
driverid truckid model jun13_miles jun13_gas may13_miles may13_gas apr13_miles apr13_gas mar13_miles mar13
A1 A1 Freightliner 9217 1914 8769 1892 14234 3008 11519 2262
A2 A2 Ford 12058 2335 2323 14114 3157
Data
A3 A3 Ford 13652 2899 12075 2603 12277 2792 9642 1864
Name
trucks
COLUMNS
ADVANCED
TABLE PROPERTIES
COLUMN NAME
DATA TYPE
SIZE
ADVANCED
ACTION
driverid
STRING
Column datatype
Delete
truckid
STRING
Delete
INT
jan09_miles
INT
Delete
jan09_gas
INT
Delete
Add New Column
Create
Cancel

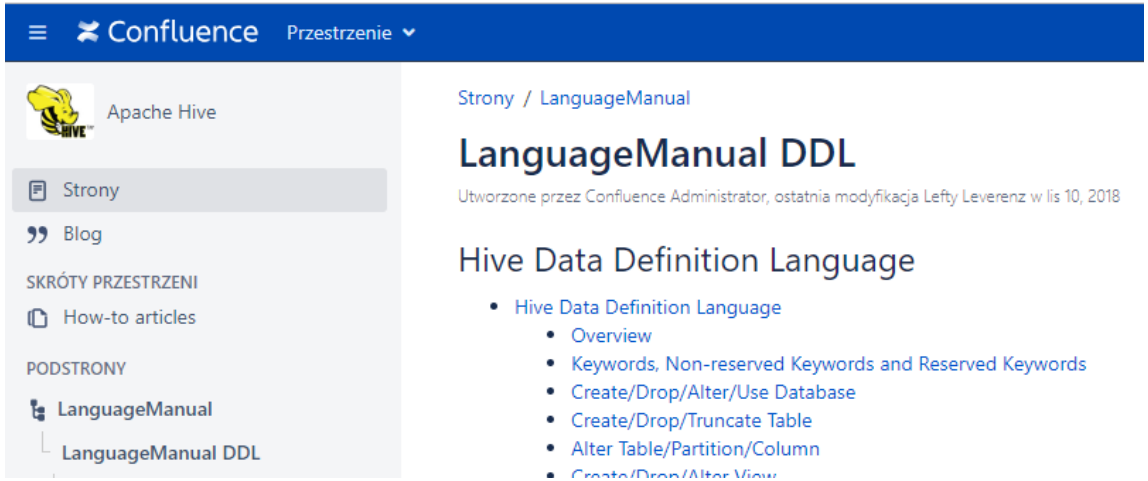

Temporary Tables
Created
Loaded
Deleted
geolocation.csv
LOAD
Temporary table B
csv
SQL
Geolocation
ORC
trucks.csv
LOAD
Temporary table T
csv
SQL
Trucks
ORC

3

Referencing Hive tables with SQL commands

(bez kreatora Ambari)

https://cwiki.apache.org/confluence/display/Hive/LanguageManual+DDL



Hive - formaty przechowywania:

https://cwiki.apache.org/confluence/display/Hive/LanguageManual+DDL

Hive supports built-in and custom file formats.

See CompressedStorage for details on compressed table storage.

Storage Format	Description
STORED BY	Stored by a non-native table format. To create or link to a non-native table, for example a table backed by HBase or Druid or Accumulo. See StorageHandlers for more information on this option.
STORED AS TEXTFILE	Stored as plain text files. TEXTFILE is the default file format, unless the configuration parameter **hive.default.fileformat** has a different setting. Use the DELIMITED clause to read delimited files. Enable escaping for the delimiter characters by using the 'ESCAPED BY' clause (such as ESCAPED BY '\') Escaping is needed if you want to work with data that can contain these delimiter characters. A custom NULL format can also be specified using the 'NULL DEFINED AS' clause (default is '\N').
STORED AS SEQUENCEFILE	Stored as compressed Sequence File.
STORED AS RCFILE	Stored as Record Columnar File format.
STORED AS PARQUET	Stored as Parquet format for the Parquet columnar storage format in Hive 0.13.0 and later; Use ROW FORMAT SERDE ... STORED AS INPUTFORMAT ... OUTPUTFORMAT syntax ... in Hive 0.10, 0.11, or 0.12.
STORED AS ORC	Stored as ORC file format. Supports ACID Transactions & Cost-based Optimizer (CBO). Stores column-level metadata.
STORED AS JSONFILE	Stored as Json file format in Hive 4.0.0 and later.
STORED AS AVRO	Stored as Avro format in Hive 0.14.0 and later (see Avro SerDe).

Hive – podstawowe typy danych:

https://cwiki.apache.org/confluence/display/Hive/LanguageManual+DDL

Typy proste:

Typy są powiązane z kolumnami w tabelach.

Liczby całkowite:

- TINYINT - 1 bajtowa liczba całkowita
- SMALLINT - 2-bajtowa liczba całkowita
- INT - 4 bajtowa liczba całkowita
- BIGINT - 8-bajtowa liczba całkowita

•Typy binarne
- BINARY - ciąg bajtów

Typ boolowski:

- BOOLEAN - TRUE / FALSE

Liczby zmiennoprzecinkowe

- FLOAT - pojedyncza precyzja
- DOUBLE - Podwójna precyzja

Liczby o ustalonej liczbie znaków

- DECIMAL - stała wartość punktowa skali i precyzji zdefiniowanej przez użytkownika

Typy ciągów

- STRING - ciąg znaków w określonym zestawie znaków
- VARCHAR - sekwencja znaków w określonym zestawie znaków o maksymalnej długości
- CHAR - sekwencja znaków w określonym zestawie znaków o określonej długości

Typy daty i godziny

- TIMESTAMP - data i godzina **bez strefy czasowej** (semantyka „LocalDateTime”)
- TIMESTAMP Z LOKALNĄ STREFĄ CZASOWĄ - czas mierzony do nanosekund (semantyka „natychmiastowa”)
- DATA - data

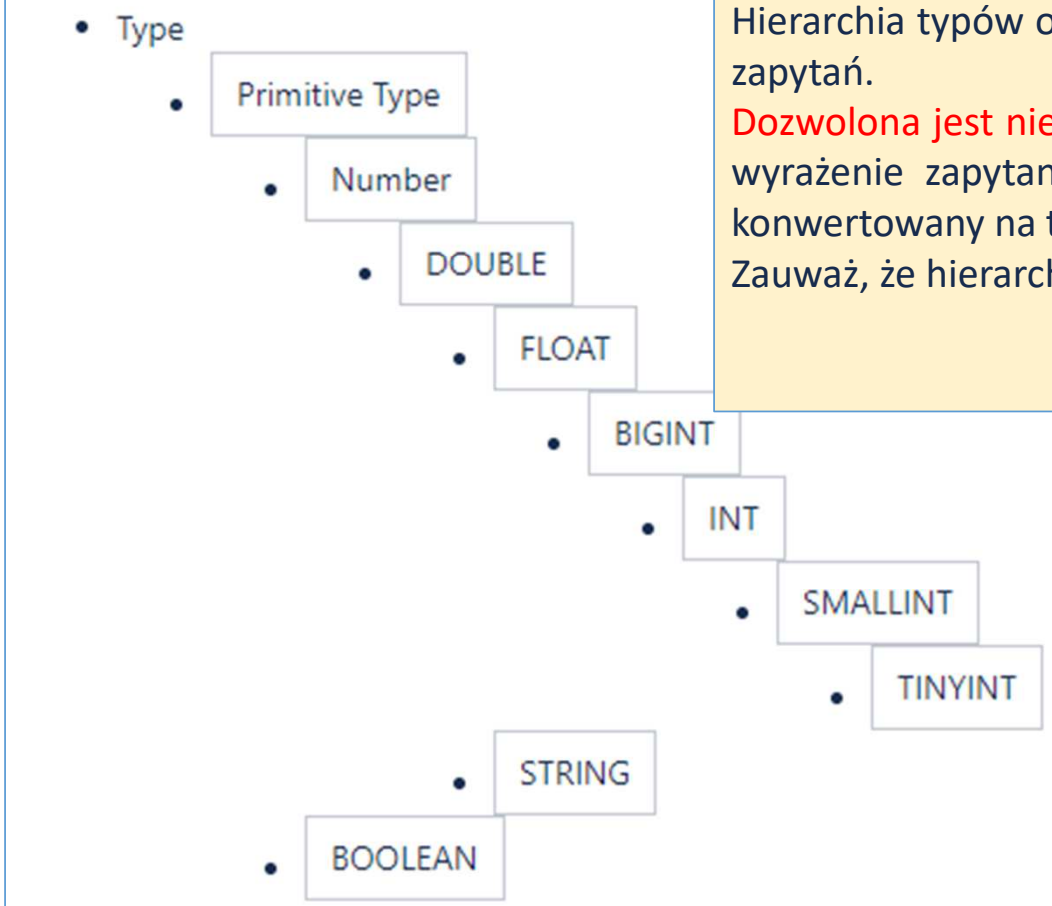
Typy binarne

BINARY - ciąg bajtów

Hive – podstawowe typy danych:

https://cwiki.apache.org/confluence/display/Hive/LanguageManual+DDL

The Types are organized in the following hierarchy (where the parent is a super type of all the children instances):



Hierarchia typów określa, w jaki sposób typy są domyślnie konwertowane w języku zapytań.
Dozwolona jest niejawna konwersja dla typów od potomka do przodka. Kiedy więc wyrażenie zapytania oczekuje typu 1, a dane są typu 2, typ 2 jest domyślnie konwertowany na typ 1, jeśli typ 1 jest przodkiem typu 2 w hierarchii typów.
Zauważ, że hierarchia typów umożliwia niejawną konwersję STRING na DOUBLE.

Hive – podstawowe typy danych:

https://cwiki.apache.org/confluence/display/Hive/LanguageManual+DDL

Typy złożone
Typy złożone można budować z typów pierwotnych i innych typów złożonych, używając:

- Struktury: dostęp do elementów tego typu można uzyskać za pomocą notacji DOT (.).

Np.: dla kolumny c typu STRUCT {a INT; b INT} dostęp do pola uzyskuje wyrażenie: c.a

- Mapy (krotki klucz-wartość): Dostęp do elementów można uzyskać za pomocą notacji ['nazwa elementu'].

Np.: w mapie M zawierającej mapowanie z 'grupy' -> gid wartość gid można uzyskać za pomocą M ['grupa']

- Tablice (listy indeksowane): Elementy w tablicy muszą być tego samego typu. Dostęp do elementów można uzyskać za pomocą notacji [n], gdzie n jest indeksem (liczonym od zera) tablicy.

Np.: dla tablicy A mającej elementy ['a', 'b', 'c'], A [1] zwróci: 'b'.

Używając typów pierwotnych i konstrukcji do tworzenia typów złożonych, można tworzyć typy o dowolnym poziomie zagnieżdżenia. Na przykład typ Użytkownik może zawierać następujące pola:
•płeć - która jest STRING.
•active - czyli BOOLEAN.

Hive – **Timestamp**:

https://cwiki.apache.org/confluence/display/Hive/LanguageManual+DDL

Znaczniki czasu były źródłem wielu nieporozumień.

Timestamp (semantyka „LocalDateTime”)

Znaczniki czasu „LocalDateTime” Java zapisują datę i godzinę jako rok, miesiąc, datę, godzinę, minutę i sekundy bez strefy czasowej. Te znaczniki czasu zawsze mają te same wartości, niezależnie od lokalnej strefy czasowej.

Na przykład wartość znacznika czasu „2014-12-12 12:34:56” jest rozkładana na pola roku, miesiąca, dnia, godziny, minuty i sekund, ale bez dostępnych informacji o strefie czasowej. Nie odpowiada żadnej konkretnej chwili. Zawsze będzie miała tę samą wartość, niezależnie od lokalnej strefy czasowej. O ile twoja aplikacja nie używa UTC konsekwentnie, znacznik czasu z lokalną strefą czasową jest zdecydowanie lepszy niż znacznik czasu dla większości aplikacji. Gdy użytkownicy mówią, że zdarzenie jest o godzinie 10:00, zawsze odnosi się do określonej strefy czasowej i oznacza punkt w czasie, a nie 10:00 w dowolnej strefie czasowej.

Znacznik czasu z lokalną strefą czasową (semantyka „Instant”)

Znaczniki czasowe Java „Instant” definiują moment w czasie, który pozostaje stały niezależnie od tego, gdzie dane są odczytywane. W związku z tym znacznik czasu będzie dostosowywany przez lokalną strefę czasową w celu dopasowania do pierwotnego punktu w czasie.

Rodzaj	Wartość w Ameryce / Los_Angeles	Wartość w Ameryce / New_York
sygnatura czasowa z lokalną strefą czasową	2014-12-12 12:34:56	2014-12-12 15:34:56
znak czasu	2014-12-12 12:34:56	2014-12-12 12:34:56

Hive – **Timestamp**:

https://cwiki.apache.org/confluence/display/Hive/LanguageManual+DDL

Porównania z innymi narzędziami:

	SQL 2003	Oracle	Sybase	Postgres	MySQL	Microsoft SQL	IBM DB2	Presto	Snowflake	Hive >= 3.1	Iceberg	Spark
timestamp	Local	Local	Local	Local	Instant	Other	Local	Local	Local	Local	Local	Instant
timestamp with local time zone		Instant							Instant	Instant		
timestamp with time zone	Offset	Offset	Offset	Instant			Offset	Offset	Offset		Instant	
timestamp without time zone	Local	Local		Local			Local					

Hive –:

https://cwiki.apache.org/confluence/display/Hive/LanguageManual+DDL

Case-insensitive

All Hive keywords are case-insensitive, including the names of Hive operators and functions.

Hive –:

Built In Functions

VIDE:

https://cwiki.apache.org/confluence/display/Hive/Tutorial#Tutorial-Custommap%2Freducescripts